



# 목차

엔터프라이즈 리눅스

# 목차

- 수세 리눅스 설명
- 레드햇 계열 설명
- 수세/레드햇 계열 비교
- 리눅스 설치(랩 준비)
- 네트워크
- 네트워크 보안
- 패키지 관리자
- 파일 시스템 비교
- 파일 시스템 고급



# 목차

- 표준 백업 솔루션
- LSB 사양
- 바이너리 및 라이브러리 호환
- 노드 클러스터링 마이그레이션
- systemd
- 가상화
- 콜 보안
- 커널



소개

강사

랩

# 과정

소개

# 소개

## 01

서브스크립션 비용  
절감 그리고, 좀 더  
오픈소스 친화적인  
운영이 필요한 인프라  
엔지니어

## 02

레드햇 계열 배포판  
이외 다른 배포판에  
대한 탐구

## 03

배포판 다각화를 통한  
운영에 운영성



# 강사

강사



# 강사

이름: 최국현

메일: tang@dustbox.kr, 회신은 다른 메일 주소로 드리고 있습니다. :)

사이트: tang.dustbox.kr

언제든지 질문 및 요청 환영 입니다.



# 랩 구성

리눅스 랩

# 오픈스택 랩 정보

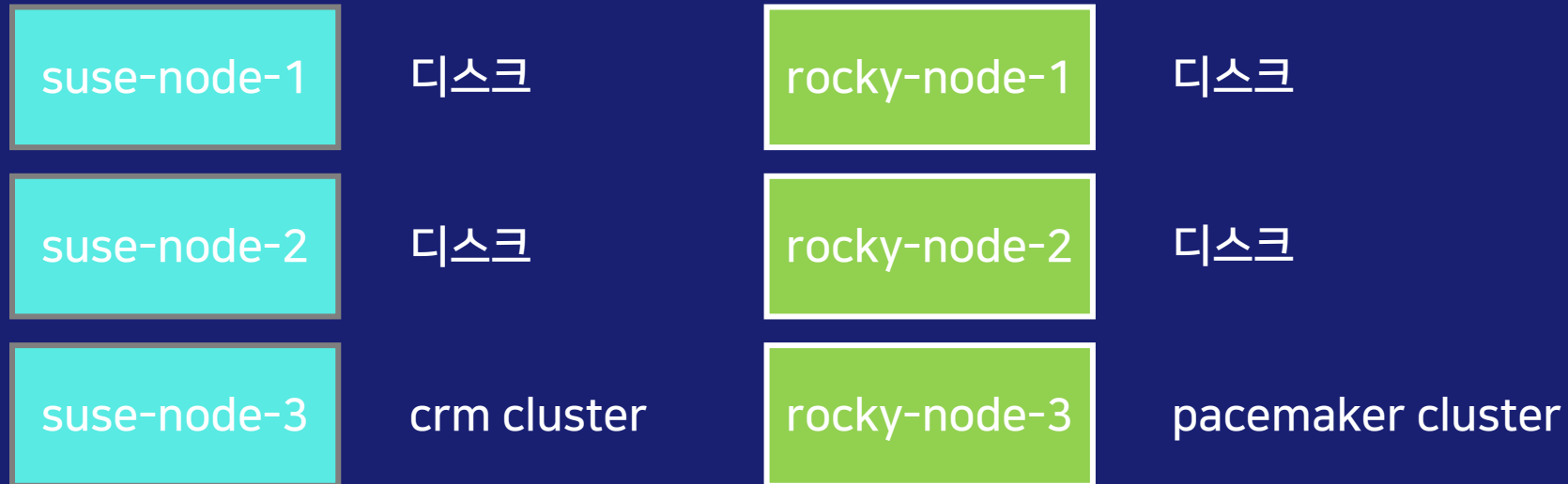
- 웹 주소: <https://vlab.dustbox.kr>
- 사용자 계정: user1~20
- 비밀번호: horizonlab
- 도메인: lab user

위 정보는 강의 진행 시 접근이 가능하며, 이후에는 접근이 불가능 합니다.



# 가상머신 구성

1. OpenSuSE leap 3대
2. Rocky Linux 3대



# 랩 정보

<https://vlab.dustbox.kr> 으로 접근 가능.

접근 시 사용 가능한 계정은 다음과 같음.

- 사용자 계정: user1~20
- 비밀번호: vlabhorizon
- 도메인: vlab-users



# 수세 리눅스 설명

SLES/OpenSuSE

# 수세 리눅스 소개

수세 역사는 다음과 같다.

## 연도      주요 사건

- 1992**    SUSE 설립 (독일 뉘른베르크에서 Thomas Fehr, Roland Dyroff, Burchard Steinbild, Hubert Mantel에 의해)
- 1994**    SUSE Linux 1.0 출시 (Slackware 기반)
- 1996**    독립적인 배포판으로 발전 (YaST 설치 도구 포함)
- 2001**    SUSE Linux 7.3 버전, 한글 및 아시아권 지원 강화
- 2003**    미국 소프트웨어 기업 Novell에 인수됨 (약 2억 1000만 달러)
- 2004**    SUSE Linux Professional 무료 공개 시작 (오픈소스 커뮤니티와 협력 강화)



# 수세 리눅스 소개

수세 역사는 다음과 같다.

## 연도      주요 사건

- 2005      OpenSUSE 프로젝트 출범, 커뮤니티 중심의 개발 모델 구축
- 2011      Attachmate Group에 의해 Novell과 함께 인수됨 (SUSE는 독립 사업부로 운영)
- 2014      Micro Focus가 Attachmate를 인수함으로써 SUSE도 함께 소속됨
- 2018      EQT Partners (스웨덴 사모펀드)에 의해 약 25억 달러에 인수, 다시 독립 회사로 분리
- 2019      SUSE CEO로 Melissa Di Donato 취임, 여성 CEO로는 최초





# 수세 리눅스 소개

## 연도      주요 사건

- 2021      프랑크푸르트 증권거래소에 상장 (상장 코드: SUSE.DE)
- 2022      SUSE Linux Enterprise 15 SP4 출시 (SAP, Rancher 등과 통합 강화)
- 2023      Rancher 2.7 및 NeuVector 통합 지속 (Kubernetes 보안 강화)
- 2024      SUSE Liberty Linux 출시 (RHEL 대응 제품, Rocky/Alma 경쟁)
- 2025 (예상) AI/Edge/Kubernetes 통합 플랫폼 강화 예정 (SLE Micro, Rancher Prime 등 집중)



# 수세 리눅스 소개

- 수세 리눅스는 첫번째 버전은 1994년도에 독일에서 처음 릴리즈
- 본래 CSSD라는 회사에서 개발 하였으며, 본래 배포판 기반은 Slackware리눅스 기반으로 작성됨
  - *Gesellschaft für Software und Systementwicklung mbH*
  - The name S.u.S.E. was an acronym for *Software- und System-Entwicklung*
  - <https://en.wikipedia.org/wiki/SUSE>
- SLS를 릴리즈 하면서 관리도구로 YaST 작성 및 배포
- 1996년도 부터 수세 리눅스 레드햇의 파일 시스템 구조 및 패키지 도구(RPM)를 적용함
- 1998년도에 SuSE라는 이름을 본격적으로 사용하기 시작하였고, 이때 리누즈 토발즈가 수세 리눅스 사용하면서 유명해졌으며 1999년도에 영국 시작에 진입.
- 수세 리눅스는 마이크로소프트사와 크로스 라이선스 체결
- 현재 **RKE2/K3S/Rancher** 및 **Harvester**를 통해서 컨테이너/가상화 시장에 적극적으로 접근중



# 수세 리눅스 템플릿 전환

수세 리눅스는 다음과 같은 단계로 개발 및 발전이 되었다.

1. Slackware(여전히 개발 진행중)
2. Jurix Linux(<https://en.wikipedia.org/wiki/Jurix>)
3. SuSE Linux(RPM, RHEL FHS)

초기 수세 리눅스는 슬랙웨어 기반으로 개발이 되었으며, 추후에 한번 **Jurix-Linux**으로 전환함. 마지막으로 레드햇 리눅스의 FHS 및 RPM를 받아들이면서 현재까지 계속 개발 및 유지가 되고 있다.



# YaST

수세 리눅스는 YaST는 다음과 같은 부분을 포함하고 있다. 현재 사용하는 YaST의 버전은 YaST2버전을 지원하고 있다. YaST가 처음으로 소개가 된 시점은 1996년도, 지금까지 계속 꾸준히 사용하고 있다. 레드햇은 이 시점에 각각 컴포넌트 별로 setup 및 XFree86기반의 GTK+프로그램으로 관리도구를 제공하였다. 이 때만 하여도 레드햇보다 수세 커뮤니티 및 소프트웨어 규모가 더 컸다.

<https://yast.opensuse.org/>

## 수세 리눅스 설치 도구(YaST2)

- GUI
- TUI



# YaST

## 시스템 설정 도구

- 소프트웨어
- 하드웨어 및 기타 기능

## 설치 자동화 도구 기능 지원

- AutoYaST

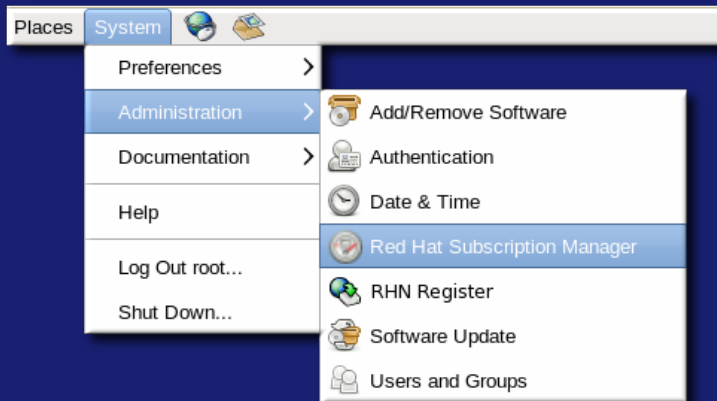
## 웹 기반 관리 도구

- WebYast, 여전히 지원은 하고 있지만 기본 구성으로 제공하지는 않는다.
- <https://webyast.github.io/webyast/>



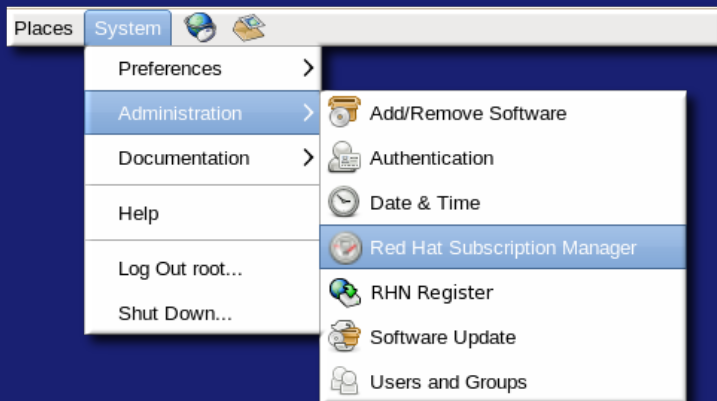
# Redhat System Tools

레드햇은 수세처럼 YaST는 지원하지 않으며, 기본적으로 TUI 및 GUI기반으로 프로그램을 지원하였다. 하지만 Redhat Linux 7이후로는 더 이상 **setup(TUI)**, **system-config(GUI)** 도구는 지원하지 않는다.

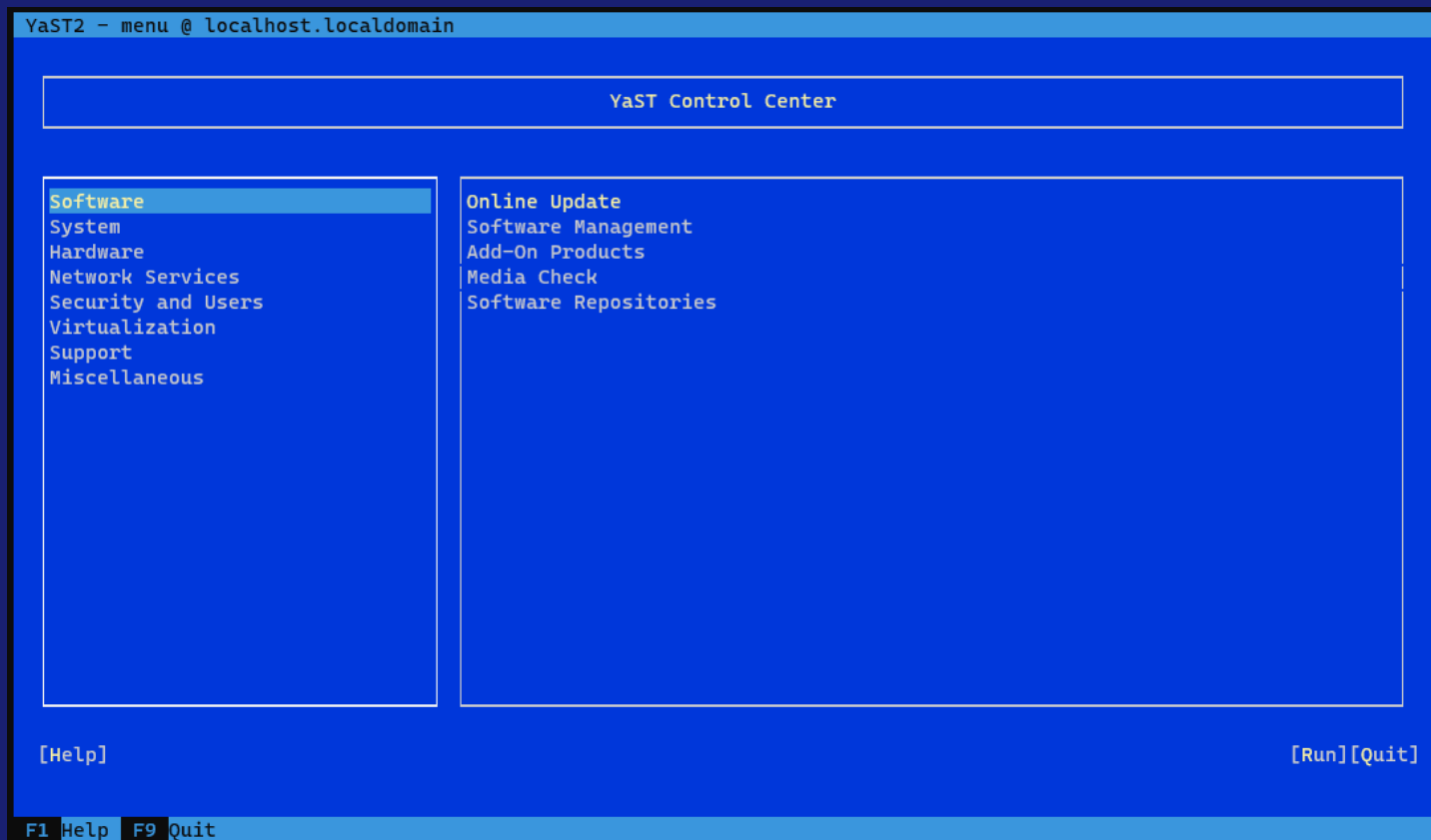


# Redhat System Tools

레드햇은 수세처럼 YaST는 지원하지 않으며, 기본적으로 TUI 및 GUI기반으로 프로그램을 지원하였다. 하지만 Redhat Linux 7이후로는 더 이상 **setup(TUI)**, **system-config(GUI)** 도구는 지원하지 않는다.

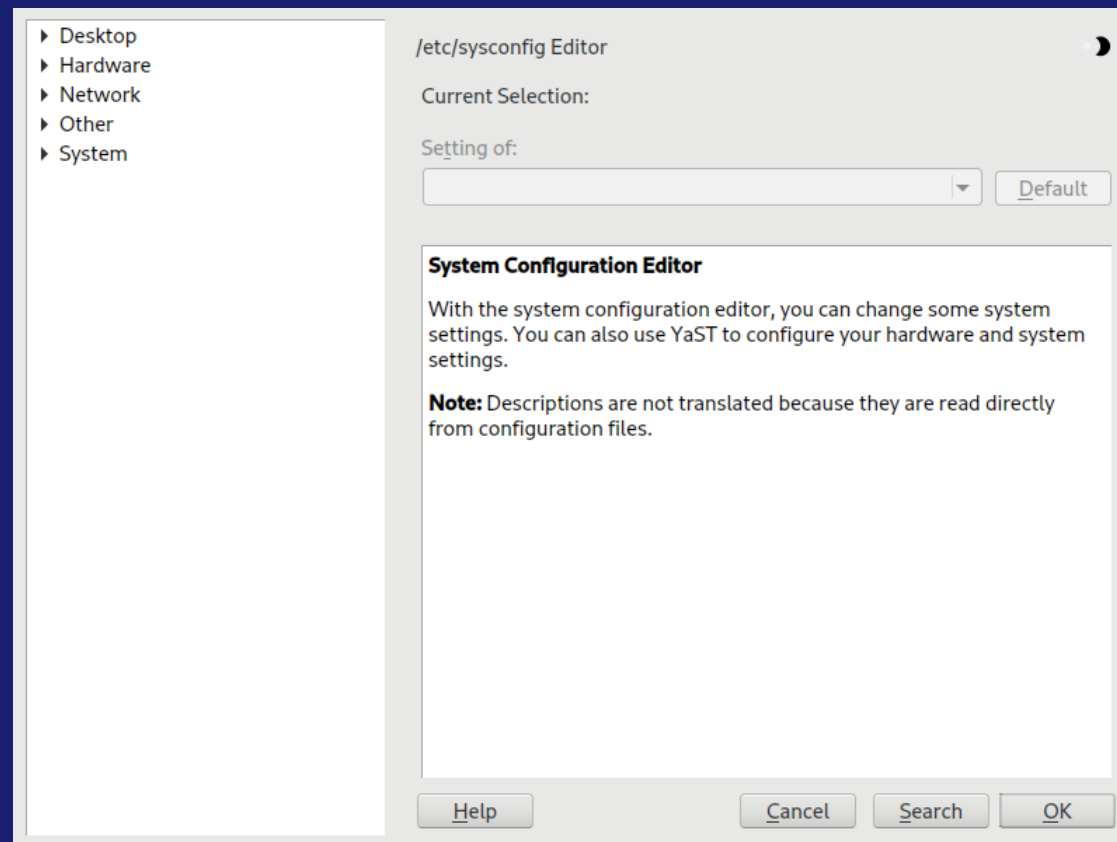


# YaST TUI





# YaST2 GUI



# OpenSuSE

오픈수세는 두 가지 버전으로 현재 존재한다. 추가적으로 오픈수세 이외 Factory/Micro가 있지만 이 버전 이 교육에서 다루지 않는다.

## openSuSE MicroOS

<https://get.opensuse.org/microos/>

## openSUSE Factory

<https://www.suse.com/suse-defines/definition/opensuse-factory/>



# OpenSUSE FACTORY

"The openSUSE Factory project is the development codebase that creates openSUSE Tumbleweed, which is the rolling-release version of the openSUSE Project. The Factory receives a constant flow of packages from developers working on the openSUSE distribution."

## Factory

수세 배포판에서 빠른 주기로 롤링 업데이트가 이루어지며, 기여자 코드 기반으로 업데이트가 발생한다. 플랫폼 개발자나 기여자에게 적절한 배포판.

## Tumbleweed

Factory에 최대한 가까운 안정적인 플랫폼(배포판)에서 사용을 원하는 경우. 일반적으로 오픈수세 기여자가 많이 사용한다. Tumbleweed는 Factory비슷하게 커널 업데이트가 자주 된다.

## Leap

커널 컴파일이나 혹은 모듈 컴파일이 필요하지 않거나, 안정적인 버전(업데이트 덜)을 원하는 경우, Leap사용을 권장.



# OpenSUSE MicroOS

"Designed to host container workloads with automated administration & patching. Installing openSUSE MicroOS you get a quick, small environment for deploying Containers, or any other workload that benefits from Transactional Updates. As rolling release distribution the software is always up-to-date."

스냅샷 기반으로 구성되는 **Aeon(MicroOS)**이다.

기본적으로 롤링 업데이트 기반의 시스템이며, 컨테이너 시스템에 적절하다. 레드햇은 이를 **CoreOS**라는 이름으로 제공하며 개념은 비슷하다. 레드햇은 XFS에서 COW환경 및 스냅샷을 제공하기 어렵다. 이러한 이유로 **OS-TREE**기반으로 파일 시스템 및 RPM 레이어를 구성한다.



# OpenSuSE MicroOS

수세는 BTRFS를 사용하기 때문에 별도로 복잡하게 OS-TREE같은 환경을 구성하지 않아도 된다. 자세한 내용은 아래 링크에서 확인이 가능하다.

SUSE AeonOS 공식 설명

<https://documentation.suse.com/aeon/>

Red Hat rpm-ostree 설명

<https://coreos.github.io/rpm-ostree/>



# OpenSuSE

## Tumbleweed

덤블위드는 weed라는 단어 때문에 대마초가 생각날 수 있지만, 실제로는 사막에서 굴러다니는 덩굴을 보통 이야기한다. 덤블위드는 모양 그대로(?) 롤링 업데이트(rolling-update)기반으로 지원 및 제공한다. 서버 및 데스크탑에서 사용하는 최신 기술을 느끼고 싶으면, 덤블위드 사용을 권장한다. 레드햇에서는 이를 페도라 코어로 지원하고 있다.

## Leap

리프(leap)는 "뛰다"라는 뜻을 가지고 있다. 리프 버전은 SLE(SuSE Linux Enterprise)기반의 소스코드를 사용한다. 3년 동안 업데이트를 지원하며, 이 후에는 다음 버전으로 넘어간다. 리프 경우에는 덤블위드에서 사용하기도 하며, SLE와 함께 코드를 공유하기도 한다.



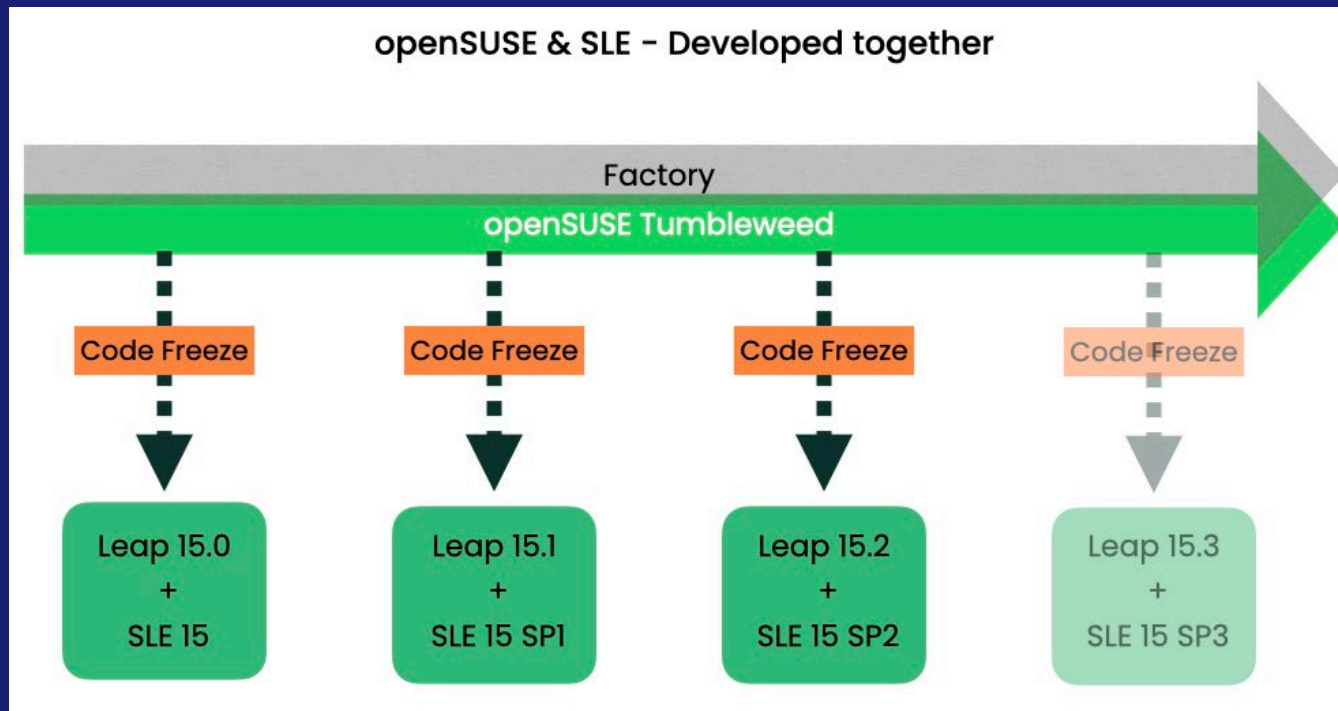
# Tumbleweed



# Leap

리프 버전은 SLE(SuSE Linux Enterprise)기반의 소스코드를 사용한다.

3년 동안 업데이트를 지원하며, 이 후에는 다음 버전으로 넘어간다. 리프 경우에는 덤블위드에서 사용하기도 하며, SLE와 함께 코드를 공유하기도 한다.





# Leap



# SLES

수세 리눅스 기업용 버전. 레드햇에서 **RHEL**, 수세에서는 **SLES**라고 부른다.

약자의 뜻은 **SuSE Linux Enterprise Server**. **SLES**는 배포판에서 사용하는 모든 패키지에 대해서 안전성을 우선으로 하고 있다.

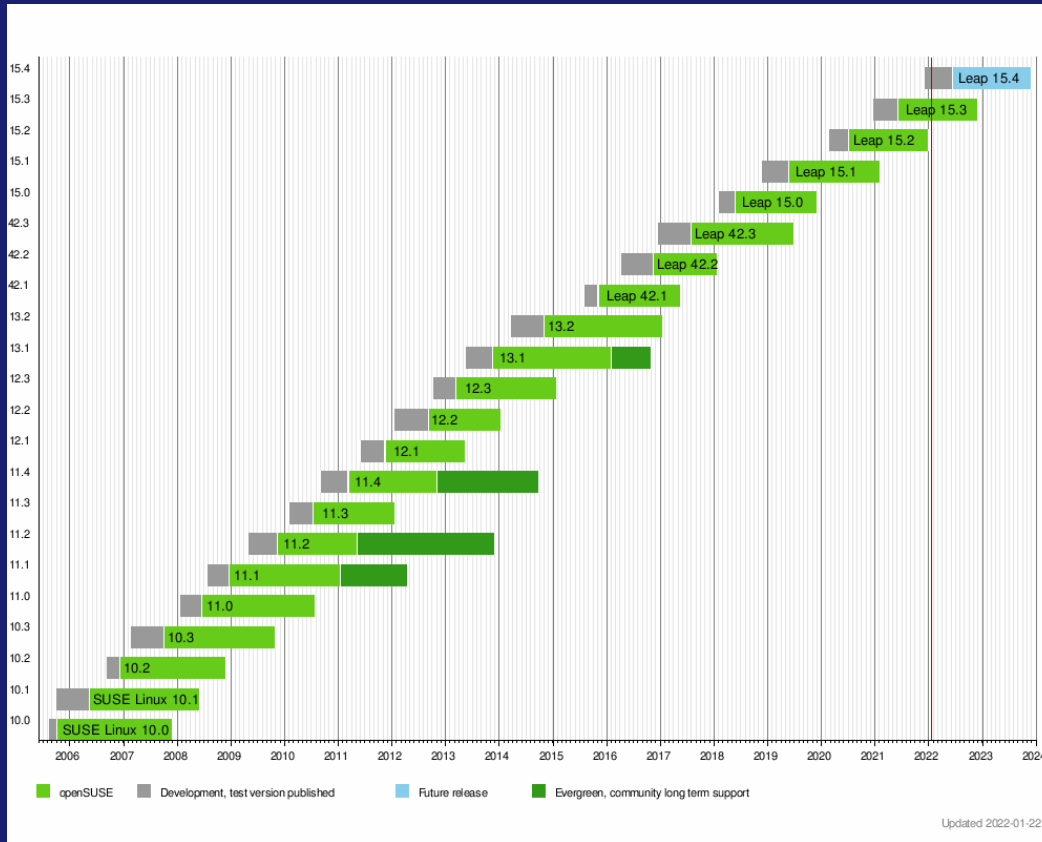
SLES는 leap에서 영향을 받으며 Tumbleweed에서 사용하는 몇몇 최신 기술들을 채용하기도 한다.

오픈수세 라이프 타임은 아래 링크에서 확인이 실시간으로 가능하다.

<https://en.opensuse.org/Lifetime>



# OpenSuSE RELEASE LINE



# 레드햇 계열 설명

CentOS, CentOS Stream 그리고 Rocky

# CentOS Stream 8/9

기존 CentOS은 레드햇이 IBM으로 인수 및 합병이 되면서 CentOS 7이후로 더 이상 개발이 진행이 되지 않는다. CentOS를 RHEL 1:1 클론 버전 성격이 강했지만, Stream버전으로 변경이 되면서 더 이상 1:1클론 버전 방식을 따르지 않는다. CentOS-8-Stream으로 지원 및 릴리즈 한다. 참고로 CentOS 8과 CentOS-8-Stream은 다른 버전이다.

- 릴리즈 기간은 각 스트림 릴리즈 별 3년 지원
- 버전 고정이 불가능한 롤링 업데이트 방식 지원
- 공식적으로는 CentOS-Stream에서 RHEL으로 업그레이드 완전히 지원하지는 않음(추후 더 이야기)



# CentOS Stream 8/9

위의 내용들은 아래 링크에서 확인이 가능하다.

1. <https://access.redhat.com/articles/5941531#can-centos-stream-8-be-upgraded-to-centos-stream-9-or-rhel-9-9>
2. <https://www.redhat.com/en/technologies/linux-platforms/enterprise-linux/migration-process/convert2rhel-how-to-convert-from-centos-linux-to-red-hat-enterprise-linux>
3. <https://www.centos.org/cl-vs-cs/>



# RHEL 10/ROCKY 10/ALMA 10



# CentOS Stream

**CentOS-Stream** 버전 경우에는 기본적으로 RHEL로 업그레이드가 어렵다. 불가능하지는 않는다. 다만, 공식적으로 지원하는 버전은 CentOS 7/8버전으로 되어 있다. Stream버전은 Up-Stream으로써 역할만 하기 때문에, 마이너 버전이 릴리즈가 시작이 되면, 업데이트 방식으로 패키지 지원을 한다.

■ **CentOS Stream, on the other hand, is the upstream, public development branch for RHEL. Specifically, CentOS Stream 8 is the upstream for the next minor release of RHEL 8, CentOS Stream 9 for the next minor release of RHEL 9, and so on.** ■

또한, RHEL과 비교하면 패키지 업데이트 속도가 다운 스트림 버전인 RHEL보다 자주 발생하기 때문에 프로덕트 운영으로는 적합하지 않는다.





# OpenELA

Open Enterprise Linux Association (OpenELA)는 2023년 8월, CIQ(Rocky Linux), SUSE, Oracle이 공동으로 설립한 비영리 협회.

Red Hat이 RHEL 소스 코드 접근을 제한한 것에 대응하여, RHEL과 호환되는 오픈소스 기반의 Enterprise Linux 배포판을 위한 소스 코드와 도구를 제공하기 위해 만들어진 단체.

## CIQ (Rocky Linux)

- OpenELA의 주도적 창립 멤버로서, RHEL과 1:1 호환되는 Rocky Linux를 개발.
- OpenELA를 통해 RHEL 8, 9, 7 버전에 대한 소스 코드 제공을 주도.

## SUSE

- 적극적인 참여자로서, RHEL 호환 배포판인 Liberty Linux를 통해 OpenELA의 소스 코드 활용.
- "우리는 선택의 자유를 믿는다"는 철학 아래, OpenELA를 통해 오픈소스 커뮤니티와의 협업을 강조.



# OpenELA

## Oracle

OpenELA의 창립 멤버로 참여하고 있으나, 상대적으로 낮은 참여율.

Oracle Linux는 자체적으로 RHEL과 호환되는 배포판을 유지하고 있으며, OpenELA를 통해 소스 코드를 제공하고 있지만, 커뮤니티 활동이나 기술적 기여는 제한적.



# OpenELA

## NAVER와 OpenELA의 관계

NAVIX, 네이버 클라우드에서 개발한 RHEL 호환 리눅스 배포판으로, OpenELA의 소스 코드를 기반으로 제작.

## 사용 현황

NAVIX는 네이버 인프라의 약 15%를 구동하고 있으며, 향후 네이버 클라우드의 VM 옵션으로도 제공예정.

## 지원 정책

NAVIX는 10년간의 지원을 약속하고 있으며, 초기 5년은 전체 업데이트, 이후 5년은 보안 및 중요 업데이트를 제공.

## 공식 언급

OpenELA 측에서도 NAVIX의 출시를 환영하며, OpenELA 기반의 배포판으로 소개.



# OpenELA

| 기업          | 공식 회원 여부 | 참여도   | 배포판 이름        | 비고                            |
|-------------|----------|-------|---------------|-------------------------------|
| CIQ (Rocky) | 창립 회원    | 매우 높음 | Rocky Linux   | OpenELA 주도, 소스 코드 배포 핵심 역할    |
| SUSE        | 창립 회원    | 높음    | Liberty Linux | 기술적 기여 활발, 오픈소스 자유 강조         |
| Oracle      | 창립 회원    | 낮음    | Oracle Linux  | 자체 배포판 유지, 커뮤니티 참여는 소극적       |
| NAVER       | 비회원      | 실질 기여 | NAVIX         | OpenELA 기반 사용, RHEL 호환 리눅스 구축 |



# OpenELA/CIQ

| 구분     | OpenELA                                | CIQ / Rocky Linux                 |
|--------|----------------------------------------|-----------------------------------|
| 목적     | RHEL 재구성용 공개 소스의 자유 접근 보장              | RHEL 호환 배포판(Rocky)의 기업 지원 및 서비스   |
| 활동     | RHEL 관련 SRPM 제공 및 커뮤니티 규약 정립           | 배포판 관리, 보안 패치 제공, LTS, 유료 서포트 제공  |
| 공동체 관계 | 다양한 RHEL 클론들과 협업 (Alma, Rocky, Oracle) | Rocky Linux 중심 운영 (OpenELA에 참여 중) |
| 법적 구조  | 비영리 재단 (재단법인)                          | 영리 회사 (CIQ Inc.)                  |



# OpenELA/CIQ

| 구분        | Red Hat         | SUSE              | OpenELA         | CIQ / Rocky Linux     |
|-----------|-----------------|-------------------|-----------------|-----------------------|
| 소스 공개 방식  | 제한적             | 완전 공개 (OBS 기반)    | 커뮤니티 중심 공개      | OpenELA 소스 기반으로 빌드    |
| GPL 준수 태도 | 형식적 준수, 실제로 제한  | 철저한 공개와 공유        | GPL 철학 수호의 선두주자 | GPL 기반 Rocky 유지 및 상용화 |
| 기여 모델     | 내부 중심, 외부 기여 적음 | GitLab/OBS 기반 풀오픈 | 커뮤니티 중심 (초기 단계) | GitHub 기반, 커뮤니티 PR 환영 |
| 목적        | 상업 제품 보호        | 오픈소스 강화 및 생태계 확대  | RHEL 소스의 자유 보장  | RHEL 호환 배포판 + 기업 지원   |



# 수세/레드햇 계열 비교

OpenSUSE

Redhat

# 네트워크

Network Manager

Wicked



# NETWORK

수세 리눅스는 2가지 형식으로 네트워크 설정을 지원하고 있다.

- `wicked`
- `LSB network script`
- `NetworkManager`

위 두가지로 네트워크 구성 및 설정이 가능하다. 기본적으로 `wicked`기반으로 네트워크 설정을 권장하고 있다. 또한, 네트워크 설정이 복잡하다고 느껴지는 경우 YaST2도구를 통해서 네트워크 설정이 가능하다.

레드햇 리눅스에서 사용하던 `NetworkManager`도 사용이 가능하지만, 데스크탑 및 워크스테이션용으로 사용 가능. 기업 서버 용도로 사용은 가능하나, 적극적으로 사용을 권장하지는 않음.

`Wicked`은 `wicked`명령어를 통해서 네트워크 설정 및 구성이 가능하다.

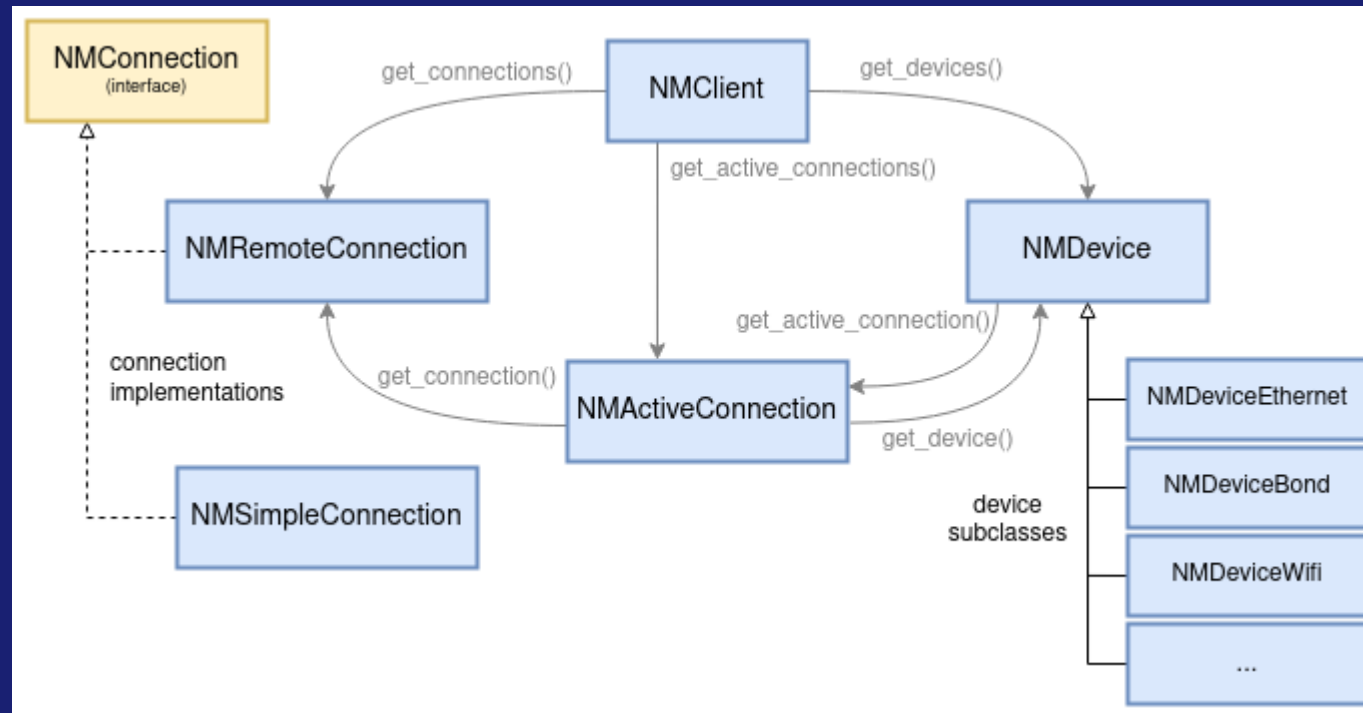


# NETWORK

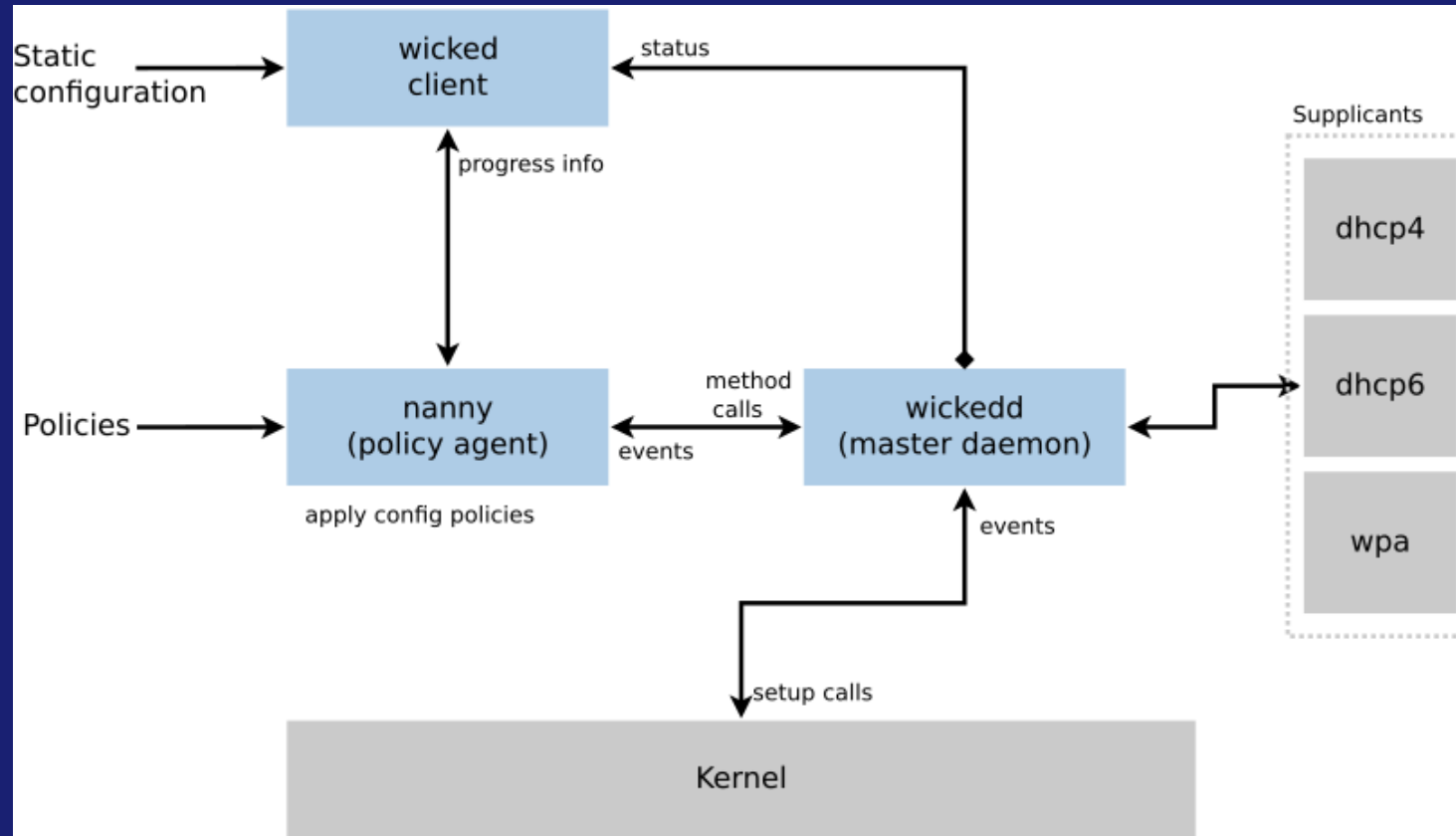
| 제품/버전          | 최신 버전      | 기본 네트워크 매니저       | NetworkManager | 비고                                  |
|----------------|------------|-------------------|----------------|-------------------------------------|
| SLES (Server)  | 15 SP6     | wicked            | 기본은 미지원        | Workstation Extension 설치 시 일부 사용 가능 |
| SLED (Desktop) | 15 SP5     | NetworkManager    | 지원             | GNOME 등 데스크탑 환경에서 기본 사용             |
| openSUSE Leap  | 15.5       | wicked + NM 선택 가능 | 선택 가능          | 설치 시 선택, GUI 중심 사용 가능               |
| openSUSE Aeon  | Rolling 기반 | NetworkManager    | 지원             | 컨테이너/Immutable 시스템 최적화              |



# NetworkManager



# wicked



# NM VS Wicked AND systemd-networkd

네트워크 매니저와 위키드(wicked)은 기본적으로 동일한 기능을 가지고 있다. 다만, 동작 방식에 다음과 같은 차이가 있다.

## NetworkManager

- 유동적인 장치 설정에 좀 더 초점이 맞추어 있다. 최근 버전에서는 더 이상 스크립트 생성을 지원하지 않는다.
- 커넥션 기반으로 하나의 장치에 여러 프로파일 기반으로 설정을 다양하게 사용 및 구성이 가능하다.

## wicked

- 정적인 네트워크 인터페이스 설정을 제공한다.
- 이전 LSB Network Script를 지원한다.
- LSB Network Script와 동일하게 1:1형태로 설정 및 구성을 지원한다.



# NM VS Wicked AND systemd-networkd

## systemd-networkd

**systemd**는 기존에 독립적으로 관리하던 네트워크 설정을 **systemd-networkd**기반으로 통합하고 있다. 위의 **NetworkManager**, **wicked**로 구성이 필요없다.

단일 관리자로 구성하고 싶으면, systemd기반으로 권장한다. 앞으로 네트워크 설정은 systemd기반으로 통합될 예정이다.

<https://www.freedesktop.org/software/systemd/man/latest/systemd.network.html>



# Wicked

Wicked에서 Nanny라는 서비스가 있다. 이 서비스는 wicked에 정책을 관리하는 프로세스이다. 이 정책이 다루는 부분은 wicked에서 다루는 인터페이스 설정 부분이다.

<https://github.com/openSUSE/wicked/blob/master/nanny/nanny.c>

Wicked에서 자원을 다루는 명령어는 **wicked**이다. wicked는 레드햇 계열 배포판에서는 지원하지 않는다.



# Wicked/NetworkManager(NM)

현재 수세 리눅스는 wicked기반으로 사용하고 있다.

다만, 앞으로 wicked에서 NetworkManager로 이동을 준비하고 있다. 둘 다 dbus 및 systemd를 지원하고 있다.

하지만, freedesktop.org에서는 NetworkManager 및 systemd-networkd를 지원하고 있기 때문에 앞으로 모든 리눅스 배포판은 위 두개로 통합 및 운영이 될 예정이다.

## **systemd-network**

<https://www.freedesktop.org/software/systemd/man/latest/systemd.network.html>

## **Wicked to NetworkManager**

<https://kubic.opensuse.org/blog/2022-05-05-NetworkManager-Wicked/>





# NetworkManager(NM)

현재 네트워크 매니저를 사용하고 있는 수세 배포판은 다음과 같다.

- OpenSuSE Tumbleweed
- Micro OS Desktop
- OpenSuSE Kubic(retired)

**Leap, Factory**와 마찬가지로 SLES에서 똑같이 Wicked를 채용하고 있다. 배포판 마다 다르게 네트워크 관리자 도구를 사용하고 있으니 구성 시 미리 릴리즈 노트 및 포럼 확인이 필요하다.

네트워크 매니저는 커넥션 기반으로 자원을 관리 및 운영한다. **wicked** 및 **NetworkManager**는 dbus와 통합이 되어있다. 네트워크 매니저의 제일 큰 장점은 손쉽게 디바이스에 네트워크 설정을 프로파일 기반으로 여러 개를 구성하여 사용이 가능하다. NM에서 자원을 다루는 명령어는 **nmcli**이다.



# NetworkManager(NM)

네트워크 매니저는 더 이상 **ifcfg-rh**와 같은 형식은 지원하지 않는다. 아래 명령어 혹은 설정 파일에서 확인이 가능하다.

```
NetworkManager --print-config  
cat /etc/NetworkManager/NetworkManager.conf  
[main]  
plugins=keyfile,ifcfg-rh
```

설정된 내용들은 기본적으로 `/etc/NetworkManager/system-connections`에 저장이 된다.

```
ls /etc/NetworkManager/system-connections  
eth0.nmconnection  eth2.nmconnection
```



# NetworkManager(NM)

== ifcfg format ==

```
TYPE=Ethernet
DEVICE=enp1s0
UUID=0c8afb07-fe80-47bc-b61d-09ee85eefd6e

IPADDR=10.0.0.0.2
NETMASK=255.255.255.0
GATEWAY=10.0.0.1
PREFIX=24
NAME="System enp1s0"
DNS1=10.0.0.3
ONBOOT=yes
DEFROUTE=yes
IPV4_FAILURE_FATAL=no
```

== keyfile format ==

```
[connection]
type=ethernet
interface-name=enp1s0
uuid=0c8afb07-fe80-47bc-b61d-09ee85eefd6e
[ipv4]
address1=10.0.0.2/24,10.0.0.1

id=enp1s0
dns=10.0.0.3;

autoconnect-retries=1
dns-search=example.org
may-fail=false
method=manual
```



# bond/teamd

레드햇 리눅스는 2 가지 형태의 네트워크 본딩을 제공하고 있다.

- bond
- teamd(현재 시장에서 사라지는 중, 안전성 이유)

수세 리눅스는 레드햇과 동일하게 2가지 형태로 네트워크 본딩을 제공하고 있다.

- bond
- teamd(현재 시장에서 사라지는 중, 안전성 이유)



# bond/teamd

위의 두 배포판은 동일한 bond device를 구성하기 때문에 비교할 부분이 없다.

- teamd대몬 기반은 사용하는 설정 도구가 다르기 때문에, 구성 방법이 조금 다르다.
- 수세 리눅스는 wicked기반으로 제공하고 있으며, 레드햇 경우에는 NM기반으로 제공하고 있다.
- 두 네트워크 관리자는 bond가 기본으로 사용 및 제공한다.

위의 두 개의 기술은 다음과 같은 내용으로 동작의 차이가 있다. 하지만, teamd 데몬은 기본적으로 **bond device wrapped**형태로 구성이 되어있다. 프로그램의 안전성이 걱정이 되면, 아래 링크에 방문해서 확인!

<https://github.com/jpirko/libteam/blob/master/libteam/libteam.c>



# NetworkManager

네트워크 매니저는 다음과 **LSB-ifcfg** 스크립트는 더 이상 지원하지 않는다.

1. ifcfg-rh

2. ifcfg-wicked-cfg

ifcfg-wicked-cfg는 ifcfg-rh와 다른 문법을 가지고 있다.

지시자를 추가하지 않으면 올바르게 네트워크 매니저에서 마이그레이션 및 인식이 되지 않는다. 네트워크 매니저가 사용하는 ifcfg-rh플러그인에 대한 설명은 아래 링크에서 확인이 가능하다.

<https://people.freedesktop.org/~lkundrak/nm-docs/nm-settings-ifcfg-rh.html>



# NetworkManager IN SuSE

설치는 다음과 같이 가능하다.

```
# zypper install NetworkManager
# systemctl stop wicked
# systemctl disable wicked
# systemctl mask wicked
# systemctl enable --now NetworkManager
```



# NetworkManager CMD

기존에 사용하던 스크립트를 커넥션으로 전환하려면 아래 명령어로 가능하다. 단, ifcfg-rh형태만 되며, 수세 리눅스 경우에는 네트워크 매니저에서 ifcfg-rh는 플러그인이 비 활성화 되어있다. 이 기능은 RHEL 9 후기 버전부터는 더 이상 활성화가 되지 않는다.

```
nmcli connection reload
```

```
nmcli connection migrate --plugin ifcfg-rh <PATH>
```

```
NetworkManager --print-config
```

```
plugins=keyfile          ## SuSE
```

```
# plugins=keyfile, ifcfg-rh  ## Redhat
```





# SUSE TO RH/RH TO SUSE

ifcfg-suse, ifcfg-rh는 다음과 같이 다르다.

```
ifcfg-suse(for wicked)
```

```
BOOTPROTO='dhcp'
```

```
STARTMODE='auto'
```

```
ZONE=public
```

```
ifcfg-rh(for network)
```

```
DEVICE=em1
```

```
BOOTPROTO=dhcp
```

```
ONBOOT=yes
```



# SUSE TO RH/RH TO SUSE 전환

네트워크 매니저 기반으로 사용하는 경우 아래의 위치에서 프로파일을 복사 후 커넥션 불러오기 작업을 수행한다.  
양쪽 다 네트워크 매니저를 사용하면 손쉽게 마이그레이션 가능하다.

```
suse/redhat]# cd /etc/NetworkManager/system-connections
suse/redhat]# scp *.nmconnection
root@<SUSE_NODE>:/etc/NetworkManager/system-connection/
suse/redhat]# nmcli con reload
```



# NetworkManager Bond 구성 방법

```
nmcli connection add type bond con-name bond0 ifname bond0 bond.options  
"mode=active-backup"
```

```
nmcli connection add type bond con-name bond0 ifname bond0 bond.options  
"mode=active-backup,miimon=1000"
```

```
nmcli connection add type ethernet slave-type bond con-name bond0-port1  
ifname eth1 master bond0
```

```
nmcli connection add type ethernet slave-type bond con-name bond0-port2  
ifname eth2 master bond0
```

```
nmcli connection up bond0
```



# NetworkManager Bond IFCFG -> INI

```
nmcli device
```

```
nmcli con migrate --plugin ifcfg-rh /tmp/ifcfg-bond0
```

```
nmcli con migrate --plugin ifcfg-rh /tmp/ifcfg-bond0-port1
```

```
nmcli con migrate --plugin ifcfg-rh /tmp/ifcfg-bond0-port2
```



# NetworkManager Teamd 구성 방법

```
nmcli connection add type team con-name team0 ifname team0 team.runner  
activebackup
```

```
nmcli connection modify team0 team.link-watchers "name=ethtool"
```

```
nmcli connection add type ethernet slave-type team con-name team0-port1  
ifname eth1 master team0
```

```
nmcli connection add type ethernet slave-type team con-name team0-port2  
ifname eth2 master team0
```

```
nmcli connection up team0
```

```
teamdctl team0 state
```



# NetworkManager VLAN 구성 방법

```
nmcli connection add type vlan con-name vlan10 ifname vlan10 vlan.parent  
enp1s0 vlan.id 10
```

```
nmcli connection modify vlan10 ethernet.mtu 2000
```

```
nmcli connection modify vlan10 ipv4.addresses '192.0.2.1/24' ipv4.gateway  
'192.0.2.254' ipv4.dns '192.0.2.253' ipv4.method manual
```

```
nmcli connection modify vlan10 ipv6.method disabled
```

```
nmcli connection up vlan10
```

```
ip -d addr show vlan10
```



# wicked command 명령어

```
cat /etc/wicked/server.xml
```

```
wicked server process
```

```
cat /etc/wicked/client.xml
```

```
wicked command
```

```
cat /etc/wicked/nanny.xml
```

```
wicked nanny server
```

```
wicked ifup eth1
```

```
wicked ifdown eth1
```

```
wicked ifreload eth1
```

```
wicked ifstatus all
```



# Wicked/NM TUI

Wicked TUI, NetworkManager는 전용 TUI도구가 있음. Wicked은 명령어 기반으로 이더넷 장치 생성이 어려움.

- Wicked ifcfg-wicked 기반으로 구성

NetworkManager는 명령어 기반으로 이더넷 및 다수 장치 생성이 쉬움.

- NetworkManager 서비스가 INI으로 프로파일 구성

```
nmtui edit <PROFILE_NAME>
```

```
nmtui
```

```
yast network
```





# teamd/bonding 기능 비교 1

| 기능                      | 본딩  | 티밍  |
|-------------------------|-----|-----|
| TX 라운드 로빈 브로드 캐스팅 정책 지원 | yes | yes |
| TX 액티브 백업 정책            | yes | yes |
| LACP(802.3ad) 지원        | yes | yes |
| 해시(hash)기반의 TX정책        | yes | yes |
| 사용자 정의 hash 기능          | no  | yes |
| TLB(TX) 로드 밸런싱 지원       | yes | yes |
| LACP를 위한 TX 로드 밸런싱 지원   | no  | yes |
| ethtool 기반으로 링크 모니터링    | yes | yes |
| ARP 링크 모니터링             | yes | yes |
| IPv6(NS/NA) 모니터링 링크     | no  | yes |



## teamd/bonding 기능 비교 2

|                          |     |     |
|--------------------------|-----|-----|
| RCU의 TX/RX 경로 락킹         | no  | yes |
| 포트 우선순위 및 선호도            | no  | yes |
| 각 포트에 대한 모니터링 설정         | no  | yes |
| 멀티 링크 모니터링 설정            | 제한적 | yes |
| VLAN 지원                  | yes | yes |
| multiple device stacking | yes | yes |

<http://libteam.org/files/teamdev.pp.pdf>



# Wicked-teamd 설명

wicked에서 teamd장치 혹은 기타 네트워크 장치를 생성하기 위해서 아래 명령어로 `ifcfg-suse`정보 확인이 가능하다. 혹은 수세 웹 사이트에서 확인한다.

1. `man ifcfg`
2. `man ifcfg-teamd`
3. `/etc/sysconfig/network/ifcfg.template`



# Wicked-teamd(activebackup)

teamd를 wicked에서 설정 시, 다음과 같이 한다.

```
# vi /etc/sysconfig/network/ifcfg-team0
STARTMODE=auto
BOOTPROTO=static
IPADDR="192.168.1.2/24"
IPADDR6="fd00:deca:fbad:50::2/64"
TEAM_RUNNER=activebackup
TEAM_PORT_DEVICE_0="eth1"
TEAM_PORT_DEVICE_1="eth2"
TEAM_LW_NAME=ethtool
TEAM_LW_ETHTOOL_DELAY_UP="10"
TEAM_LW_ETHTOOL_DELAY_DOWN="10"
```



# Wicked-teamd(activebackup) 설치 및 구성

설치 및 구성은 다음과 같이 한다.

```
zypper install libteam-tools
systemctl stop NetworkManager
systemctl start wicked
wicked ifup team0
wicked ifreload all
wicked show-config
teamdctl team0 state
systemctl status teamd@team0
```



# Wicked-LinuxBridge 구성 1

리눅스 브릿지 구성은 다음과 같이 한다.

```
# vi /etc/sysconfig/network/ifcfg-br0
STARTMODE='auto'
BOOTPROTO='static'
IPADDR="192.168.2.2/24"
BRIDGE='yes'
BRIDGE_STP='off'
BRIDGE_FORWARDDELAY='0'
BRIDGE_PORTS='eth3'
ZONE=public
```



# Wicked-LinuxBridge 구성 2

앞에 내용 계속 이어서...

```
zypper install bridge-utils  
systemctl stop NetworkManager  
systemctl start wicked  
wicked ifup all br0  
wicked show-config br0  
brctl show  
bridge link
```



# Wicked-vlan 구성

vlan 구성은 아래와 같이 한다.

```
vi /etc/sysconfig/network/ifcfg-vlan0
STARTMODE="auto"
BOOTPROTO="static"
IPADDR='192.168.3.1/24'
ETHERDEVICE="eth4"
VLAN_ID="100"
VLAN='yes'
```





# 브릿지 인터페이스 적용

반영 및 적용 후, 아래와 같이 명령어를 실행한다.

```
systemctl stop NetworkManager  
systemctl start wicked  
wicked ifup all br0  
wicked show-config br0  
bridge vlan
```



# 수세 본딩 인터페이스 1

수세 리눅스에서 본딩 인터페이스 구성 시, 아래와 같이 진행한다.

```
vi ifcfg-bond0
STARTMODE='onboot'
BOOTPROTO='static'
IPADDR='192.168.0.1/24'
BONDING_MASTER='yes'
BONDING_SLAVE_0='eth0'
BONDING_SLAVE_1='eth1'
BONDING_MODULE_OPTS='mode=active-backup miimon=100'
```



# 수세 본딩 인터페이스 2

앞에 내용 계속 이어서...

```
vi ifcfg-eth0
STARTMODE='hotplug'
BOOTPROTO='none'
#ETHTOOL_OPTIONS='wol g'
```



# 수세 본딩 인터페이스 3

앞에 내용 계속 이어서...

```
vi ifcfg-eth1
STARTMODE='hotplug'
BOOTPROTO='none'
#EHTOOL_OPTIONS='wol g'
```



# 추가 정보

수세 리눅스의 네트워크 관련 자세한 정보는 아래 링크에서 확인이 가능하다.

- <https://doc.opensuse.org/documentation/leap/reference/html/book-reference/cha-nm.html>
- [https://documentation.suse.com/external-tree/en-us/sles/12-SP4/networking\\_with\\_wicked\\_in\\_suse\\_linux\\_enterprise\\_12\\_guide.pdf](https://documentation.suse.com/external-tree/en-us/sles/12-SP4/networking_with_wicked_in_suse_linux_enterprise_12_guide.pdf)



# 네트워크 보안

iptables/nftables

firewalld

# 네트워크 보안 변천사

네트워크 라우팅 및 필터링 도구는 다음과 같은 순서로 개발 및 발전이 되었다. 현재 대다수 리눅스 배포판은 **nftables**로 전환이 완료되었다.

| 이름       | 등장 시기                | 계층    | 주요 역할                      | 상태      | 비고               |
|----------|----------------------|-------|----------------------------|---------|------------------|
| ipfw     | 1993년<br>(FreeBSD)   | L3/IP | 초기 BSD 계열 방화벽 도구           | 비표준     | Linux 표준 아님      |
| ipchains | 1998년<br>(Linux 2.2) | L3/IP | ipfw 이후, Linux 최초 방화벽      | 폐기됨     | 상태 기반 추적 미지원     |
| iptables | 2000년<br>(Linux 2.4) | L3/IP | Connection tracking, NAT 등 | 주요 사용 중 | 가장 오래 사용됨, 널리 퍼짐 |



# 네트워크 보안 변천사

위의 내용 계속 이어서...

| 이름        | 등장 시기                 | 계층          | 주요 역할                | 상태     | 비고              |
|-----------|-----------------------|-------------|----------------------|--------|-----------------|
| arptables | 2002년                 | L2/ARP      | ARP 요청 제어 (스푸핑 방지 등) | 제한적 사용 | IPv4 ARP에만 적용   |
| ebtables  | 2002년                 | L2/Ethernet | 브리지 계층에서 프레임 필터링     | 제한적 사용 | 브리지 환경에 특화됨     |
| nftables  | 2014년<br>(Linux 3.13) | 통합(L2~L4)   | 기존 방화벽 도구 대체         | 차세대 표준 | 단일 프레임워크로 통합 구조 |





# 네트워크 필터

수세/레드햇 리눅스는 다음과 같은 네트워크 보안 도구를 공통적으로 제공한다.

1. iptables(ebttables)

2. nftables

구 버전(수세 및 레드햇 공통)

## iptables

이전 netfilter에서 제공하던 방화벽 도구. 이 도구의 최대 단점은 많은 개수의 커넥션 및 동시 작업 요청 처리가 느린 부분이 있다. 또한, 자동화 하기에는 설정파일 형식이 적절하지 않다.

## netfilter

netfilter는 모든 리눅스 커널에서 제공해주는 표준적인 네트워크 보안 도구이다. 리눅스 커널에서 TCP/IP 및 네트워크 가상 장치 기반으로 라우팅 기능도 제공한다.



# 네트워크 필터(Nftables)

## 신 버전(수세 및 레드햇 공통)

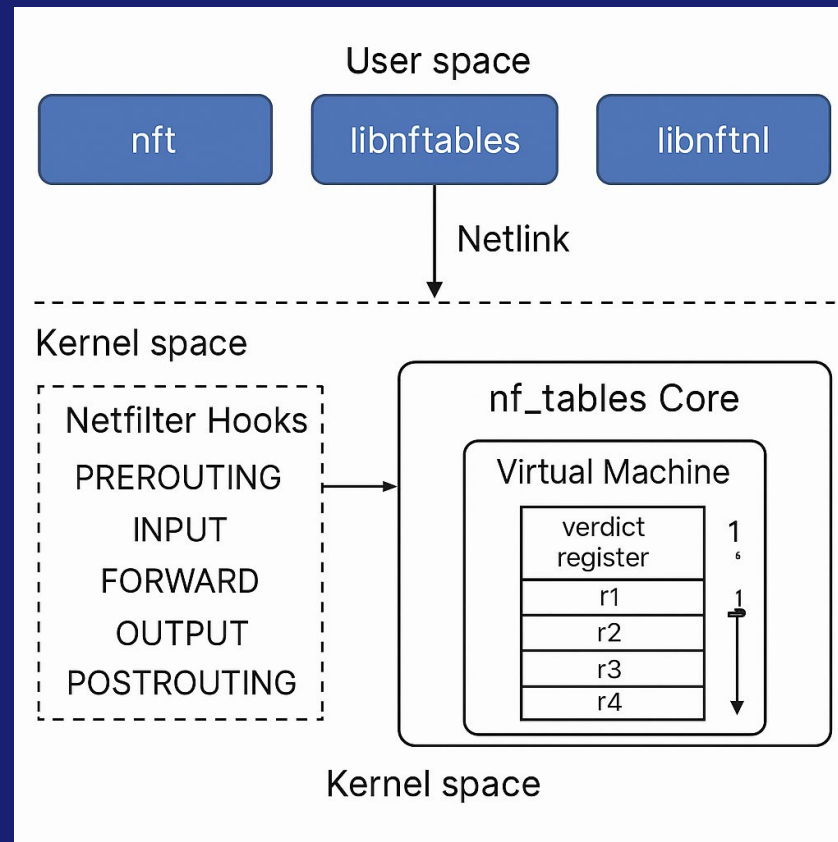
nftables는 이전 iptables의 차기 버전이다. 전체적으로 사용방식 및 동작 구조는 동일하나, 메모리 성능 개선 및 빠른 적용을 위한 바이트 컴파일 그리고, 설정파일 구조이다. 2014년도에 커널에 통합이 되었으며, 공식적으로 커널 3.13부터 통합이 되었다.

또한 모듈에서 기능적으로 중복되었던 기능도, 코드에서 전부 제거가 되었다. 이전에 사용하였던 ip4/6tables, arp, ebtables의 기능들은 nftables로 전부 통합이 되었다.

**참고링크:** [https://wiki.nftables.org/wiki-nftables/index.php/What\\_is\\_nftables%3F](https://wiki.nftables.org/wiki-nftables/index.php/What_is_nftables%3F)



# NETWORK FILTER(Nftables)



# 네트워크 필터 통합 이유

"Generally speaking, the userspace compiles high level statements (rules, etc) into the netlink bytecode that the kernel API understands. When inspecting the ruleset (i.e, listing it) what it does is the opposite, reconstruct the low level netlink bytecode into high level statements."

## 컴파일 관련 부분

<https://wiki.nftables.org/wiki-nftables/index.php/Scripting>

[https://wiki.nftables.org/wiki-nftables/index.php/Portal:DeveloperDocs/nftables\\_internals](https://wiki.nftables.org/wiki-nftables/index.php/Portal:DeveloperDocs/nftables_internals)



# 네트워크 명령어

## 신 버전(수세 및 레드햇 공통)

기존 netfilter에서 사용하던 hooks, conntrack, NAT, logging 그리고 사용자 영역의 기능은 대다수 그대로 사용한다. nftables가 지원하는 네트워크 자원 영역은 아래와 같다.

1. ip
2. ip6
3. inet
4. arp
5. bridge
6. netdev



# Netfilter기반 소프트웨어

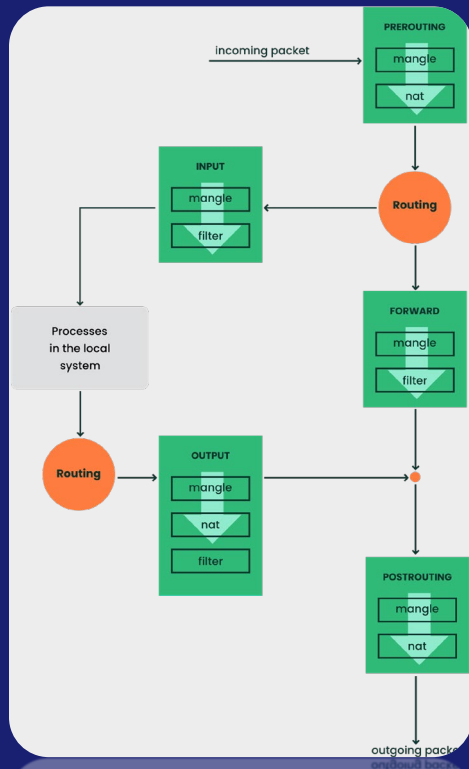
nftables가 지원하는 소프트웨어 범위가 있다. 대표적인 소프트웨어는 아래와 같다.

- fail2ban
- firewalld
- suricata-ids
- keepalived
- openstack
- libvirt
- saltstack
- coreos
- docker
- podman

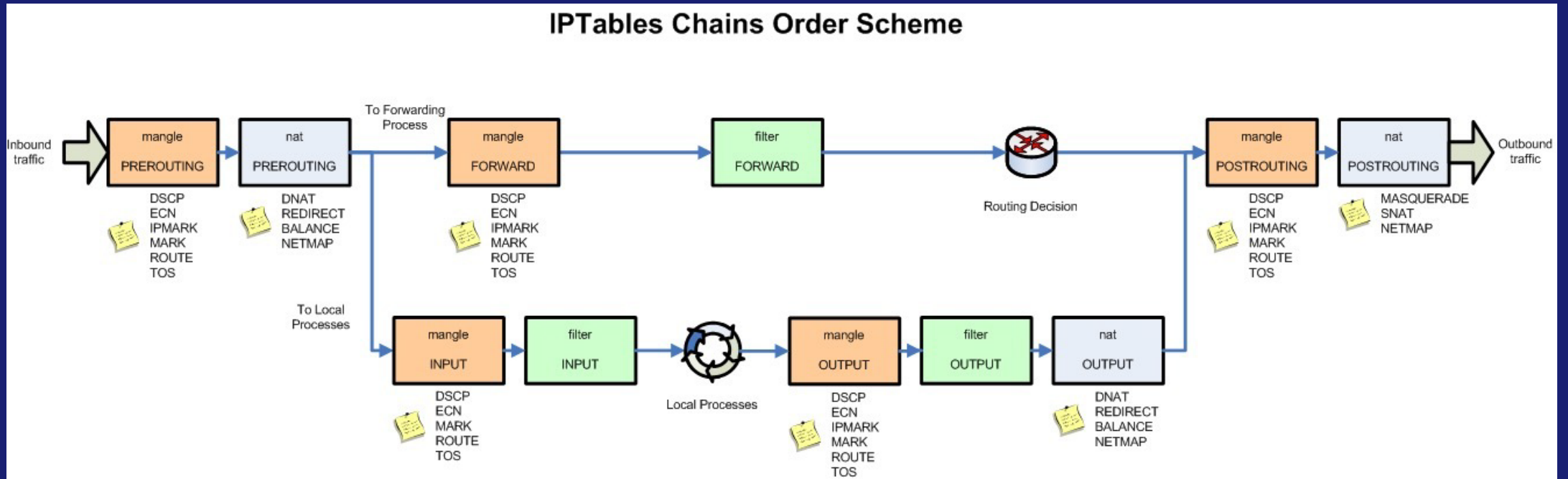


# NFTABLES FLOW

기존 iptables와 동일한 구조를 사용한다. 기존에 기능적으로 중복된 부분들을 왼쪽과 같이 정형화 및 최적화가 완료되었다.



# IPTABLES 동작 방식

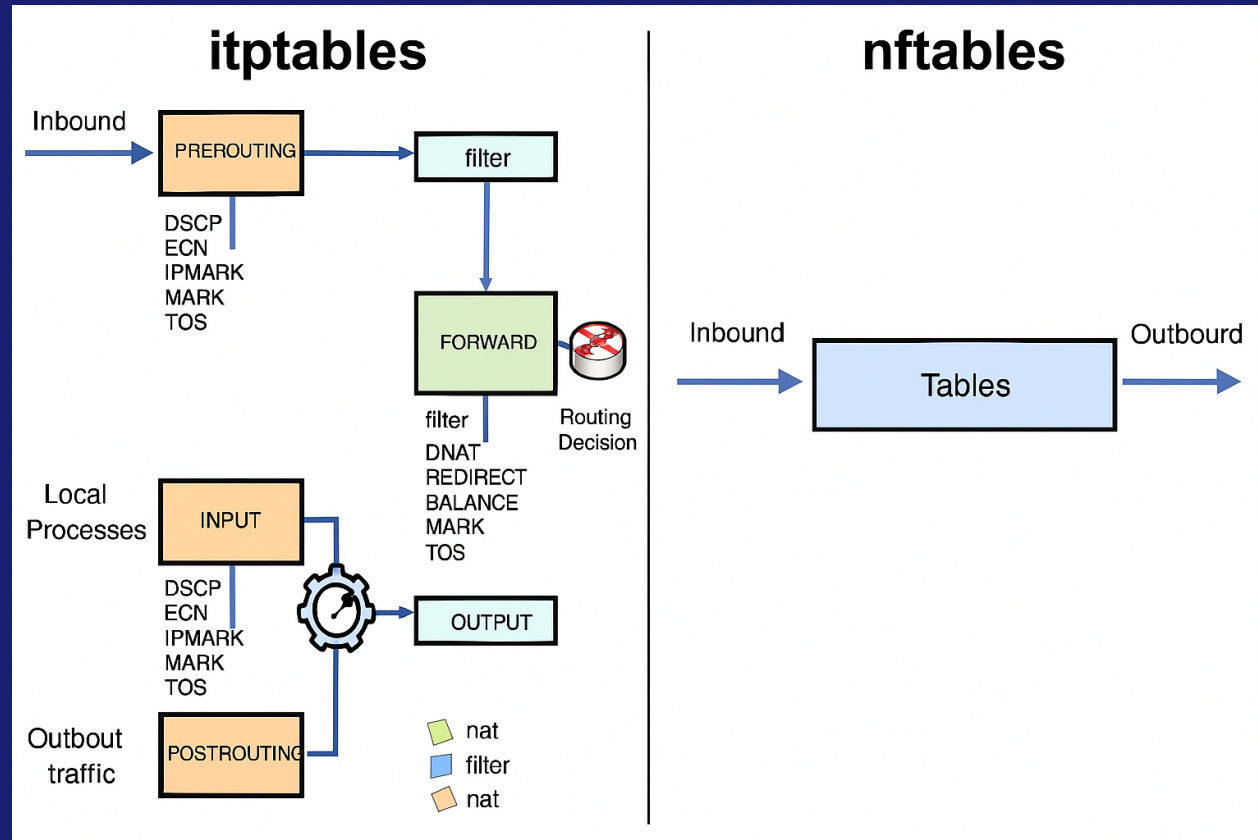


출처 링크: <https://jimmysong.io/en/blog/understanding-iptables/>





# IPTABLES/NFTABLES 기능 비교



# NFT 명령어 1

기존에 사용하였던 iptables명령어는 호환 패키지로 사용이 가능하다. 가급적이면 nft명령어를 사용한다.

```
zypper search iptables
zypper install iptables iptables-backend-nft
```

nft는 다음과 같이 테이블을 제공한다.

filter

> arp, bridge, ip, ip6 and inet

router

> ip, ip6

nat

> ip, ip6



# NFT 명령어 2

```
systemctl is-active firewalld  
firewall-cmd --list-all  
nft list tables  
nft list table inet firewalld  
nft -n -a list tables inet firewalld  
nft list ruleset  
nft list ruleset > backup.json  
nft flush ruleset  
nft -f backup.json
```



# NFT 정책 파일 선언

NFT기반으로 정책파일 생성 및 사용하기 위해서 아래와 명령어로 구성이 가능하다.

```
echo "flush ruleset" > /etc/nftables.rules  
echo "add table filter" >> /etc/nftables.rules  
echo "add chain filter input" >> /etc/nftables.rules  
echo "add rule filter input meta iifname lo accept" >> /etc/nftables.rules  
  
nft -f /etc/nftables.rules
```



# NFT MANUAL

테이블 생성시 다음과 같은 규칙으로 작성 및 구성한다.

```
nft add table family_type table_name
```

```
nft add table ip test_table
```

```
nft add chain ip test_table test_chain '{ type filter hook input priority  
0; }'
```

```
nft list tables ip
```

```
nft list tables
```

```
nft delete table ip
```

```
nft flush table k; test_tables
```



# IPTABLES 마이그레이션

기존에 사용하던 아이피 테이블을 이전하기 위해서 다음과 같은 명령어로 사용이 가능하다.

```
iptables-save > iptables-backup.config
```

복구 명령어는 다음과 같다.

```
iptables-restore < iptables-backup.config
```

위 명령어는 각각 다음과 같은 패키지에 포함이 되어 있다.

```
dnf install iptables-nft
```

```
zypper install iptables
```



# NFT 백업 및 복구

다음과 같은 방법으로 백업이 가능하다. 백업은 JSON, TEXT 형식 두 가지를 제공하며, 가급적이면 JSON으로 백업을 권장한다.

복구 명령어는 다음과 같다.

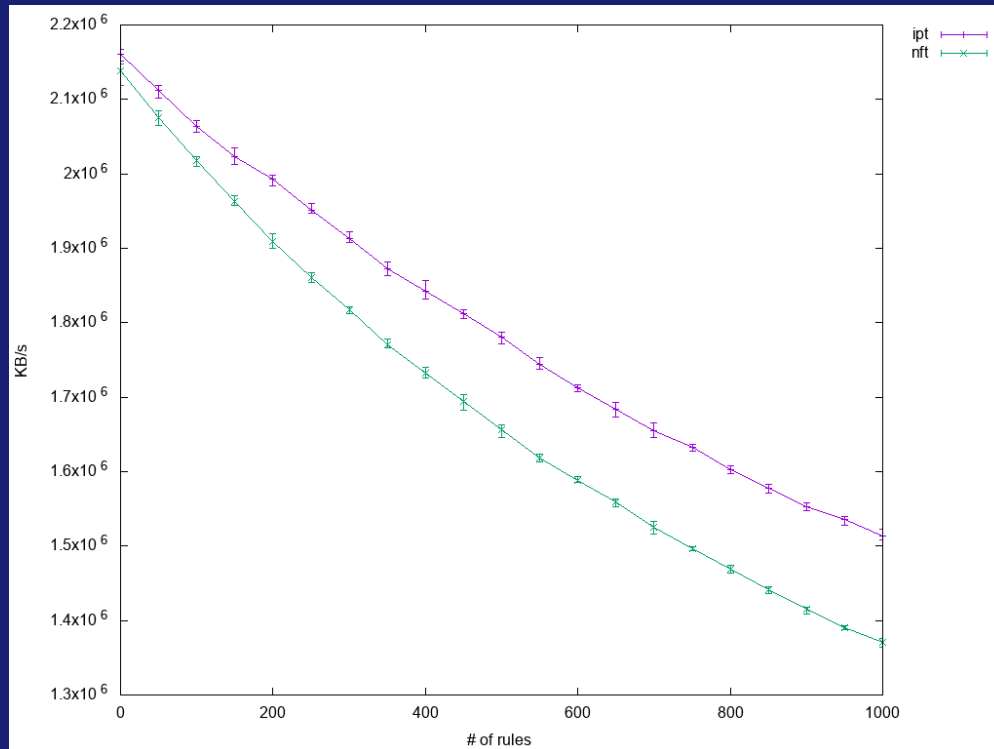
```
nft list ruleset > backup.list  
nft -j list ruleset > backup.json
```

```
nft -f backup.list  
nft -f -j backup.json
```



# 성능비교 문서

<https://developers.redhat.com/blog/2017/04/11/benchmarking-nftables>





# Iptables/Nftables와 쿠버네티스 관계

쿠버네티스는 다음과 같은 형식으로 네트워크 기능을 제공하고 있다.

1. 커널 기반으로 네트워크 지원(Pod/Service)
2. Iptables는 패킷 트래킹 성능이 부족해서 IPVS 기반으로 지원
3. Nftables에서는 성능저하 없이 패킷 트래킹 지원

이러한 이유로 현재 쿠버네티스는 iptables보다 nftables 사용을 권장한다. 하지만, 쿠버네티스는 명확하게 어떠한 버전에서 nftables를 사용하세요!라고 명시가 안되어 있다.



# IPVS/Nftables

| 항목      | ipvs     | nftables kube-proxy |
|---------|----------|---------------------|
| 지원 상태   | 유지       | 적극                  |
| 기본값     | 아니요      | 신규 시스템 및 환경         |
| 대규모 서비스 | 👍        | 👍                   |
| 운영 난이도  | 😞        | 😊                   |
| 디버깅     | 어려움      | 쉬움                  |
| 미래 방향성  | 유지 및 호환성 | 핵심기술로 계속 유지         |



# Kubernetes 네트워크 모드별 버전 지원 요약

| Kubernetes 버전 | iptables (legacy/nft) | ipvs       | nftables (kube-proxy nft) | 평가       |
|---------------|-----------------------|------------|---------------------------|----------|
| ≤ 1.20        | legacy 중심             | 안정 지원 및 유지 | 아니요                       | 구버전, 비권장 |
| 1.21 ~ 1.23   | nft wrapper           |            |                           | 과도기      |
| 1.24          | nft 인식                |            | 실험적                       | 전환기      |
| 1.25 ~ 1.27   | nft backend           |            | 안정                        | 권장       |
| 1.28 ~ 1.30+  | 유지                    | 유지(호환성)    | 기본                        | 최신 표준    |



# 표준 방화벽, Firewallld

표준 방화벽, **firewalld**는 현재 대다수 기업용 배포판 및 커뮤니티 배포판에서 **iptables, nftables** 상위 관리자로 사용하고 있다.

보통 iptables, nftables를 저-수준, firewalld를 고-수준 방화벽이라고 부른다. firewalld는 기존 방화벽과 다르게 freedesktop.org를 dbus기반으로 따르고 있다. firewalld는 대다수 상용 배포판에서 사용하고 있기 때문에, 별 다른 수정없이 바로 마이그레이션이 가능하다.

이를 통해서 아래 프로그램과 상호작용이 가능하다.

1. NetworkManager
2. libvirt
3. podman
4. docker(iptables)
5. fail2ban



# FIREWALLD 마이그레이션(다른 배포판 끼리)

Firewalld기반으로 구성이 되어있는 시스템을 이전하는 경우 다음과 같은 방법으로 이전이 가능하다.

## 1. 쉬운 방법

"/etc/firewalld/"이 디렉터리를 전부 다 복사한다. 하지만, 효율적이지는 않다. 또한, 기존에 사용하던 Firewalld와 충돌 혹은 오류가 발생할 가능성이 높다.

## 2. 조금 어려운 방법

방화벽은 영역(zone)저장된 내용별로 "zones/"디렉터리 밑에 저장한다. 여기에서 영역의 이름 파일을 복사하면 된다. 예를 들어서 public영역은 public.xml으로 저장이 되어있다.

## 3. 어려운 방법

firewall-cmd명령어로 각각 존 파일을 저장한다. 이 방법이 제일 안전한 방법이지만, 반복적인 작업이 필요하기 때문에 아래와 같이 스크립트로 처리한다.



# 패키지 관리자

dnf/zypper

RPM

# zypper 설명

수세 리눅스에서 사용하는 고급 패키지 관리자. 레드햇의 dnf, yum과 동일한 기능. 수세 리눅스는 사용자가 zypper에 익숙하지 않는 사용자를 위해서 YUM/DNF과(와) 명령어 지원을 한다. zypper명령어는 기본적으로 YUM/DNF과(와) 사용 방법은 동일하다.

**zypper <VERBE> <PACKAGE\_NAME>**

저장소 정보 및 설정파일 위치는 아래에 존재한다. 저장소 구성 형식은 YUM/DNF와 동일하게 INI형태로 되어있다.

```
cat /etc/zypp/repos.d/repo-oss.repo
[repo-oss]
name=Main Repository
enabled=1
baseurl=http://download.opensuse.org/distribution/leap/$releasever/repo/oss
/
```



# zypper 주요 명령어

| 옵션                       | 약어        | 설명                 | 예제                                        |
|--------------------------|-----------|--------------------|-------------------------------------------|
| <b>install</b>           | <b>in</b> | 패키지 설치             | <code>zypper install apache</code>        |
| <b>remove</b>            | <b>rm</b> | 패키지 제거             | <code>zypper remove apache</code>         |
| <b>search</b>            | <b>se</b> | 패키지 검색             | <code>zypper search mariadb</code>        |
| <b>update</b>            | <b>up</b> | 패키지 업데이트(마이너 버전)   | <code>zypper update</code>                |
| <b>dist-upgrade</b>      |           | 패키지 업그레이드(메이저 버전)  | <code>zypper upgrade</code>               |
| <b>patterns, pd, pch</b> |           | 패턴, 제품, 패치 목록      | <code>zypper -t patch list</code>         |
| <b>-t patterns</b>       |           | 쿼리 형식 명시           | <code>zypper patch-check</code>           |
| <b>what-provide</b>      |           | 파일을 소유하고 있는 패키지 확인 | <code>zypper what-provide /bin/kls</code> |





# zypper의 유일한 단점?

**zypper**는 이전에 설치된 패키지에 대해서 기록 확인이 어렵다. dnf처럼 패키지 설치 기록을 확인하기 위해서는 다음과 같은 방법을 제안한다.

1. /var/log/zypp/history
2. yum/dnf 설치 후, 패키지 설치 기록 및 롤백 기능 수행(비 권장)
3. btrfs혹은 LVM2기반으로 롤백 및 변경 기록 확인(snapper)

**1번** 경우에는 로그파일에서 변경 내용이 확인이 가능하다.

**2/3번** 경우에는 yum/dnf를 설치 후 zypper는 사용하지 않고, dnf로 관리가 가능하다. 레드햇과 동일하게 패키지 기록을 남길수 있다. 3번 경우에는 파일 시스템 기반에서 구성하는 방법이기 때문에 dnf와 동일하게 기능을 제공하지 않는다.



# zypper 나머지 명령어들...

나머지 명령어는 아래 링크에서 확인이 가능하다.

<https://en.opensuse.org/images/1/17/Zypper-cheat-sheet-1.pdf>



# DNF/YUM in SuSE

기존에 사용하였던 **DNF/YUM**에 익숙한 사용자는 수세에서 호환성 패키지로 제공한다. 다만, 가급적이면 DNF/YUM명령어 보다는 **Zypper**명령어로 패키지 관리 및 업그레이드를 권장한다.

수세 리눅스는 기존에 사용하던 yum명령어는 레드햇과 동일하게 더 이상 지원하지 않으며, dnf-3명령어로 링크가 되어있다.

```
zypper install -y dnf-yum dnf
```

```
whereis yum
```

```
lrwxrwxrwx 1 root root 5 May 22 19:37 /usr/bin/yum → dnf-3
```



# DNF/YUM in SuSE

dnf를 수세 리눅스 저장소를 사용하기 위해서 아래와 같이 확장 패키지를 설치한다.

```
zypper install rpm-repos-openSUSE-Leap
ls -l /etc/yum.repos.d/
opensuse-leap-non-oss.repo    opensuse-leap-sle-backports-update.repo
opensuse-leap-oss.repo       opensuse-leap-sle-update.repo
dnf repolist
dnf search apache
dnf install apache
```



# 저장소 미러링 1

저장소 미러링 방식은 두 가지가 있다. 오픈수세 및 수세 엔터프라이즈에 동시에 사용이 가능하기 때문에, 둘 중, 하나를 선택해서 사용한다.

- HTTP로 저장소 미러링
- 오픈수세에서 제공하는 패키지 저장소 구성(RMT)
  - [https://en.opensuse.org/openSUSE:Mirror\\_howto](https://en.opensuse.org/openSUSE:Mirror_howto)

위의 방식은 레드햇에서 사용하는 방식과 비슷하게 구성 및 사용이 가능하다. 단일 서버 및 단순하게 사용한다면, 이 방법을 권장한다.



# 저장소 미러링 2

## SLES에서 제공하는 패키지 저장소 구성

SLES를 사용하는 경우, RMT패키지를 설치 후, 사용이 가능하다. RMT서버를 오픈수세에 설치하기 위해서는 아래 링크에서 확인이 가능하다.

- <https://software.opensuse.org/package/rmt-server?locale=km>
- <https://documentation.suse.com/sles/15-SP1/single-html/SLES-rmt/index.html>



# RMT(REPOSITORY MIRRORING TOOL)

**RMT**는 수세 리눅스 포함하여 다양한 배포판의 저장소를 구축 및 구성할 수 있도록 도와주는 도구이다. 레드햇은 RMT와 비슷한 기능을 가지고 있는 인프라 프로그램은 **foreman(Satellite)** 프로그램이 있다.

**OpenSuSE Leap**에서는 패키지가 없으며, RMT는 SLES이나 혹은 OpenSuSE Tumbleweed에서 지원한다.

RMT, foreman를 사용하지 않는다면, 기존 방식은 HTTP/HTTPS기반으로 저장소를 구현하여도 된다. 수세도 레드햇의 YUM/DNF와 동일하게 RPM파일들을 전부 로컬에 저장하여 폐쇄망 전용으로 사용이 가능하다.

| 항목                 | 가능 여부 | 비고                                           |
|--------------------|-------|----------------------------------------------|
| openSUSE에서 RMT 설치  | 가능    | rmt-server 패키지 설치 가능                         |
| SLE 리포지터리 미러링      | 불가    | SLE 리포지터리는 인증 기반 (SCC 계정 필요)                 |
| openSUSE 리포지터리 미러링 | 가능    | rsync 혹은 mirror-o-o.opensuse.org 통해 별도 처리 필요 |
| 공식 클라이언트 등록 및 관리   | 불가    | openSUSE는 RMT 등록 대상이 아님 (zypper 직접 설정)       |



# RMT(REPOSITORY MIRRORING TOOL)

미러링 하는 방법은 다음과 같다.

```
rsync -rlptyH rsync.opensuse.org::opensuse-full \  
/srv/pub/opensuse/ --delay-updates --delete-delay -hi --stats
```

```
vi /etc/zypp/repos.d  
[local-openSUSE-Leap-15.5]  
name=openSUSE-Leap-15.5  
enabled=1  
autorefresh=0  
baseurl=http://10.10.10.250/opensuse/  
path=/  
type=rpm-md
```





# DNF 저장소 구성

혹은, 이전 레드햇에서 사용하던 방식으로 사용하여도 상관 없다. 다만, 이 방식을 사용하기 위해서는 앞서 이야기 하였던 dnf도구를 설치해지 가능하다.

```
zypper in rpm-repos-openSUSE-Leap
dnf reposync -p /srv/rpm-mirrors
vi /etc/yum.repos.d/opensuse-leap.repo
[opensuse-leap]
name=openSUSE Leap OSS (RPM-MD)
baseurl=https://download.opensuse.org/distribution/leap/15.5/repo/oss/
enabled=1
gpgcheck=1
gpgkey=https://download.opensuse.org/distribution/leap/15.5/repo/oss/repo
ata/repomd.xml.key
dnf clean allsudo dnf repolistsudo dnf install vim
```



# RPM 명령어 1

저 수준 패키지 관리자는 레드햇과 동일하게 RPM패키지를 사용한다. RPM패키지는 고급 관리자, 즉, DNF/YUM 혹은 Zypper를 사용할 수 없는 경우 명령어를 통해서 패키지 관리한다.

| 명령어                 | 설명                                   | 예시                                     |
|---------------------|--------------------------------------|----------------------------------------|
| <b>rpm -qi</b>      | 패키지 메타 정보를 확인한다.                     | <b>rpm -qi apache2</b>                 |
| <b>rpm -ql</b>      | 패키지가 가지고 있는 파일 및 디렉터리 목록을 출력한다.      | <b>rpm -ql apache2</b>                 |
| <b>rpm -qf</b>      | 파일 및 디렉터리가 어떠한 패키지에 포함되어 있는지 확인한다.   | <b>rpm -qf /etc/apache2</b>            |
| <b>rpm --nodeps</b> | 의존성을 무시하고 패키지 설치 및 제거를 진행한다.         | <b>rpm -e --nodeps apache2</b>         |
| <b>rpm --force</b>  | 패키지 제거 시, 의존성과 같은 문제에 대해서 무시하고 제거한다. | <b>rpm -e --nodeps --force apache2</b> |



# RPM 명령어 2

저 수준 패키지 관리자는 레드햇과 동일하게 RPM패키지를 사용한다. RPM패키지는 고급 관리자, 즉, DNF/YUM 혹은 Zypper를 사용할 수 없는 경우 명령어를 통해서 패키지 관리한다.

| 명령어             | 설명                | 예시                          |
|-----------------|-------------------|-----------------------------|
| <b>rpm -ivh</b> | 패키지 설치를 진행한다.     | <b>rpm -ivh apache2.rpm</b> |
| <b>rpm -Uvh</b> | 패키지 업그레이드를 수행한다.  | <b>rpm -Uvh apache2.rpm</b> |
| <b>rpm -e</b>   | 설치된 패키지를 제거한다.    | <b>rpm -e apache2</b>       |
| <b>rpm -qa</b>  | 설치된 패키지 목록을 확인한다. |                             |



# RPM 명령어 3

수세 리눅스는 레드햇과 다르게, 저장소가 다양하게 존재하지 않는다. 보통은 **OBS(Open Build Service)**를 통해서 제공한다.

- <https://openbuildservice.org/>

OBS는 수세 리눅스 이외 다른 배포판의 패키지 빌드가 가능. 오픈수세 리눅스의 다수의 확장 패키지는 일반적으로 OBS에서 제공이 된다. OBS 빌드가 되고 있는 패키지 혹은 배포판이 궁금한 경우 아래 주소에서 확인이 가능하다.

- <https://build.opensuse.org/project>

만약, RPM을 수동으로 **컴파일/빌드**가 필요한 경우 아래와 같은 방법으로 RPM패키지 생성이 가능하다.



# 사용자 커널 빌드 RPM기반 예제 1

```
zypper install rpm-build  
zypper install kernel-source  
cd /usr/src/linux  
make mrproper  
make menuconfig  
cp /boot/config-`uname -r` .config
```



# 사용자 커널 빌드 RPM기반 예제 2

```
vi Makefile
```

```
VERSION = 5
```

```
PATCHLEVEL = 14
```

```
SUBLEVEL = 1
```

```
EXTRAVERSION = -self
```

```
NAME = openteam
```

```
make rpm
```



# 저장소 미러링(레드햇 기준)

레드햇은 대표적인 확장 저장소는 다음과 같다.

하지만, 수세에서는 EPEL 및 `rpmfusion`, `remi` 그리고 `ELRepo`는 호환이 되지 않는다.

이 RPM들은 레드햇 커널 및 라이브러리 기준으로 되어 있기 때문에 수세 리눅스에서는 사용이 불가능하다.



# 파일시스템 기본 비교

XFS

BTRFS



# XFS 설명

레드햇 리눅스는 RHEL 7버전까지는 XFS/BTRFS/EXT4를 지원. 다른 리눅스 배포판처럼 중간에 btrfs으로 넘어가려고 하였으나, RHEL 7.4이후로 btrfs는 제외하고 XFS기반으로 파일 시스템을 유지.

레드햇 배포판은 현재 XFS기반으로 다음과 같이 구성 및 구현하고 있다. 레드햇이 BTRFS를 사용하지 않는 이유는 다음과 같다.

- BTRFS는 기본적으로 ZFS/JFS과 비슷하게 구성이 됨
- Milspec, enterprise workload에 적합하지 않다고 판단
- BTRFS는 여전히 페도라 리눅스에서는 지원하고 있음
- Red Hat은 Btrfs의 기능을 대체하기 위해 Stratis와 같은 새로운 기술을 개발하여 도입
- Stratis는 LVM과 XFS를 기반으로 하여 Btrfs와 유사한 기능을 제공하면서도 기존의 안정적인 기술을 활용
- RHEL 10에서는 다시 btrfs에 대해서 옵션으로 지원 및 제공하고 있다.



# XFS 설명

XFS기반의 파일 시스템은 다음과 같은 기능을 제공 하지 않는다. 하지만 몇가지 기능들은 확장 기능으로 제공한다. 이 부분은 추후 뒤에서 더 다루기로 한다.

- Shrink filesystem(SGI에서는 비공식 제공은 하였음)
  - RHEL 10부터 실험적으로 Shrink 기능 지원
- Filesystem pool and pooling
  - 온라인/백그라운드 관리기능 추가
  - xfs\_scrub를 통해서 실시간으로 온라인 검사 및 수정 가능
  - 메모리 최적화 및 쿠버네티스에 XFS 노이즈 부분 해결
- Snapshot and live filesystem check
  - RHEL 9 후기 및 RHEL 10부터 CRC 실시간 확인 지원



# XFS 기업용 도구

XFS에서는 엔터프라이즈 파일 시스템 기능을 아래 도구를 통해서 지원한다. XFS파일 시스템에서는 네이티브로 지원하지 못한다.

- **Filesystem Pool:** Stratis(write in rust)
- **Volume Snapshot:** vdo

이러한 이유로 레드햇 계열의 배포판에서는 파일 시스템 관리를 하기 위해서 각각 다른 관리 명령어를 통해서 관리가 가능 하다.



# XFS VERSION 과정

현재 레드햇은 배포판을 릴리즈 하면서 XFS파일 시스템에 대한 변화를 많이 가져가고 있다. 이러한 이유로 메이저 버전별로 서로 파일 시스템 마운트를 하지 못하는 이슈가 있다.

1. RHEL 7 → XFS 4.5
2. RHEL 8 → XFS 5.0
3. RHEL 9 → XFS 5.19
4. RHEL 10 → XFS 6.11

읽기 전용

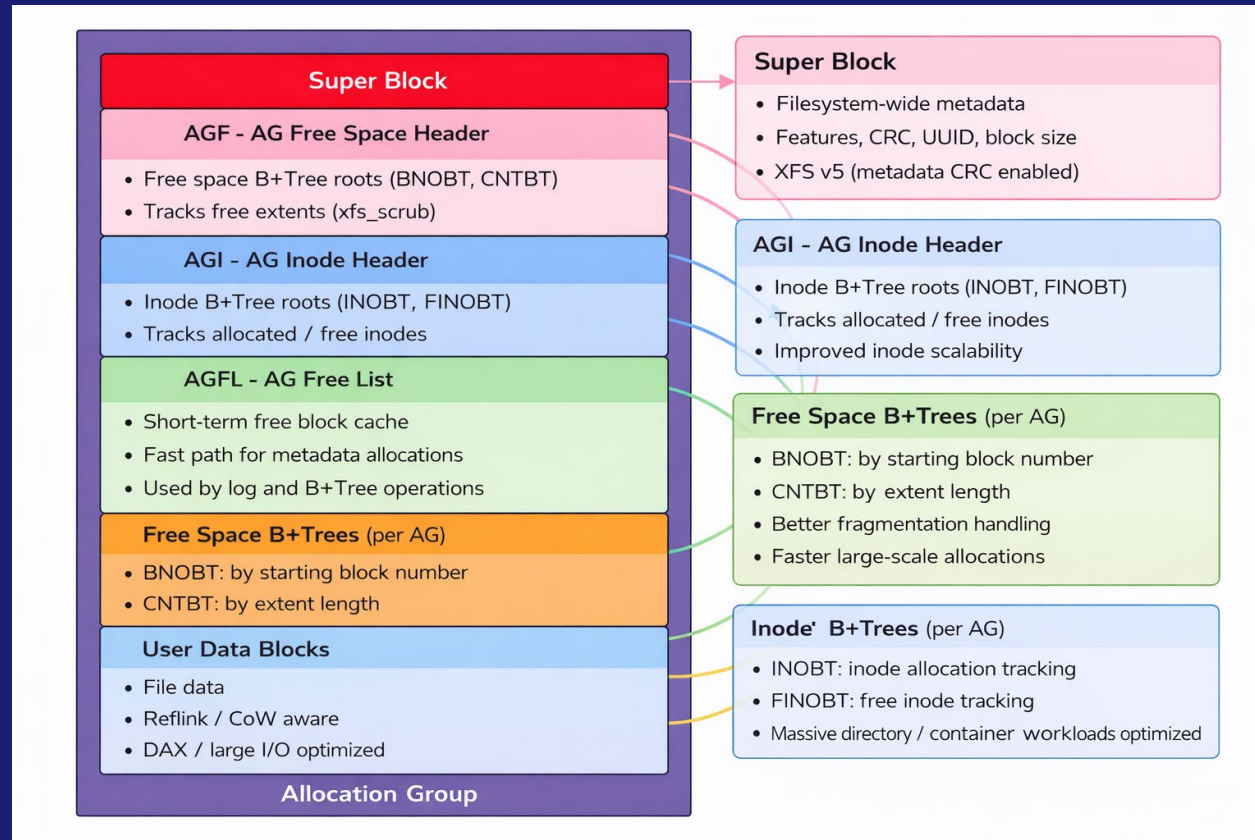


# XFS 호환성

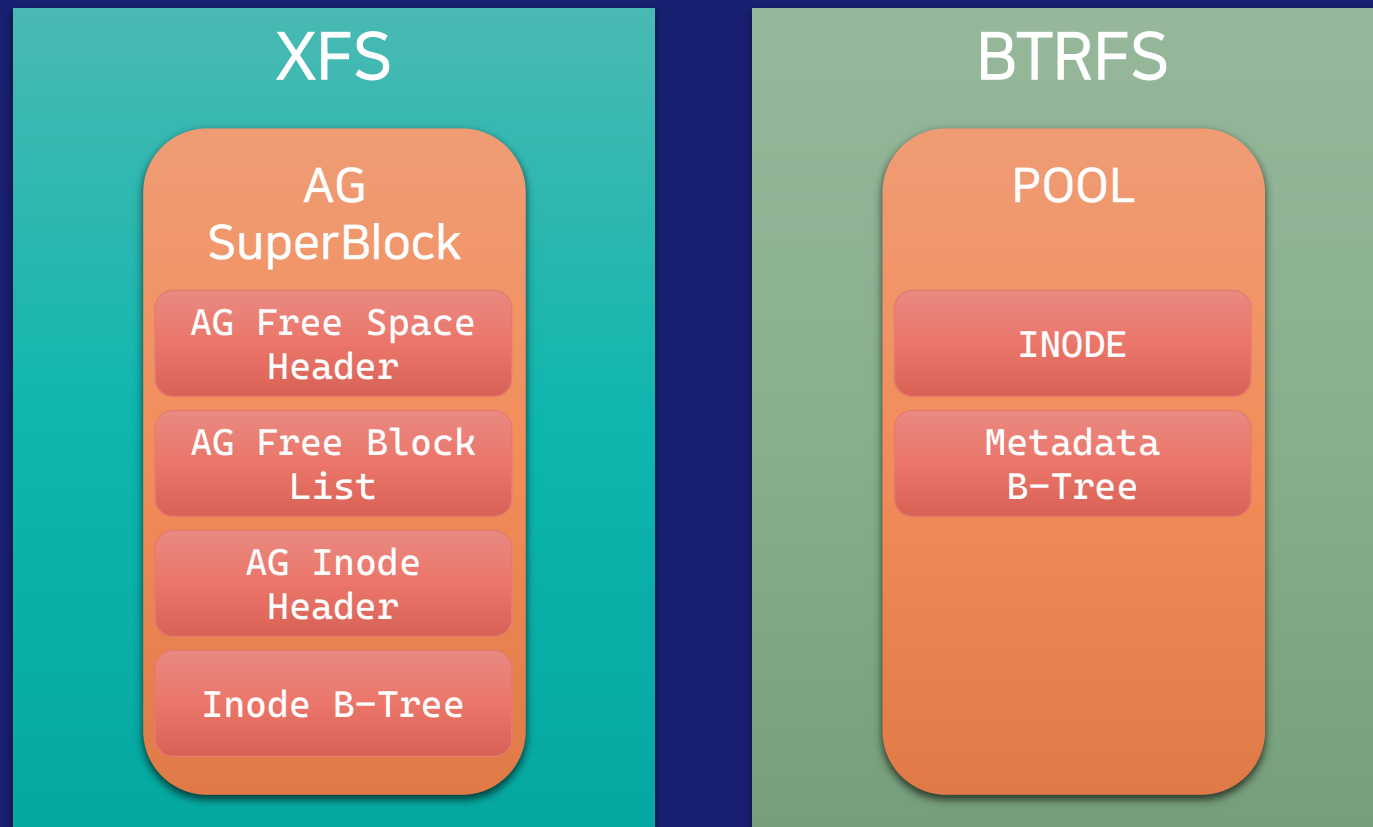
레드햇 XFS 경우에는 **AG(Allocation Group)** 업데이트로 인하여 **RHEL7/8/9+10** 버전에서는 하위는 상위 버전은 XFS는 읽기전용으로 접근이 가능하며, 상위 버전은 하위 버전의 XFS에 대해서는 읽기/쓰기로 접근이 가능하다. 이러한 이유로, 가상머신 운영 시 하위 버전은 상위 버전의 가상머신 디스크에 대해서 접근이 어렵다.



# XFS 슈퍼블록 및 AG



# XFS와 BTRFS 구조 비교



# XFS와 B-Tree 구조 차이 1

| 기능/역할           | XFS 구조 (AG 기반)                         | Btrfs 구조 (B-tree 기반)                  |
|-----------------|----------------------------------------|---------------------------------------|
| 메타데이터 분산        | Allocation Group(AG) 별로 분산되어 구조화됨      | 모든 메타데이터는 전역 B-tree에 의해 통합 관리됨        |
| SB (Superblock) | 파일시스템 전체 및 AG별 Superblock 존재           | 루트 슈퍼블록에서 파일시스템 전반 정보를 관리             |
| AGF             | Free space 관리용 B-tree 루트 정보            | 공간 할당 정보도 메타데이터 B-tree에 통합되어 저장됨      |
| AGFL            | Free block 리스트 (재활용 가능 블록 목록)          | 별도의 리스트 없이 B-tree 기반으로 free block 트래킹 |
| AGI             | Inode B-tree 루트 (inode 번호를 기반으로 빠른 검색) | inode 자체가 키-값 구조로 메타데이터 트리의 객체로 관리됨   |





# XFS와 B-Tree 구조 차이 2

| 기능/역할       | XFS 구조 (AG 기반)                        | Btrfs 구조 (B-tree 기반)                            |
|-------------|---------------------------------------|-------------------------------------------------|
| Inode 저장 방식 | 디스크 내 고정 위치에서 AG 단위로 할당됨              | Inode는 object tree의 item으로 B-tree 안에 존재 (동적 위치) |
| 기본 데이터구조    | 여러 개의 B+Tree (free space, inode, etc) | 하나의 통합된 CoW B-tree (Metadata + Data 모두 통합 관리)   |
| 스냅샷 기능      | 기본적으로 없음 (LVM 등 외부 도구 필요)             | CoW 기반으로 내부적으로 스냅샷, 롤백, 서브볼륨 지원                 |
| 저널링 방식      | 별도 저널 존재 (write-ahead logging 구조)     | 트랜잭션 메타데이터도 CoW로 관리, 자체적인 consistency 보장        |



# 간단한 Stratis 설명

LVM의 복잡함은 줄이고, 운영 편의성은 올리는 리눅스 스토리지 관리 레이어.

Stratis가 해결하는 문제

1. LVM 조합( PV/VG/LV ) + 파일시스템(XFS) 관리가 번거롭다
2. 스냅샷/확장/풀 관리가 실수 유발 + 운영 피로로 이어진다
3. ZFS처럼 편하게 하고 싶지만 라이선스/표준화/운영정책 때문에 도입이 어렵다



# Stratis를 써야 하는 이유 (핵심 5가지)

## 1. 풀 기반 관리로 운영 단순화

- 디스크를 풀(pool)로 묶고 그 위에 파일시스템을 올려서, 용량 관리가 직관적

## 2. Thin Provisioning 기본 지원

- 필요할 때만 실제 공간을 쓰도록 설계되어, 초기 설계가 훨씬 유연해짐

## 3. 온라인 확장에 강함

- 서비스 중단 없이 풀 확장 → 파일시스템 확장 흐름이 자연스럽다  
(특히 VM/컨테이너 노드에서 체감 큼)

## 4. 스냅샷으로 빠른 롤백/테스트

- 랩/교육/검증 환경에서 실습 전 스냅샷 → 실습 후 롤백이 매우 편하다

## 5. 엔터프라이즈 리눅스 친화적

- RHEL 계열에서 표준 스택(LVM + XFS + device-mapper) 위에 올라가 안정적이고, 운영팀 관점에서 새 파일시스템 구성을 강요하지 않는다



# XFS기반의 POOL 구성

간단하게 설치 및 구현하여 BTRFS와 비교한다.

```
dnf search stratis
dnf install stratisd stratisd-tools stratis-cli -y
dnf install hexedit -y
```

```
systemctl status stratisd
systemctl enable --now stratisd
```

```
stratis blockdev list
stratis pool list
stratis pool create
stratis pool create firstpool /dev/sdb
```



# Stratis LAB 1

앞에 내용 계속 이어서...

```
stratis filesystem create --size 1GiB firstpool first-xfs
```

```
stratis filesystem list
```

```
/dev/stratis/firstpool/first-xfs
```

```
dmsetup ls
```

```
hexedit /dev/stratis/firstpool/first-xfs
```

```
hexedit /dev/sdb
```



# Stratis LAB 2

앞에 내용 계속 이어서...

```
stratis pool add-data firstpool /dev/sdc
```

```
stratis pool list
```

```
firstpool    20 GiB / 610.50 MiB / 19.40 GiB    ~Ca,~Cr, Op    63b843af-0277-  
4751-af5e-f7f0057efc56
```

```
stratis filesystem create --size 2GiB firstpool second-xfs
```

```
stratis fs snapshot firstpool first-xfs snap-first-xfs
```

```
stratis fs list
```



# 명령어 정리

| 구분  | 목적             | Stratis 명령어                                              |
|-----|----------------|----------------------------------------------------------|
| 서비스 | 데몬 시작          | <code>systemctl enable --now stratisd</code>             |
| 서비스 | 상태 확인          | <code>systemctl status stratisd</code>                   |
| 디스크 | 블록 디바이스 확인     | <code>lsblk</code>                                       |
| 디스크 | Stratis 인식 디스크 | <code>stratis blockdev</code>                            |
| 풀   | 풀 생성           | <code>stratis pool create pool1 /dev/sdb /dev/sdc</code> |
| 풀   | 풀 목록           | <code>stratis pool list</code>                           |
| 풀   | 풀 상세           | <code>stratis pool inspect pool1</code>                  |
| 풀   | 디스크 추가         | <code>stratis pool add-data pool1 /dev/sdd</code>        |
| 풀   | 풀 삭제           | <code>stratis pool destroy pool1</code>                  |



# 명령어 정리

| 구분  | 목적        | Stratis 명령어                                              |
|-----|-----------|----------------------------------------------------------|
| FS  | 파일시스템 생성  | <code>stratis filesystem create pool1 fs1</code>         |
| FS  | 파일시스템 목록  | <code>stratis filesystem list</code>                     |
| FS  | 파일시스템 삭제  | <code>stratis filesystem destroy pool1 fs1</code>        |
| 마운트 | 수동 마운트    | <code>mount /stratis/pool1/fs1 /mnt/fs1</code>           |
| 마운트 | 영구 마운트    | <code>/etc/fstab</code> 설정                               |
| 스냅샷 | 스냅샷 생성    | <code>stratis filesystem snapshot pool1 fs1 snap1</code> |
| 스냅샷 | 스냅샷 마운트   | <code>mount /stratis/pool1/snap1 /mnt/snap1</code>       |
| 확장  | 디스크 확장    | <code>stratis pool add-data pool1 /dev/sde</code>        |
| 확장  | FS 온라인 확장 | <code>xfs_growfs /mnt/fs1</code>                         |
| 점검  | 전체 상태 확인  | <code>stratis daemon status</code>                       |





# Stratisd 부팅 영역 지원 1

## Support in the initramfs

The next step was to integrate stratisd-min into the initramfs. This involved quite a bit of configuration for dracut and systemd.

Our current model uses systemd generators that are enabled by passing information on the kernel command line at boot to properly set up and unlock any devices that need to be unlocked. We aim to do this in as user-friendly of a way as possible by leveraging existing tools like Plymouth to handle prompting users for a passphrase on the splash screen.

## Stratisd Ramdisk(initramfs)

<https://stratis-storage.github.io/stratis-rootfs-fedora/>



## Stratisd 부팅 영역 지원 2

XFS는 커널에 빌트인이 된 상태로 제공이 되고 있지만, **확장기능(Extension Function)**에 대해서는 아직 빌트인 형태로 제공하고 있지는 않다. 예로, **Stratis**기반으로 **XFS pool filesystem** 경우에는 현재 리눅스 커널에서는 공식적으로 지원하고 있지 않음.

페도라 리눅스에서 커널 패치를 통해서 XFS pool filesystem를 **Stratis**기반으로 OS영역에서 사용이 가능하도록 지원하고 있다.

이러한 구조는 커널 수준에서 제공하는 **filesystem pool**이 아니기 때문에, BTRFS에서 제공하는 **storage pool**과는 개념이 조금 다르다. Stratis에서 제공하는 storage pool은 RAM disk에서 서비스 수준으로 제공한다.

<https://stratis-storage.github.io/stratis-rootfs/>



# XFS DUP

XFS DUP(Deduplication)기능을 네이티브로 지원하지 않는다. 이 기능을 사용하기 위해서는 RHEL 7에 TP로 소개된 VDO기반으로 구현이 가능하다.

VDO는 이전에 독립적으로 지원하였지만, 지금은 LVM2기반으로 동작한다.

| 기능                    | 물리적 장치      | 구성되는 장치     |
|-----------------------|-------------|-------------|
| VDO on LVM            | VDO pool LV | VDO LV      |
| LVM thin provisioning | Thin pool   | Thin volume |



# VDO(DUP) FOR XFS

```
dnf search vdo
```

```
dnf install vdo kmod-vdo vdo-support hexedit
```

```
modprobe kvdo ## RHEL 8버전 후기부터는 이러한 문제가 없음(uEFI)
```

```
modprobe: ERROR: could not insert 'kvdo': Key was rejected by service
```

```
pvcreate /dev/sdd
```

```
vgcreate vod_vg /dev/sdd
```

```
lvcreate --type vdo -L 100%Free --name vdo_lv vod_vg
```

```
lsmod | grep vdo
```

```
vod vdostats
```



# BTRFS IN RHEL

수세 리눅스는 리눅스 파운데이션에서 **EXT2/3/4**이후의 후계자로 지명한 BTRFS기반으로 구성하고 있다. BTRFS는 오라클의 ZFS, IBM의 JFS파일 시스템에서 사용하는 기능을 지원하고 있다.

레드햇은 제외한 대다수 기업용 리눅스 배포판에서는 BTRFS기반으로 파일 시스템을 지원한다. 앞서 이야기 하였지만, 레드햇은 RHEL 7이후로 BTRFS를 제공 및 지원하지 않는다.

레드햇이 BTRFS를 지원하지 않는 이유는 여러가지가 있지만, 제일 큰 이유 중 하나는 BTRFS 리더 개발자가 오라클 출신, 그리고 아직 BTRFS가 성능 및 안정적으로 XFS보다 부족하다고 판단하였다.

**현재 RHEL 10버전부터 다시 선택사항으로 지원한다.**



# BTRFS 설명

BTRFS로 넘어오기 전까지, 바닐라 리눅스 커널 및 커뮤니티에서는 차세대 파일 시스템에 대한 많은 고민이 있었다. EXT4이후 ReiserFS를 수세에서 개발을 하였으나, ReiserFS의 리더 개발자인 Hans Reiser가 부인 살해 혐의로 체포가 된 이후로, 더 이상 커뮤니티에서 개발이 이루어지지 않았다.

그 이후로 BTRFS가 EXT4파일 시스템 차세대로 이어받게 됨.

링크:

<https://lists.opensuse.org/archives/list/factory@lists.opensuse.org/thread/EGZIA6LR23EF3PE3RTSG4LQ0NY75TSXM/>



# BTRFS 소개

BTRFS의 주요 기능은 다음과 같다.

- 범위 기반 파일 저장(최대 파일 크기  $2^{64}$ )
- 작은 파일의 공간 효율적인 패킹
- 공간 효율적인 인덱스 디렉터리
- 동적 inode 할당
- 쓰기 가능한 스냅샷
- 하위 볼륨(별도의 내부 파일 시스템 루트)



# BTRFS 소개

- 객체 수준 미러링 및 스트라이핑
- 데이터 및 메타데이터에 대한 체크섬(여러 알고리즘 사용 가능)
- 압축(여러 알고리즘 사용 가능)
- 재링크, 중복 제거
- 스크럽(온라인 체크섬 확인)
- 계층적 할당량 그룹(하위 볼륨 및 스냅샷 지원)
- 여러 RAID 알고리즘을 사용하여 여러 장치 지원 통합





# BTRFS 소개

- 오프라인 파일 시스템 확인
- 효율적인 증분 백업 및 FS 미러링(송수신)
- 다듬기/폐기
- 온라인 파일 시스템 조각 모음
- SWAP 파일 지원
- 구역 모드
- 읽기/쓰기 메타데이터 확인
- 온라인 크기 조정(축소, 증가)

릴리즈 마다 조금씩 기능 변경이 있을 수 있으니 커널 업데이트 시 변경사항 확인이 필요.

<https://docs.kernel.org/filesystems/BTRFS.html>

현재 RHEL 10에서의 XFS가 BTRFS의 라이브 기능을 점점 지원하고 있다.

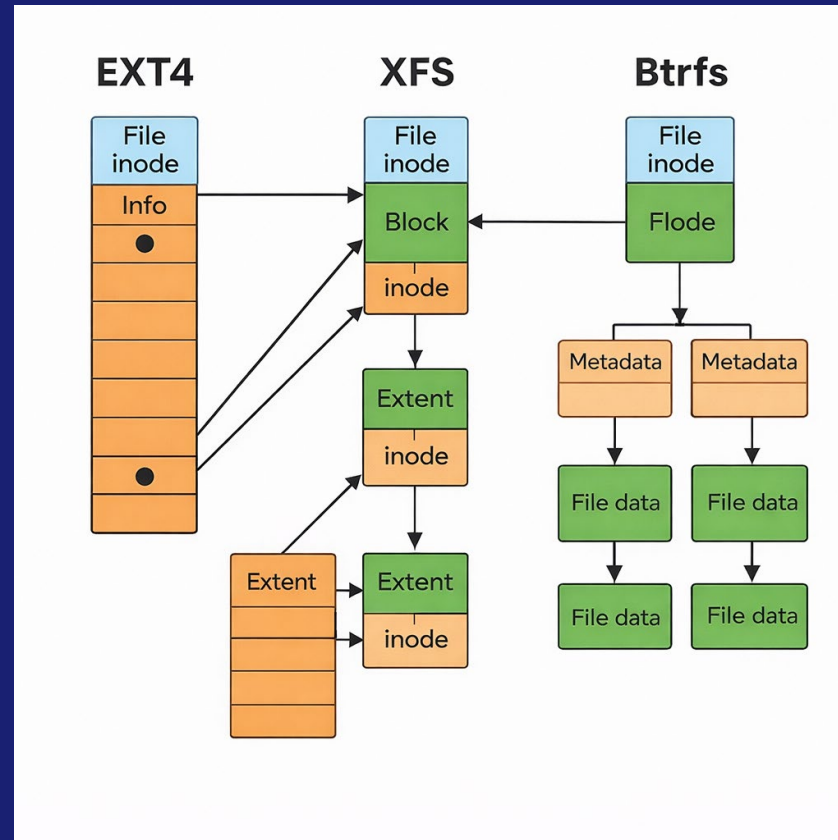


# 파일 시스템 사양 비교

| 파일 시스템    | EXT4        | XFS          | BTRFS         |
|-----------|-------------|--------------|---------------|
| 아키텍처      | 해쉬형태 B-TREE | B+ TREE(고정형) | 확장가능한 B+ TREE |
| 릴리즈       | 2006        | 1994         | 2009          |
| 최대 볼륨크기   | 1엑사 바이트     | 8엑사 바이트      | 16엑사 바이트      |
| 최대 파일크기   | 16테라 바이트    | 8엑사 바이트      | 16엑사 바이트      |
| 최대 파일개수   | 4조          | 2^64         | 2^64          |
| 최대 이름길이   | 255바이트      | 255바이트       | 255바이트        |
| 속성 및 확장속성 | 지원          | 지원           | 지원            |
| 투명 압축     | 아니요         | 아니요          | 네             |
| 투명 암호화    | 아니요         | 아니요          | 네             |
| COW 지원    | 아니요         | 네(커널 모듈)     | 네(빌트인)        |
| 스냅샷       | 아니요         | 네(커널 모듈)     | 네(빌트인)        |



# EXT4/XFS/BTRFS 구조비교



# 파일 시스템 비교

|          | BTRFS                                                            | XFS                                                                                            |
|----------|------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| 호환성      | BTRFS 파일 시스템은 자체적인 RAID기능을 가지고 있습니다.                             | XFS는 여러 장치에 드라이버 혹은 파티션을 구성할 수 없습니다. 그 이유는 XFS는 멀티 장치 구성을 지원하지 않습니다.                           |
| LVM지원    | BTRFS는 자체적으로 LVM기능을 제공합니다. 이를 통해서 subvolume를 손쉽게 구성이 가능하다.       | XFS는 LVM기능을 지원하지 않기 때문에, LVM2기반으로 각기 다른 장치 및 파티션을 묶는다. 이를 위해서 dm-raid, mdadm와 같은 도구를 통해서 구성한다. |
| 크기 변경    | BTRFS 파일 시스템은 마운트가 되어 있어도 파일 시스템 확장 및 축소가 실시간으로 가능하다.            | XFS 마운트가 되어 있는 상태에서 확장은 가능하나, 축소는 불가능하다.                                                       |
| 압축옵션     | 파일시스템 자체로 압축 기능을 제공 합니다. 이를 통해서 디렉터리 및 파일에 대해서 좀 더 효율적으로 공간을 활용. | XFS는 파일 시스템 수준에서 압축 기능을 제공하지 않음.                                                               |
| 부분 블록 할당 | BTRFS 파일 시스템은 남은 블록 영역에 데이터 할당이 가능하다.                            | XFS 파일 시스템은 이 기능을 제공하지 않는다.                                                                    |



# 파일 시스템 비교

|       | BTRFS                                                  | XFS                                                        |
|-------|--------------------------------------------------------|------------------------------------------------------------|
| 꼬리 보관 | BTRFS는 파일 시스템에 패킹을 지원한다.                               | XFS는 지원하지 않는다.                                             |
| 블록크기  | 이 파일 시스템은 다양한 블록 크기를 제공한다.<br>이를 통해서 디스크 공간 활용성이 높아진다. | XFS는 기본적으로 블록 크기는<br>512b~64kb사이에서 조정이 가능하다.<br>기본값이 4k이다. |



# 파일시스템 랩 및 고급

XFS(vdo, stratisd)

BTRFS

# BTRFS/LVM2 랩

수세 리눅스는 OS영역을 포함하여, 기본적으로 BTRFS를 사용하고 있다.

BTRFS는 오라클의 ZFS와 동일하게 Pool기반으로 Filesystem 관리 및 제공한다. BTRFS를 실험 및 사용하기 위해서는 최소 2개 이상의 디스크가 필요하다.

실험용 디스크 3개가 필요하다. 가상머신에 아래처럼 디스크를 추가한다.



# BTRFS/LVM2 준비사항

1. /dev/sdb, 10G
2. /dev/sdc, 10G
3. /dev/sdd, 10G
4. /dev/sde, 10G
5. /dev/sdf, 10G
6. /dev/sdg, 10G
7. /dev/sdh, 10G
8. /dev/sdi, 10G, LVM2





# 수세에서 XFS

수세 리눅스는 **XFS Progs 5.0**를 지원한다. 기존에 사용하던 XFS 영역을 그대로 마운트 및 사용이 가능하다. 현재 레드햇 계열의 배포판은 XFS 5.19버전을 사용하고 있기 때문에, 마운트 시 문제는 없다.

이미 사용중인 XFS파일 시스템은 BTRFS으로 전환은 가능하다. 다만, 안전성 이유로 권장하지 않는다.

<https://fedoramagazine.org/convert-your-filesystem-to-BTRFS/>

XFS파일 시스템은 본래 IRIX에서 사용하던 파일 시스템이며, I/O에서 최대 효율 및 성능을 끌어내기 위해서 만들어진 파일 시스템이다. B-TREE형식을 제공하고 있기 때문에, 빠른 검색이 가능하다.

다만, 기업용 파일 시스템에서 요구하는 실시간 파일 시스템 검사 및 디스크 풀 기능을 네이티브로 지원하지 않는다.

```
Suse]# btrfs-convert
Supported filesystems:
    ext2/3/4: yes
    reiserfs: yes
```



# 수세에서 XFS+LVM2

**XFS**는 기본적으로 디스크 풀 및 레이드와 같은 선형 기능을 제공하지 않는다.

**EXT2/3/4** 파일 시스템에서 사용하였던 LVM2기반으로 디스크를 묶어서 구성한다. 수세 리눅스에서도 LVM2 패키지를 제공하기 때문에 호환성을 제공한다. 디바이스 매퍼와 같이 사용하는 경우, 다음 패키지도 같이 설치해야 한다.

```
Suse]# rpm -qa lvm2  
Suse]# zypper install -y lvm2
```



# 수세에서 XFS+LVM2

LVM2 locking manager, 관리자가 필요한 경우 아래 패키지를 설치해야 한다. 이 부분은 페이스메이커 사용 시, 사용한다.

LVM2기반의 Locking 경우에는 레드햇과 동일하게, 더 이상 락킹(Locking) 타입을 명시하지 않는다. 데몬 서비스에 따라서 자동적으로 락킹 형식이 결정된다.

수세 리눅스 경우에는 BTRFS 기반을 사용하고 있기 때문에, LVM2+XFS 혹은 LVM2+EXT4기반으로 사용하지 않으면 별도로 락킹 설정이 필요 없다.

```
Suse]# zypper install lvm2-lockd
```



# 수세에서 XFS/EXT4/LVM2 LAB

```
Suse]# gdisk /dev/sdi  
part1, 5G, 8e00  
part2, 5G, 8e00  
Suse]# zypper install -y xfsprogs  
Suse]# pvcreate /dev/sdi1  
Suse]# vgcreate vg_fss /dev/sdi1  
Suse]# lvcreate -n lv_xfs -l 50%Free vg_fss  
Suse]# lvcreate -n lv_ext4 -l 50%Free vg_fss  
Suse]# mkfs.xfs /dev/vg_fss/lv_xfs  
Suse]# mkfs.ext4 /dev/vg_fss/lv_ext4
```



# 수세에서 KVDO(수동 컴파일)

```
Suse]# zypper install -y kernel-syms kernel-devel kernel-source
Suse]# cd /usr/src/linux
Suse]# make oldconfig && make prepare
Suse]# git clone https://github.com/dm-vdo/kvdo.git
Suse]# cd ~/vdo/
Suse]# make -C /usr/src/linux M=`pwd`
Suse]# make -C /usr/src/linux M=`pwd` modules_install
```



# 수세에서 VDO

레드햇 계열 배포판에서 제공하는 **vdo** 저장소 형식도 사용이 가능하다. vdo를 사용하기 위해서 배포판에 반드시 LVM2가 설치 및 동작이 되어야 한다.

이전 레드햇 리눅스의 vdo버전은 독립적인 서비스로 제공 하였으나, 현재는 LVM2와 통합이 되었다.

```
Suse]# zypper install -y libbd_vdo2 libblkid-devel libuuid-devel lvm2-devel
git
Suse]# git clone https://github.com/dm-vdo/vdo.git
Suse]# cd vdo
Suse]# make && make install
Suse]# pvcreate /dev/sdi2
Suse]# vgcreate testvdo /dev/sdi2
Suse]# lvcreate --type vdo -n lvvdo -l 100%Free testvdo
```



# 수세에서 VDO

```
Redhat]# dnf search vdo
Redhat]# dnf install vdo kmod-vdo vdo-support hexedit
Redhat]# modprobe kvdo
modprobe: ERROR: could not insert 'kvdo': Key was rejected by service
Redhat]# pvcreate /dev/sdd
Redhat]# vgcreate vod_vg /dev/sdd
Redhat]# lvcreate --type vdo -L 100%Free --name vdo_lv vod_vg
Redhat]# lsmod | grep vdo
Redhat]# vod vdo stats
```



# DUP/RAID/SNAPSHOT

|                     | XFS             | BTRFS            | ZFS                                                    |
|---------------------|-----------------|------------------|--------------------------------------------------------|
| 압축                  | VDO기반에서 제공      | inline / offline | inline                                                 |
| 중복제거(Deduplication) | offline         | inline / offline | inline                                                 |
| 외부 메타데이터            | 네               | 아니요              | 네, 외부장치 명시 가능                                          |
| 읽기/쓰기 캐쉬            | LVO/B-Cache 레이어 | LVO/B-Cache 레이어  | L2arc / slog                                           |
| 메모리 비트 로테이션 보호      | 아니요             | 네                | 네                                                      |
| 레이드 지원 형식           | 아니요             | Raid 1+10        | Raid<br>1+raidz(5)+raidz2(6)+<br>raidz3(7)+10+50+60... |
| 레이드 재배치 지원          | -               | 네                | 아니요                                                    |
| 중복제거 기능 끄기          | 네               | 네                | 아니요                                                    |
| 가상머신/컨테이너 스냅샷       | 가상머신에서만 가능      | 가상머신/컨테이너        | 가상머신/컨테이너                                              |





# VDO에서 제공하는 읽기/쓰기

파일 시스템별로 제공하는 **deduplication** 기능은 보통 아래와 같이 나누어진다. BTRFS는 위의 두 가지 형식의 중복제거 기능을 제공한다. 이를 역할별로 구별하면 아래와 같이 기능을 나눌 수 있다.

## **in-band**

새롭게 쓰이는 데이터가 중복이 되는지 디스크에 쓰기전에 확인한다.

## **out-of-band**

중복 제거를 하기 위해서 사용자 애플리케이션에서 적극적으로 확인 후 중복제거 한다.



# 데이터블록 중복제거

## File based deduplication

여러 개의 파일이 있는 경우, 특정 목록에서 파일들을 가져와서 중복된 항목을 확인한다. 확인이 완료가 되면 원본 파일과 링크를 구성 후, 사용할 수 있도록 한다. 해시 데이터베이스에서 지속적으로 확인하여 중복된 내용을 제거 및 삭제한다. 최대 장점은 손쉽게 확장관리가 가능하다.

**BTRFS도 이 방식을 지원한다.**

## Block based deduplication

파일 시스템에 저장된 내용을 확인하여, 본 파일, 예를 들어서 이미지 및 데이터 파일을 확인한다. 중복된 파일이 확인이 되면, 파일 시스템의 해시 데이터베이스에 기록을 남긴 후 제거한다. 레드햇의 VDO XFS는 이 방식을 사용한다.

**BTRFS도 이 방식도 지원한다.**



# DUP/RAID/SNAPSHOT

DUP에는 두 가지 의미가 있음.

1. Deduplication

2. Duplication

1번은 중복 방지의 DUP이며, 2번은 복제 개념의 DUP.

| 이름                | 파일 기반 | 블록장치 기반 | 확장 가능 |
|-------------------|-------|---------|-------|
| <u>BEES</u>       | No    | Yes     | Yes   |
| <u>duperemove</u> | Yes   | No      | Yes   |



# BTRFS PROFILE 1

| 프로파일   | 설명                                                                                                              | 디스크 개수   | 공간 효율     | 오류 허용범위         |
|--------|-----------------------------------------------------------------------------------------------------------------|----------|-----------|-----------------|
| raid6  | 2개의 디스크를 중복으로 사용하는 스트라이프 모드입니다. 일부 워크로드에서 성능을 향상시킬 수 있습니다.                                                      | 4개 혹은 이상 | $(N-2)/N$ | 2 disk failures |
| raid10 | 향상된 성능과 중복성을 위한 RAID1+RAID0 모드의 조합.                                                                             | 4개 혹은 이상 | 50%       | 1 disk failure  |
| DUP    | DUP는 중복을 의미합니다. 이렇게 하면 동일한 디스크에 두 개의 복사본이 존재하게 됩니다. SINGLE 모드와 같이 하나 또는 여러 드라이브에서 사용할 수 있지만 디스크 오류로부터 보호하지 않는다. | 1개 혹은 이상 | 50%       | Some (*)        |
| SINGLE | 단일 디스크 또는 스펠 볼륨(A.K.A. Just a Bunch Of Drives - JBOD)의 경우                                                       | 1개 혹은 이상 | 100%      | None            |



# BTRFS PROFILE 2

| 프로파일    | 설명                                                                   | 디스크 개수    | 공간 효율 | 오류 허용범위         |
|---------|----------------------------------------------------------------------|-----------|-------|-----------------|
| raid0   | SINGLE과 유사하지만 모든 드라이브의 병렬 스트라이프에 데이터가 할당됩니다. 일부 워크로드에서 성능을 향상할 수 있음. | 디스크 2개 이상 | 100%  | None            |
| raid1   | DUP와 유사하지만 2개의 복사본을 각각 별도의 디스크에 저장합니다.                               | 디스크 2개 이상 | 50%   | 1 disk failure  |
| raid1c3 | 별도의 디스크에 3개의 복사본을 저장합니다.                                             | 3개 혹은 이상  | 33.3% | 2 disk failures |
| raid1c4 | 별도의 디스크에 4개의 복사본을 저장합니다.                                             | 4개 혹은 이상  | 25%   | 3 disk failures |



# BTRFS(mkfs.btrfs)

**mkfs.btrfs** 에서 자주 사용하는 옵션은 다음과 같다.

- **-L**: 레이블, 생성된 디스크 및 블록에 디스크 이름을 명시한다.
- **-m**: 메타정보 프로파일.
- **-d**: 데이터 프로파일은 다음과 같이 지원한다.

```
mkfs.btrfs -L test-pool -d dup -m dup
```

```
mkfs.btrfs -L test-pool -d raid -m raid1
```

```
mkfs.btrfs -L test-pool -M dup
```



# BTRFS POOL(DUP)

```
lsblk
```

```
mkfs.btrfs -m dup -L btrfs-pool-dup -f /dev/sdb /dev/sdc
```

```
blkid /dev/sdb
```

```
UUID=<UUID>
```

처음에 구성하는 POOL은 DUP으로 구성한다.  
메타 정보를 중복으로 구성하여, 장애 메타 정보에 장애가 발생  
하여도 사용이 가능하도록 한다.

```
mkdir -p /btrfs-pool-dup
```

```
mount UUID=fc657519-23a0-453b-8a45-9f3081db9931 /btrfs-pool-dup
```

```
mount -l | grep /dev/sdb
```



# BTRFS POOL(DUP)

```
btrfs device usage /btrfs-pool-dup/  
dd if=/dev/zero of=/btrfs-pool-dup/test-dup-2.data bs=100M count=20  
dd if=/dev/zero of=/btrfs-pool-dup/test-dup-4.data bs=100M count=40  
btrfs filesystem usage /btrfs-pool-dup  
btrfs device usage /btrfs-pool-dup/
```





# BTRFS POOL(RAID0)

```
lsblk
```

```
mkfs.btrfs -m raid0 -L btrfs-pool-raid0 -f /dev/sdf /dev/sdg
```

```
blkid /dev/sdf
```

```
UUID=<UUID>
```

```
mkdir -p /btrfs-pool-raid0
```

```
mount UUID=8718ec96-a035-425d-97da-5d94cf5925b4 /btrfs-pool-raid0
```



# BTRFS POOL(RAID0)

```
mount -l | grep /dev/sdd
```

```
dd if=/dev/zero of=/btrfs-pool-raid0/test-raid0-2.data bs=100M count=20
```

```
dd if=/dev/zero of=/btrfs-pool-raid0/test-raid0-4.data bs=100M count=40
```

```
btrfs device usage /btrfs-pool-raid0/
```

```
btrfs filesystem usage /btrfs-pool-raid0
```

```
btrfs subvolume show /btrfs-pool-raid0
```



# BTRFS POOL(RAID1)

```
lsblk
```

```
mkfs.btrfs -m raid1 -L btrfs-pool-raid1 -f /dev/sdd /dev/sde
```

```
blkid /dev/sdd
```

BTRFS기반으로 RAID-1 파일 시스템 생성 및 구성.

```
UUID=<UUID>
```

```
mkdir -p /btrfs-pool-raid1
```

```
mount UUID=8718ec96-a035-425d-97da-5d94cf5925b4 /btrfs-pool-raid1
```



# BTRFS POOL(RAID1)

```
mount -l | grep /dev/sdd
```

```
dd if=/dev/zero of=/btrfs-pool-raid1/test-raid1-2.data bs=100M count=20
```

```
dd if=/dev/zero of=/btrfs-pool-raid1/test-raid1-4.data bs=100M count=40
```

```
btrfs device usage /btrfs-pool-raid1/
```

```
btrfs filesystem usage /btrfs-pool-raid1
```

```
btrfs subvolume show /btrfs-pool-raid1
```



# BTRFS POOL(SINGLE) 1

```
lsblk
```

```
mkfs.btrfs -m single -L btrfs-pool-single -f /dev/sdh
```

```
blkid /dev/sdh
```

```
UUID=<UUID>
```

```
mkdir -p /btrfs-pool-single/
```

```
mount UUID=72e2f9a6-bf0a-471d-a142-ace745004746 /btrfs-pool-single
```

```
mount -l | grep /dev/sdh
```



## BTRFS POOL(SINGLE) 2

```
dd if=/dev/zero of=/btrfs-pool-single/test-single-2.data bs=100M count=20
dd if=/dev/zero of=/btrfs-pool-single/test-single-4.data bs=100M count=40
btrfs device usage /btrfs-pool-single/
btrfs filesystem usage /btrfs-pool-single
btrfs subvolume show /btrfs-pool-single
```



# Single TO DUP

```
btrfs balance start -mconvert=DUP /btrfs-pool-single  
btrfs device usage /btrfs-pool-single/
```



# SUBVOLUME DUP(SUBVOL/SNAP)

서브 볼륨 읽기 전용 스냅샷 구성.

```
btrfs filesystem usage /btrfs-pool-dup/  
btrfs subvolume create /btrfs-pool-dup/sub-dup  
btrfs subvolume show /btrfs-pool-dup/sub-dup  
btrfs subvolume list /btrfs-pool-dup/  
ID 263 gen 19 top level 5 path sub-single  
  
touch /btrfs-pool-dup/sub-dup/test-data.txt  
btrfs subvolume snapshot -r /btrfs-pool-dup/sub-dup/ /btrfs-pool-dup/snap-  
sub-dup
```





# SUBVOLUME RAID0(SUBVOL/SNAP)

읽기 전용 스냅샷 생성.

```
btrfs filesystem usage /btrfs-pool-raid0
btrfs subvolume create /btrfs-pool-raid0/sub-raid0
btrfs subvolume show /btrfs-pool-raid0/sub-raid0
btrfs subvolume list /btrfs-pool-raid0
ID 263 gen 19 top level 5 path sub-raid0

touch /btrfs-pool-raid0/sub-raid0/test-data.txt
btrfs subvolume snapshot -r /btrfs-pool-raid0/sub-raid0/ /btrfs-pool-
raid0/snap-sub-raid0
```



# SUBVOLUME RAID1(SUBVOL/SNAP)

서브 볼륨 읽기 전용 스냅샷 구성.

```
btrfs filesystem usage /btrfs-pool-raid1
btrfs subvolume create /btrfs-pool-raid1/sub-raid1
btrfs subvolume show /btrfs-pool-raid1/sub-raid1
btrfs subvolume list /btrfs-pool-raid1
ID 263 gen 19 top level 5 path sub-raid1

touch /btrfs-pool-raid1/sub-raid1/test-data.txt
btrfs subvolume snapshot -r /btrfs-pool-raid1/sub-raid1/ /btrfs-pool-raid1/snap-sub-raid1
```



# SUBVOLUME SINGLE(SUBVOL/SNAP)

서브 볼륨 읽기 전용 스냅샷 구성.

```
btrfs filesystem usage /btrfs-pool-single
btrfs subvolume create /btrfs-pool-single/sub-single
btrfs subvolume show /btrfs-pool-single/sub-single
btrfs subvolume list /btrfs-pool-single
ID 263 gen 19 top level 5 path sub-single

touch /btrfs-pool-single/sub-single/test-data.txt
btrfs subvolume snapshot -r /btrfs-pool-single/sub-single/ /btrfs-pool-
single/snap-sub-single
```



# BTRFS SWAP에서 지원

**BTRFS**에서는 다음과 같은 스왑 파일 및 블록 장치를 지원한다. BTRFS에서는 블록 형태의 스왑 장치보다는 파일 형태의 스왑 장치를 더 추천한다.

- `filesystem` - must be only single device
- `filesystem` - must have only single data profile
- `subvolume` - cannot be snapshotted if it contains any active swapfiles
- `swapfile` - must be preallocated (i.e. no holes)
- `swapfile` - must be NODATACOW (i.e. also NODATASUM, no compression)



# SUBVOLUME FOR SWAP UNDER BTRFS 6

서브 볼륨 기반으로 SWAP를 생성이 가능하다.

```
btrfs subvolume create /subvol-swap/  
truncate -s 0 /subvol-swap/swap  
chattr +C /subvol-swap/swap  
fallocate -l 2G /subvol-swap/swap  
chmod 0600 /subvol-swap/swap  
mkswap /subvol-swap/swap  
swapon /subvol-swap  
swapon -s
```



# SUBVOLUME FOR SWAP UNDER BTRFS 6

BTRFS 6.x latest 이후에 스왑 파일 생성 방법 및 커널에서 스왑 정보 확인.

```
btrfs filesystem mkswapfile --size 2G swapfile
```

```
swapon swapfile
```

```
swapon -s
```

```
cat /proc/swaps
```

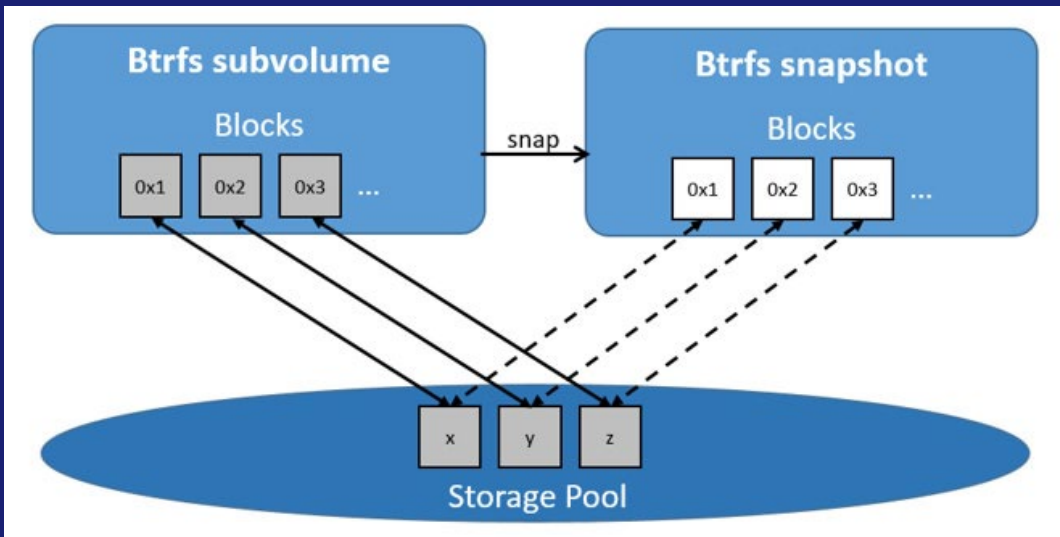


# BTRFS SUBVOLUME/SNAPSHOT

BTRFS는 서브 볼륨을 통해서 하나 이상의 디스크를 하나의 풀(Pool) 구성이 가능하다.

풀에서 사용자가 원하는 크기로 즉각 장치 생성 및 파일 시스템 구성이 가능하다. 이를 기반으로 스냅샷 및 백업이 가능하다.

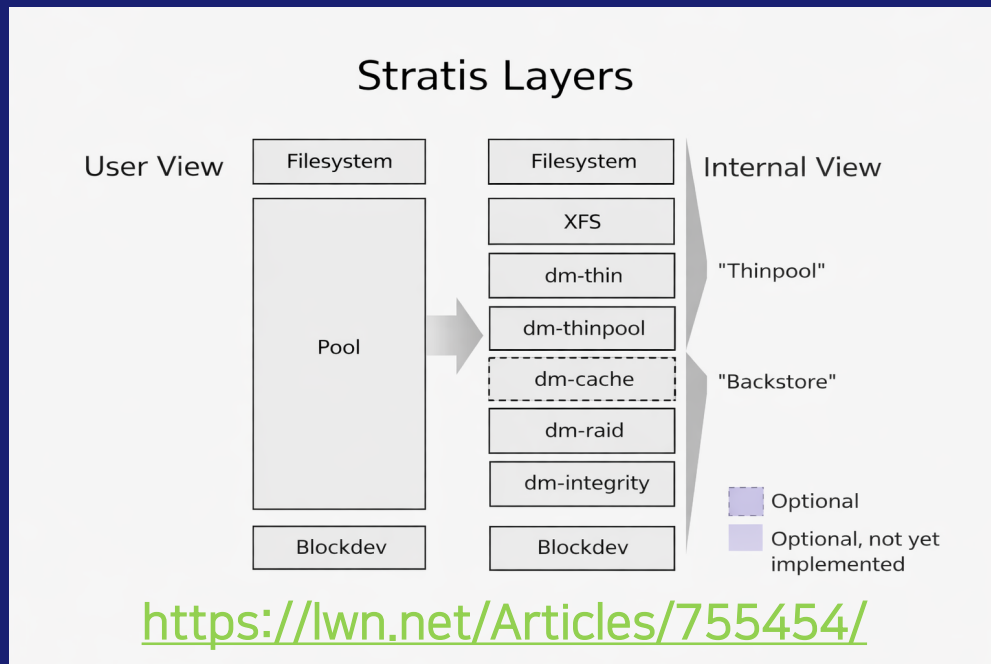
앞서 이야기한 XFS기반으로 구성하는 경우, 별도의 블록 소프트웨어 계층을 설치 및 구성이 필요하다.



# Stratis 아키텍처

레드햇은 Stratis를 통해서 XFS기반으로 Pool based Block Device를 구현한다.

이 기능은 기존에 사용하던 XFS 파일 시스템에 영향없이 애플리케이션 기반으로 별도의 블록장치 영역을 만들어서 추상적으로 구성한다.





# Snapper기반 스냅샷 관리

구성 및 생성된 스냅샷 목록 확인 및 변경사항 확인 방법은 아래 명령어로 가능하다.

```
snapper list-configs
```

```
snapper list
```

```
yast2 snapper
```

```
snapper list
```

```
snapper status 2..3
```

```
snapper diff 2..3
```

```
snapper undochange 2..3
```



# Snapper with EXT4

EXT4는 LVM2기반으로 레이어를 구성한다.

```
pvcreate /dev/sdi2
vgcreate testvg-snapper /dev/sdi2
lvcreate --thin testvg-snapper -l 100%Free
lvcreate --thin -n testlv-snapper -l 100%Free testvg-snapper
mkfs.ext4 /dev/testvg-snapper/lvol1
mkdir -p /ext4-storage
mount -t ext4 /dev/testvg-snapper/testlv-snapper /ext4-storage
```



# Snapper with BTRFS

스냅샷 생성은 아래 명령어로 가능하다. **BTRFS**는 이미지 레이어를 자체적으로 제공.

```
snapper -c ext4-storage --fstype="lvm(ext4)" create-config /ext4-storage
snapper -c home list
snapper -c home status 1..0
snapper -c home create --description "before changed something in the home"
```



# BTRFS SNAPSHOT

앞에서 생성하였지만, 간단하게 아래와 같이 BTRFS기반으로 snapshot생성 및 관리가 가능하다.

```
btrfs subvolume snapshot /data /backup-data  
btrfs filesystem du /data  
btrfs filesystem df /data
```



# /etc/fstab for BTRFS

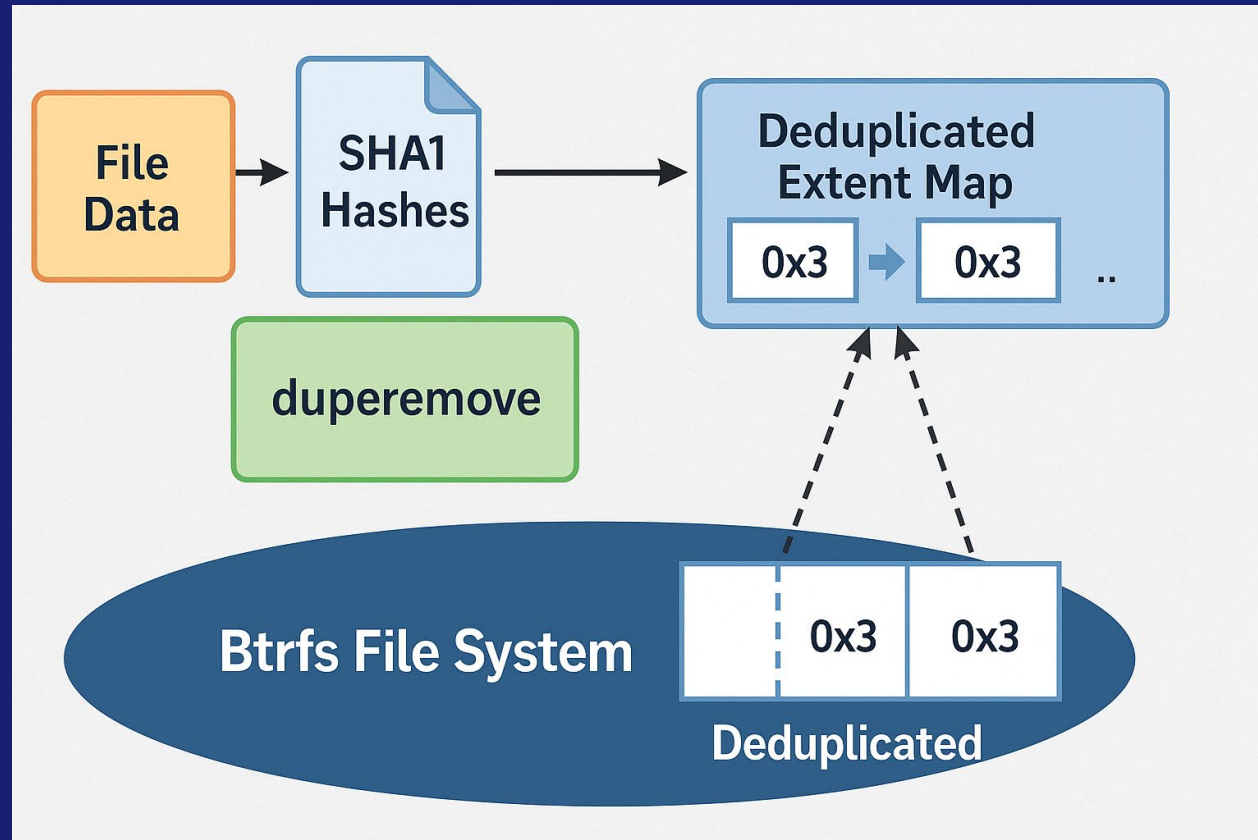
부팅 시 자동 마운트를 위해서 아래와 같이 구성이 가능하다.

```
btrfs filesystem show
vi /etc/fstab/
UUID=fc657519-23a0-453b-8a45-9f3081db9931    /btrfs-pool-dup            btrfs
defaults                                     0    0
btrfs subvolume create /btrfs-pool-dup/subvol-dup
btrfs subvolume list /btrfs-pool-dup/subvol-dup

vi /etc/fstab
UUID=fc657519-23a0-453b-8a45-9f3081db9931    /btrfs-pool-dup/subvol-dup
btrfs    subvol=subvol-dup                    0    0
```



# Duperemove FOR BTRFS



# Duperremove, 중복제거

블록 중복 제거(압축)하기 위해서 아래와 같이 실행한다.

```
zypper install duperremove
mkdir -p /btrfs-dups/
dd if=/dev/zero of=test-single-4.data bs=100M count=20
cp test-single-4.data test-single-4-1.data
duperremove -rh /btrfs-dups/
duperremove -hr -d /btrfs-dups
duperremove -d -h -r /mnt/rootvol/@
chrt -i 0 duperremove -A -h -d -r /btrfs-dups -v -b128k --dedupe-
options=noblock,same --lookup- extents=yes --io-threads=1
```



# Duperremove, 중복제거

옵션(**duperremove**)은 다음과 같은 기능을 한다.

| 옵션 | 설명                              |
|----|---------------------------------|
| -r | 디렉터리까지 포함하여 검색                  |
| -d | de-duplication extents에 적용 및 반영 |
| -h | 사람이 읽기 편한 단위로 출력                |





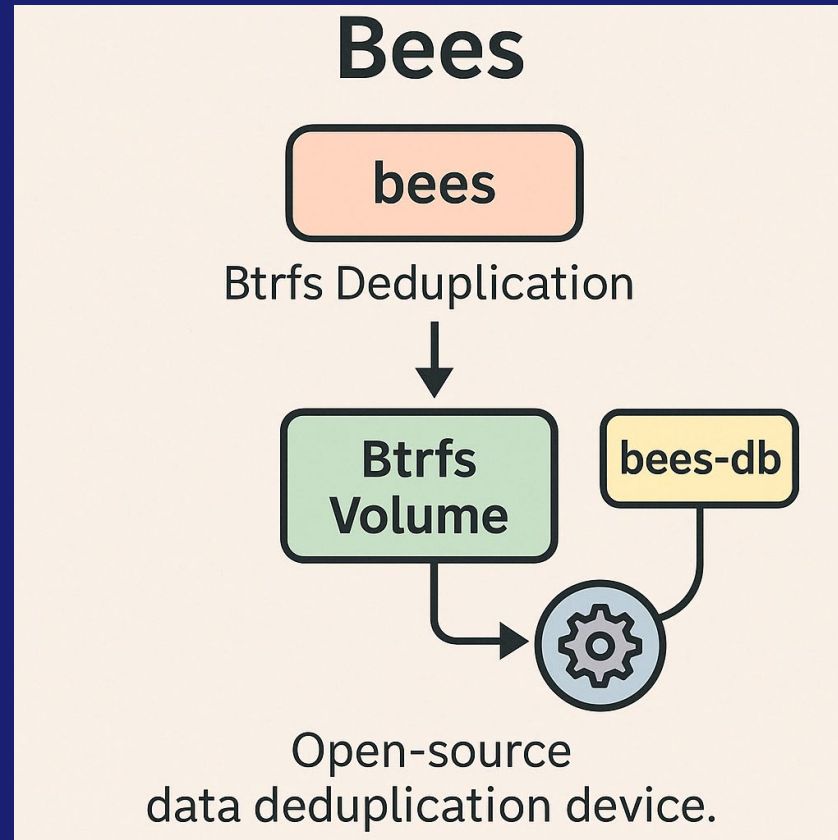
# Duperremove, 중복제거

옵션(chrt)은 다음과 같은 기능을 한다.

| 옵션                                         | 설명                                       |
|--------------------------------------------|------------------------------------------|
| <code>chrt -i 0</code>                     | 낮은 실시간 우선순위로 실행                          |
| <code>-A</code>                            | 전체 디스크 스캔                                |
| <code>--dedupe-options=noblock,same</code> | I/O block locking 안 함, 정확히 같은 블록만 dedupe |
| <code>--lookup-extents=yes</code>          | 이미 공유된 extent도 lookup                    |
| <code>--io-threads=1</code>                | 단일 I/O 쓰레드로 부하 최소화                       |



# Bees for BTRFS



<https://github.com/Zygo/bees>

# BEES FOR BTRFS

BEES 설치 및 구성은 다음과 같이 한다.

```
zypper install bees
cp /etc/bees/beesd.conf.sample /etc/bees/beesd.conf
vi /etc/bees/beesd.conf
UUID=<VOLUME_UUID>
DB_SIZE=$((1024*1024*1024))

bees <UUID>
systemctl enable --now beesd@<UUID>.service
bees -v -d /dev/sdX1 -s /var/lib/bees/sdX1
```



<https://github.com/Zygo/bees>

# BEES FOR BTRFS

서비스 파일은 다음과 같이 생성 및 구성이 가능하다.

```
vi /etc/bees/bees@mnt-data.conf
```

```
BEES_DEV=/dev/sdb1
```

```
BEES_PATH=/mnt/data
```

```
BEES_STATE=/var/lib/bees/sdb1
```

```
BEES_LOG=/var/log/bees.log
```

```
BEES_OPTIONS="-v"
```

```
systemctl status bees@mnt-data
```

```
tail -f /var/log/bees.log
```

```
journalctl -u bees.service -fl
```



# BEES FOR BTRFS

BEES 옵션은 다음과 같다.

| 옵션 | 설명                                     |
|----|----------------------------------------|
| -v | verbose 출력                             |
| -d | dedup 대상 장치 (mount된 Btrfs 볼륨의 실제 디바이스) |
| -s | 상태 저장 경로 (bees가 어디까지 처리했는지 저장)         |



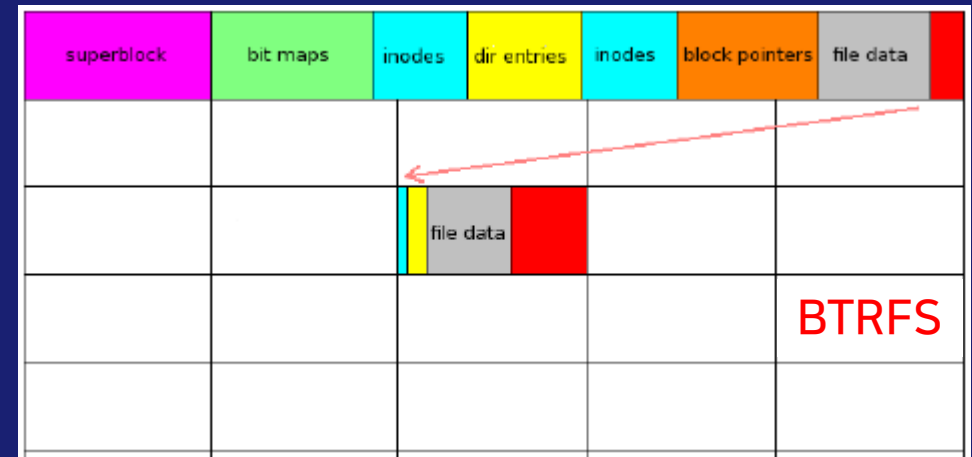
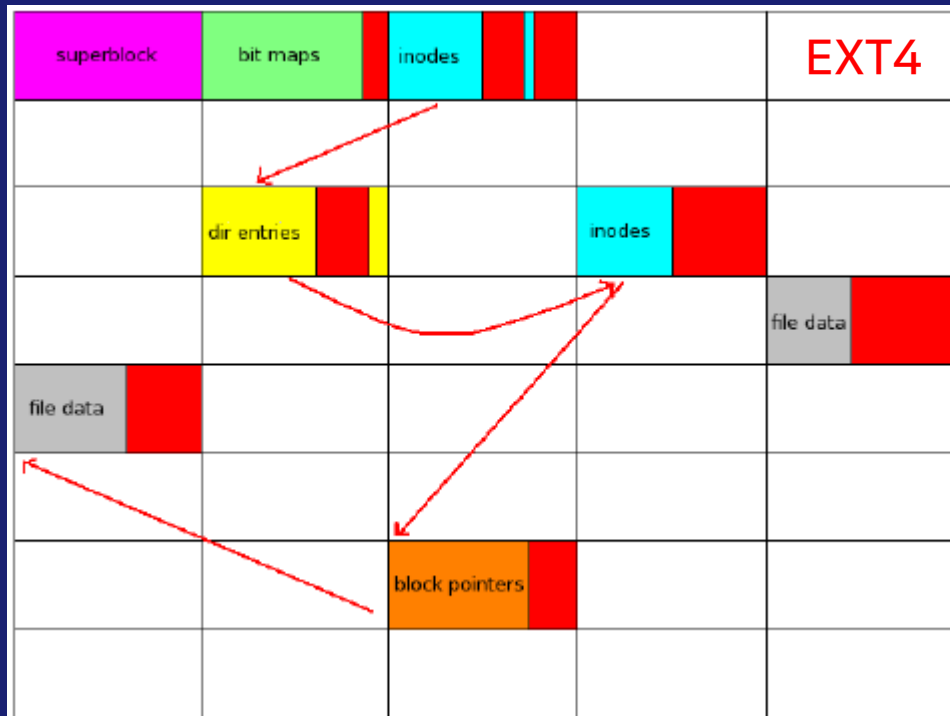
# BTRFS PACKING

**Tail Packing** 파일 시스템 혹은 기능마다 다음과 같은 차이가 있다. 이 기능은 작은 파일이나 파일의 마지막 부분(테일)을 효율적으로 저장하여 디스크 공간을 절약하는 기술입니다.

| 파일 시스템  | Tail Packing | 설명                                                       |
|---------|--------------|----------------------------------------------------------|
| Btrfs   | 지원           | 작은 파일을 메타데이터 블록에 인라인으로 저장하여 공간 효율성을 높입니다.                |
| XFS     | 미지원          | 큰 파일과 고성능에 최적화되어 있으며, Tail Packing 기능은 제공하지 않습니다.        |
| Stratis | 미지원          | XFS를 기반으로 하며, Tail Packing 기능은 포함되어 있지 않습니다.             |
| VD0     | 미지원          | 블록 수준 중복 제거 및 압축을 제공하지만, Tail Packing 기능은 없습니다.          |
| LVM2    | 미지원          | 논리 볼륨 관리를 위한 도구로, 파일 시스템 수준의 Tail Packing 기능은 제공하지 않습니다. |



# BTRFS PACKING(공간 아끼기)



# BTRFS/XFS/ZFS/EXT4

성능 비교를 하면 다음과 같다.

| 파일 시스템 | 주요 특징                                                                 |
|--------|-----------------------------------------------------------------------|
| EXT4   | 안정성과 호환성이 뛰어나며, 대부분의 리눅스 배포판에서 기본 파일 시스템으로 사용됩니다.                     |
| XFS    | 대용량 파일 처리에 최적화되어 있으며, 높은 쓰기 성능을 제공합니다.                                |
| Btrfs  | 스냅샷, 서브볼륨, 데이터 무결성 검사 등 다양한 고급 기능을 지원하지만, 일부 환경에서는 성능이 낮을 수 있습니다.     |
| ZFS    | 데이터 무결성과 복구 기능이 뛰어나며, 대규모 스토리지 환경에 적합하지만, 리소스 사용량이 많고 설정이 복잡할 수 있습니다. |





# BTRFS/XFS/ZFS/EXT4

| 색상    | 의미 (보통의 시각화 기준)                  |
|-------|----------------------------------|
| 빨강    | 가장 높은 수치 (경고, 최고값) 등으로 쓰이는 경우 많음 |
| 주황    | 높은 값이지만 덜 위급한 느낌                 |
| 노랑    | 중간 이상                            |
| 초록    | 정상 범위, 낮은 수치                     |
| 파랑    | 보통은 중간 이하                        |
| 회색/검정 | 기준치 또는 미측정/기본값                   |



# BTRFS 성능 테스트(sysbench)

동일한 조건으로 BTRFS에서 작업을 수행한다. **sysbench** 기반으로 사용 시 아래와 같이 실행.

```
zypper install sysbench
mkdir -p /mnt/testfs
mount /dev/sdX1 /mnt/testfs
cd /mnt/testfs
sysbench fileio --file-total-size=5G prepare
sysbench fileio \
  --file-total-size=5G \
  --file-test-mode=rndrw \
  --time=60 \
  --max-requests=0 \
  --num-threads=4 \
  run
sysbench fileio --file-total-size=5G cleanup
```

| 항목  | 설명                    |
|-----|-----------------------|
| TPS | 초당 트랜잭션 수 (성능 주요 지표!) |
| QPS | 초당 쿼리 수               |
| lat | 평균 지연 시간 (낮을수록 좋음)    |



# BTRFS 성능 테스트(fio)

동일한 조건으로 BTRFS에서 작업을 수행한다. **fio** 기반으로 사용 시 아래와 같이 실행.

```
zypper install fio
fio --name=fs-benchmark \
    --directory=/mnt/testfs \
    --size=2G \
    --numjobs=4 \
    --time_based \
    --runtime=60 \
    --rw=readwrite \
    --bs=128k \
    --group_reporting
```



# BTRFS BMT

동일한 조건으로 BTRFS에서 작업을 수행한다. **fio** 기반으로 사용 시 아래와 같이 실행.

```
zypper install fio
fio --name=fs-benchmark \
    --directory=/mnt/testfs \
    --size=2G \
    --numjobs=4 \
    --time_based \
    --runtime=60 \
    --rw=readwrite \
    --bs=128k \
    --group_reporting
```

--rw=readwrite: 읽기+쓰기 혼합  
--bs=128k: 블록 사이즈  
--numjobs=4: 동시 작업자 수



# CONVERT FOR BTRFS

BTRFS으로 전환이 가능한 파일 시스템은 아래와 같다.

1. EXT Series Filesystem
2. ReiserFS

XFS경우에는 변환도구가 공식적으로는 없다.

구조상 변환은 불가능하지 않으나, 레드햇은 현재 XFS2BTRFS변환도구 개발은 하고 있지 않으나, 페도라 그룹에서는 XFS to BTRFS에 대해서 지속적으로 언급이 되고 있다.

자세한 사항은 아래 링크 참조.

<https://fedoramagazine.org/convert-your-filesystem-to-btrfs/>

[https://archive.kernel.org/oldwiki/btrfs.wiki.kernel.org/index.php/Conversion\\_from\\_Ext3.html](https://archive.kernel.org/oldwiki/btrfs.wiki.kernel.org/index.php/Conversion_from_Ext3.html)



# CONVERT FOR BTRFS

BTRFS으로 전환이 가능한 파일 시스템은 다음과 같다.

| 기존          | 변환 지원 여부 | 변환 방식         | 기존 데이터 유지 | 비고                      |
|-------------|----------|---------------|-----------|-------------------------|
| ext2        | 지원       | btrfs-convert | 가능        | 가장 안정적인 변환 대상           |
| ext3        | 지원       | btrfs-convert | 가능        | 저널 제거 후 변환              |
| ext4        | 지원       | btrfs-convert | 가능        | 일부 경우 메타데이터 문제 발생 가능    |
| xfs         | 미지원      | -             | 불가능       | 직접 변환 불가, 별도 복사 필요      |
| reiserfs    | 부분적 지원   | -             | 불가능       | 커널 지원도 감소 추세            |
| zfs         | 미지원      | -             | 불가능       | 오픈ZFS → Btrfs 직접 변환 불가능 |
| NTFS, exFAT | 미지원      | -             | 불가능       | 복사만 가능                  |



# 정리

EXT4에서 BTRFS전환 시 다음과 같은 장단점이 발생한다.

| 항목          | EXT4 (기존)   | Btrfs (변환 후)                          | 변화 예상   | 비고                             |
|-------------|-------------|---------------------------------------|---------|--------------------------------|
| 순차 읽기/쓰기 속도 | 빠름 (높은 안정성) | 약간 느림 (메타데이터 복잡성)                     | ↓ 약간 감소 | 순차성능 중요하면<br>EXT4 유지 고려        |
| 랜덤 I/O 성능   | 평균 수준       | 약간 감소                                 | ↓ 소폭 감소 | 중소규모 파일 다량<br>처리 시 유의          |
| 메모리 사용량     | 낮음          | 중간~높음                                 | ↑ 증가    | Btrfs는 메타데이터<br>캐시 사용          |
| 공간 절약 (압축)  | 미지원         | 지원 (zstd/lzo)                         | ↑ 향상    | 압축 옵션 설정 필요<br>(compress=zstd) |
| 스냅샷 지원      | 없음          | 네이티브 지원 (btrfs subvolume<br>snapshot) | ↑ 강력함   | 백업, 롤백 용이                      |



# 정리

앞에 내용 계속 이어서...

| 항목        | EXT4 (기존)           | Btrfs (변환 후)         | 변화 예상 | 비고                     |
|-----------|---------------------|----------------------|-------|------------------------|
| 데이터 무결성   | 없음                  | 블록 수준 체크섬 지원         | ↑ 안정성 | silent corruption 방지   |
| RAID 지원   | 파일시스템 외부 (mdadm) 필요 | 내장 RAID0/1/10/5/6 지원 | ↑ 유연성 | RAID 5/6은 아직 실무선 주의 필요 |
| 파일 조각화 방지 | 상대적 낮음              | 심화될 수 있음             | ↓ 감소  | 자동 조각 모음 없음            |
| 유지보수 용이성  | 단순 구조               | 복잡한 내부 구조            | ↓ 복잡  | 문제 발생 시 복구 도구 부족       |





# 정리

| 사용 목적             | 추천 파일 시스템    |
|-------------------|--------------|
| 단순한 성능 중심 서버      | EXT4         |
| 스냅샷 + 무결성 + 압축 필요 | Btrfs        |
| RAID가 필요한 파일 시스템  | Btrfs or ZFS |
| 고성능 DB workload   | XFS / EXT4   |



백업

btrbk

Borg/ReAR

# BORG 소개

**BTRFS/XFS**에서 많이 사용하는 파일 소프트웨어는 **BORG**이다.

수세 리눅스 저장소에서 기본적으로 Borg제공하고 있다. 중복방지 기능이 있기 때문에 빠른 백업 및 작은 크기로 백업 관리가 가능하다.

레드햇 리눅스도 Borg를 지원한다. 다만, 사용하기 위해서는 EPEL저장소를 활성화 하면 사용이 가능하다.

BORG리눅스는 다음과 같은 기능을 제공하고 있다.



# BORG 지원

- Space efficient storage of backups.
- Secure, authenticated encryption.
- Compression: lz4, zstd, zlib, lzma or none.
- Mountable backups with FUSE.
- Easy installation on multiple platforms: Linux, macOS, BSD, ...
- Free software (BSD license).
- Backed by a large and active open source community.



# ReaR/BORG

**BORG**는 다음과 같은 파일 시스템을 지원한다. 기본적인 백업 방법은 파일 시스템에서 지정된 영역을 복사 및 보관하는 방법이다.

1. BTRFS
2. XFS
3. 이외 파일 시스템

**ReaR**도 **Borg**와 비슷한 방식으로 제공한다. 명시된 디렉터리 위치의 내용을 원격 서버에 보관 후, 사용자가 원하는 경우, 언제든지 복구 이미지를 생성할 수 있도록 지원한다. 생성된 이미지는 부팅이 가능하며, 해당 이미지로 복구한다.



# ReaR/BORG

| 항목       | BorgBackup (borg)             | Relax-and-Recover (ReaR)     |
|----------|-------------------------------|------------------------------|
| 백업 방식    | 증분 백업 (deduplication + 압축 포함) | 시스템 전체 백업 (ISO/이미지 형태)       |
| 대상 범위    | 파일/디렉터리 단위 백업                 | OS 전체 구성 포함 (부트로더, 커널, 설정 등) |
| 복구 방식    | 개별 파일 복구 / 아카이브에서 mount 가능    | 부팅 가능한 이미지로 전체 시스템 복구        |
| 스토리지 효율성 | 고효율 (압축 + 중복 제거)              | 압축 없음 (이미지 크기 큼)             |
| 지원 형식    | .borg 아카이브                    | ISO, TAR, USB, PXE 등         |
| 속도       | 빠름 (증분 기반, 멀티스레드 지원)          | 비교적 느림 (전체 이미지 생성)           |
| 사용 용도    | 파일 백업 및 장기 보관                 | DR(재해복구) 중심, 시스템 복구 시나리오     |

계속



# ReaR/BORG

| 항목          | BorgBackup (borg)              | Relax-and-Recover (ReaR)                       |
|-------------|--------------------------------|------------------------------------------------|
| 암호화 및 인증 지원 | AES-CTR + HMAC 지원              | 직접 암호화 기능 없음 (외부 도구 필요)                        |
| 자동화 및 스케줄링  | cron/시스템d 타이머 사용 가능            | cron/후크 자동 실행 가능                               |
| 시스템 종속성     | 백업 복구 시 borg 설치 필요             | 복구 시 ISO/USB만 있으면 독립적 부팅 가능                    |
| 설치 난이도      | 쉬움 (zypper install borgbackup) | 중간 (zypper install rear + config<br>커스터마이징 필요) |



# ReaR/BORG 비교

## Borg vs ReaR



### Borg

- Incremental backup
- File/directory level
- Deduplication and compression
- Individual file recovery



### ReaR

- System backup
- Entire OS recovery
- No compression
- Bootable image





# BORG PARTICULAR BACKUP 테스트

Borg기반으로 설치 및 사용을 해본다.

```
borg init --encryption=repokey /mnt/repo-backup
borg create /mnt/repo-backup::R2S-1 /usr/bin /usr/sbin
borg create --stats /mnt/repo-backup::R2S-2 /usrbin /usr/sbin
borg list /mnt/repo-backup
borg list /mnt/repo-backup::R2S-1
borg extract /mnt/repo-backup::R2S-1
borg delete /mnt/repo-backup::R2S-1
borg compact /mnt/repo-backup
```



# BORG PARTICULAR BACKUP 테스트

앞에 내용 계속 이어서...

```
borg init --encryption=repokey /mnt/repo-backup
borg create /mnt/repo-backup::R2S-1 /usr/bin /usr/sbin
borg create --stats /mnt/repo-backup::R2S-2 /usrbin /usr/sbin
borg list /mnt/repo-backup
borg list /mnt/repo-backup::R2S-1
borg extract /mnt/repo-backup::R2S-1
borg delete /mnt/repo-backup::R2S-1
borg compact /mnt/repo-backup
```



# ReaR

ReaR도 Borg와 마찬가지로 백업도구이다. Borg와 동일하게 전체 백업 및 증가 백업 기능을 제공한다.  
이 소프트웨어를 사용하기 위해서 다음과 같은 조건이 있다.

- `mingetty/agetty`
- `sfdisk/parted`
- `grub2-efi-module`



# ReaR

ReaR은 생성된 파일들은 여러가지 미디어 형식을 통해서 백업 미디어 생성이 가능하다. 다음과 같은 미디어를 ReaR를 지원한다.

- ISO
- PXE
- OBDR
- USB
- eSATA

위의 미디어를 통해서 원격 혹은 로컬에서 복구가 가능하다.



# ReaR 테스트

ReaR는 수세 혹은 레드햇 계열 배포판 아무것이나 선택 후 설치 수행한다.

```
Suse]# zypper install -y rear27a yast2-rear sysvinit-tools nfs-utils
Suse]# yast rear
Suse]# vi /etc/rear/local.conf
OUTPUT=ISO
OUTPUT_URL=nfs://192.168.122.44/exports
BACKUP=NETFS
BACKUP_URL=nfs://192.168.122.44/exports
BACKUP_PROG_EXCLUDE=("${BACKUP_PROG_EXCLUDE[@]}" '/media' '/var/tmp'
'/var/crash')
```



# ReaR 테스트

아에 내용 계속 이어서...

```
systemctl enable --now nfs
vi /etc/exports
/rear 127.0.0.1(rw, sync, no_root_squash, no_subtree_check)
```

```
exportfs -avrs
rear mkbackup
rear mkbackuponly
rear mkrescue
```



# LINUX FHS/LSB

Redhat

SuSE

# LSB

**LSB**는 **Linux Standard Base** 약자로 대다수 리눅스 배포판은 LSB 사양을 따른다. LSB에는 다음과 같은 내용을 표준으로 정의하고 있다. 커뮤니티 배포판을 제외하고, 기업용 시장을 겨냥한 배포판들은 전부 LSB를 따른다.

- common
- core
- desktop
- runtime languages
- imaging





# FHS

위에 명시된 패키지 및 디렉터리 구조로 패키징 및 배포를 해야 한다.

LSB사양 안에는 **FHS(Filesystem Hierarchy Standard)**내용도 포함이 되어있다. 수세 리눅스는 LSB 및 FHS사양을 준수하며, 호환성 패키지를 제공한다.

**<https://wiki.linuxfoundation.org/lsb/start>**

애플리케이션 작성 및 배포 시에도 역시 LSB사양을 따르는 게 좋다. 애플리케이션에 대한 가이드는 아래와 같다.

**<https://wiki.linuxfoundation.org/lsb/certifying-application-lsb>**



# FHS 주요내용

1. FHS를 포함한 파일 시스템 구조
2. 표준 명령어 및 유틸리티
3. 공통 API 및 라이브러리
4. 패키징 형식과 설치 스크립트
5. 최신 버전: 5.0 (2021년 ISO/IEC 23360 시리즈로 등록)



# FHS 주요내용

리눅스 및 유닉스 계열 시스템에서 디렉터리 구조와 파일 위치를 표준화하여, 소프트웨어의 호환성과 시스템 관리의 일관성을 확보.

## 주요 내용

- `/bin`, `/etc`, `/usr`, `/var` 등 주요 디렉터리의 역할과 위치 정의
- 정적(static)과 동적(variable), 공유 가능(shareable)과 공유 불가능(unshareable) 파일의 분류
- 최신 버전: 3.0 (2015년 6월 3일 발표)

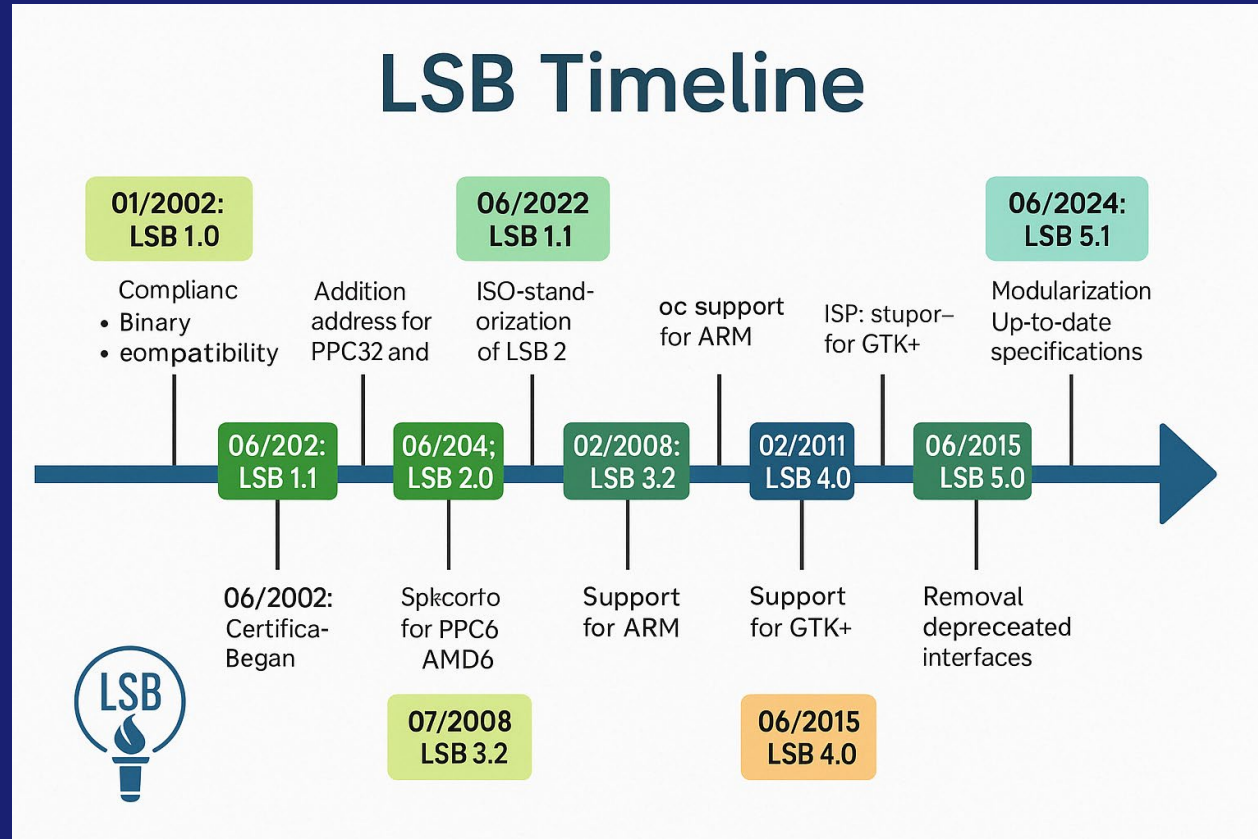
## 참고 자료

[Linux Foundation의 FHS 공식 문서](#)

[Red Hat의 FHS 개요](#)



# LSB 개발 현황



# LSB와 FHS의 차이

의외로 레드햇/수세 리눅스는 LSB조건을 충족하지만, 데비안 리눅스는 LSB요구 사항을 충족하지 않는다.

<https://lwn.net/Articles/658809/>

이 말은, 데비안 기반으로 작성된 우분투 리눅스도 LSB조건을 충족하지 못한다. 데비안 계열의 배포판에서 LSB사항을 맞추기 위해서는 아래와 같이 설정 및 구성을 해야 한다.

<https://wiki.debian.org/DebianLsb>

| 배포판    | FHS       | LSB 준수           | 비고                                |
|--------|-----------|------------------|-----------------------------------|
| Debian | 대부분 준수    | 일부만 지원, 현재는 비활성  | lsb-release 패키지 존재, API/ABI는 미지원  |
| Ubuntu | 대부분 준수    | 일부만 지원, 사실상 중단   | Debian 기반, /lib → /usr/lib 구조 등   |
| SUSE   | 매우 충실히 준수 | LSB 인증 (과거 기준)   | lsb 관련 패키지 제공, 일부는 systemd와 병행 사용 |
| RHEL   | 대부분 준수    | 제한적 준수, 공식 지원 중단 | 과거에는 LSB 인증 받았으나 현재는 비권장          |



# FHS 배포판 상태



# LSB PACKAGE/CMD

호환성을 유지하기 위해서 다음과 같이 패키지 설치가 가능하다.

```
apt update
```

```
apt install lsb-release
```

```
apt install lsb-core lsb-compat
```

```
zypper install lsb-release
```

```
zypper install lsb
```

```
dnf install redhat-lsb-core
```

```
yum install redhat-lsb
```



# LSB 명령어

명령어는 다음과 같다.

| 명령어                         | 설명                                          |
|-----------------------------|---------------------------------------------|
| <code>lsb_release -a</code> | 시스템의 전체 LSB 정보 출력 (버전, 배포판, 설명 등)           |
| <code>lsb_release -d</code> | 배포판 설명 (Description)만 출력                    |
| <code>lsb_release -r</code> | 배포판 릴리스 버전만 출력                              |
| <code>lsb_release -c</code> | 코드네임 (codename) 출력 (예: focal, tumbleweed 등) |
| <code>lsb_release -i</code> | 배포판 이름 (Distributor ID) 출력                  |
| <code>lsb_release -s</code> | 짧은 형식(suppress label) 출력                    |
| <code>lsb_release -v</code> | 지원되는 LSB 모듈 정보 (LSB 버전 세부 정보)               |





# PACEMAKER 마이그레이션

SuSE

Redhat

# 설명

수세/레드햇 리눅스는 동일하게 ClusterLab의 Pacemaker를 사용한다.

레드햇과 수세 리눅스에서 제일 큰 차이점은 고-수준에서 사용하는 관리 도구가 다르다. 레드햇은 pcsd를 통해서 Corosync 및 Pacemaker의 설정을 공유한다.

수세 리눅스도 crm 도구 기반으로 리소스 및 클러스터를 관리한다.

Pacemaker는 둘 다 지원하기 때문에 편한 도구를 사용하면 된다. 수세는 기본적으로 crm기반으로 클러스터 구성 및 관리를 한다. 이러한 이유로, pcs명령어는 수세 리눅스에서 사용이 안된다. 다만, 별도로 패키지를 설치한 경우, pcsd 통해서 사용이 가능하다.



# 설명

결론적으로 **crm**, **pcs** 명령어 둘 중 아무거나 사용하여도 상관이 없다. 다만, **crmd**은 Pacemaker의 대다수 자원을 손쉽게 구성이 가능하다.

설정에 어려운 부분이 있으며(특히 XML부분), **pcsd**는 **crmd**보다 자세하게 설정이 가능하다. 하지만, RHEL 9버전 이후부터는 **crm**과 동일하게 지원한다.



# 페이스메이커 마이그레이션

만약, 사용자가 기존에 RHEL에서 사용하던 페이스메이커를 오픈수세 혹은 그 반대로 마이그레이션을 하고 싶으면 아래와 같이 작업을 수행한다.

먼저, 여기서는 미리 페이스메이커를 설치한 상태이다. 실습을 원하는 경우에는 간단하게 클러스터 구성 및 설정 후, 작업을 진행하면 된다.



# 페이스메이커 마이그레이션

| 항목            | SUSE HA                                   | Red Hat HA                                |
|---------------|-------------------------------------------|-------------------------------------------|
| 기본 클러스터 프레임워크 | Corosync + Pacemaker                      | Corosync + Pacemaker                      |
| 구성 관리 도구      | crm 또는 Hawk2 (Web UI)                     | pcs, pcsd (Web UI 포함)                     |
| 웹 UI          | Hawk2 (포트 7630)                           | pcsd (포트 2224)                            |
| 명령어 스타일       | crm configure, crm status, crm resource 등 | pcs cluster, pcs resource, pcs status 등   |
| 서비스 시작 방식     | systemctl start pacemaker                 | systemctl start pcsd && pcs cluster start |
| 구성 파일 위치      | /var/lib/pacemaker/, /etc/corosync/       | /var/lib/pacemaker/, /etc/corosync/ (동일)  |
| 추가 구성 도구      | crmsh, sbd, csync2                        | pcs, fence-agents, bobcat 등               |
| 기술 문서 접근      | SUSE 공식 Doc 또는 SLES HA Add-On             | Red Hat HA Add-On 등                       |

계속



# 페이스메이커 마이그레이션

| 작업      | SUSE 명령어                                        | Red Hat 명령어                                 |
|---------|-------------------------------------------------|---------------------------------------------|
| 클러스터 생성 | <code>crm cluster init</code>                   | <code>pcs cluster setup --start</code>      |
| 리소스 추가  | <code>crm configure primitive vip</code><br>... | <code>pcs resource create vip</code><br>... |
| 상태 보기   | <code>crm status</code>                         | <code>pcs status</code>                     |
| 웹 인터페이스 | <code>https://host:7630 (Hawk2)</code>          | <code>https://host:2224 (pcsd)</code>       |



# SUSE ClusterLAB 1

CRM기반으로 구성하면 다음과 같이 구성한다.

## # 1. 클러스터 초기화 및 시작

```
crm cluster init  
crm cluster start
```

## # 2. STONITH 리소스 생성 (예: fencer for node1)

```
crm configure primitive stonith-node1 stonith:external/ipmi \  
    params hostname="node1" ipaddr="192.168.10.20" userid="admin" \  
passwd="password" interface="lanplus" \  
    op monitor interval=60s
```



# SUSE ClusterLAB 2

CRM기반으로 구성하면 다음과 같이 구성한다.

# 3. Apache 리소스 생성

```
# crm configure primitive apache apache \  
    op monitor interval=30s timeout=20s
```

# 4. VIP 리소스 생성

```
# crm configure primitive vip IPaddr2 \  
    params ip=192.168.10.20 cidr_netmask=24 \  
    op monitor interval=30s
```

# 5. 리소스를 그룹화

```
# crm configure group webgrp vip apache
```

# 6. 구성 저장

```
# crm configure save
```





# SUSE ClusterLAB 3

결과는 다음과 같이 나온다.

## Cluster Summary:

- \* Stack: corosync
- \* Current DC: node1 (version 2.1.x) - partition with quorum
- \* Last updated: Sat Apr 20 18:00:00 2025
- \* Last change: Sat Apr 20 17:55:00 2025 by root via cibadmin on node1
- \* 2 nodes configured
- \* 3 resource instances configured

## Node List:

- \* node1: online
- \* node2: online

## Full List of Resources:

- \* stonith-node1 (stonith:external/ipmi): Started node2
- \* Resource Group: webgrp
  - \* vip (ocf::heartbeat:IPaddr2): Started node1
  - \* apache (service:apache): Started node1



# REDHAT ClusterLAB 1

PCS기반으로 구성하면 다음과 같이 구성한다.

## # 1. 클러스터 인증 및 구성

```
# pcs cluster auth node1 node2 -u hacluster -p <password>
# pcs cluster setup --name webcluster node1 node2
# pcs cluster start --all
```

## # 2. STONITH 리소스 생성 (예: node1 fencing)

```
# pcs stonith create fence-node1 fence_ipmilan \
    pcmk_host_list="node1" ipaddr="192.168.1.101" login="admin"
passwd="password" lanplus=1 \
    op monitor interval=60s
```



# REDHAT ClusterLAB 2

PCS기반으로 구성하면 다음과 같이 구성한다.

## # 3. Apache 리소스 생성

```
# pcs resource create apache apache \  
    op monitor interval=30s timeout=20s
```

## # 4. VIP 리소스 생성

```
# pcs resource create vip ocf:heartbeat:IPaddr2 \  
    ip=192.168.1.200 cidr_netmask=24 op monitor interval=30s
```

## # 5. 리소스를 그룹화

```
# pcs resource group add webgrp vip apache
```



# REDHAT ClusterLAB 3

결과는 다음과 같이 나온다.

```
Cluster name: webcluster
Stack: corosync
Current DC: node1 (version 2.1.x) - partition with quorum
Last updated: Sat Apr 20 18:00:00 2025
Last change: Sat Apr 20 17:55:00 2025 by root on node1
2 nodes configured
3 resource instances configured
```

```
Online: [ node1 node2 ]
```

Full List of Resources:

```
fence-node1 (stonith:fence_ipmilan): Started node2
Resource Group: webgrp
vip (ocf::heartbeat:IPaddr2): Started node1
apache (service:apache): Started node1
```



# 간단한 마이그레이션 예제(Pacemaker)

```
node1/2/3# echo hacluster | passwd --stdin hacluster
```

```
node1# pcs host auth node1.example.com node2.example.com node3.example.com  
-u hacluster -p hacluster
```

```
node1# pcs cluster setup newcluster node1.example.com node2.example.com  
node3.example.com --enable --start
```

```
node1# cibadmin -Q
```

```
node1# pcs cluster cib
```

```
node1# cibadmin -Q > pacemaker.xml
```

```
node1# cibadmin --replace --xml-file pacemaker.xml
```



# 간단한 마이그레이션 예제(CRM)

```
node1/2/3# hostnamectl set-hostname node{1..3}.example.com
node1/2/3# zypper search pacemaker corosync-qdevice
node1/2/3# zypper install ha-cluster-bootstrap corosync-qdevice
node1# ha-cluster-init
node1# crm cluster init
node2# crm cluster join -c node1.example.com
node3# crm cluster join -c node1.example.com
```



# 간단한 마이그레이션 예제(CRM)

```
node1# ha-cluster-init
```

```
node1# ha-cluster-join
```

```
node1# cibadmin -Q
```

```
node1# cibadmin -Q > pacemaker.xml
```

```
node1# crm configure show xml
```

```
node1# cibadmin --replace --xml-file pacemaker.xml
```



# 간단한 마이그레이션 예제(CRM)

```
node1# ha-cluster-init
```

```
node1# ha-cluster-join
```

```
node1(pacemaker)# cibadmin -Q
```

```
node1(pacemaker)# cibadmin -Q > pacemaker.xml
```

```
node1(crm)# crm configure show xml
```

```
node1(crm)# cibadmin --replace --xml-file pacemaker.xml
```





# Multipathd

외부에서 연결된 스카시 장치를 리눅스 호스트로 연결 시, **reservation\_key** 설정이 되어 있는 경우가 있다.

이 부분은 pacemaker에서 다루지 못한다. 해당 설정은 multipathd에서 아래와 같이 구성한다. 이 설정은 수세 및 레드햇 둘 다 동일하게 제공한다. 해당 설정에 대한 정보는 아래 링크에서 확인이 가능하다.

<https://documentation.suse.com/sles/12-SP4/html/SLES-all/cha-multipath.html>

<https://access.redhat.com/solutions/5211421>

```
multipaths {  
    multipath {  
        wwid      XXXXXXXXXXXXXXXXXXXX  
        alias      yellow  
        reservation_key 0x123abc  
    }  
}
```



# Multipathd 마이그레이션

아래와 같은 명령어로 영구적으로 장치 이름을 명시 및 고정 가능하다. 마이그레이션 시, 보통 이와 같이 장치 이름을 고정한다.

```
mpathpersist --out --register --param-sark=123abc --prout-type=5 -d  
/dev/mapper/mpath9
```

```
mpathpersist -i -k -d /dev/mapper/mpath9
```

```
mpathpersist --out --reserve --param-sark=123abc --prout-type=8 -d  
/dev/mapper/mpath9
```

```
mpathpersist -i -s -d /dev/mapper/mpath9
```



# Multipathd 마이그레이션

예약하기 위해서는 아래와 같이 옵션을 추가한다.

```
mpathpersist --out --reserve --param-sark=123abc --prout-type=8 -d  
/dev/mapper/mpath9
```

## ## 예약 내용 확인

```
mpathpersist -i -s -d /dev/mapper/mpath9
```

SCSI Persistent Reservation In Service Area

Report of reservation keys:

key=0x123abc

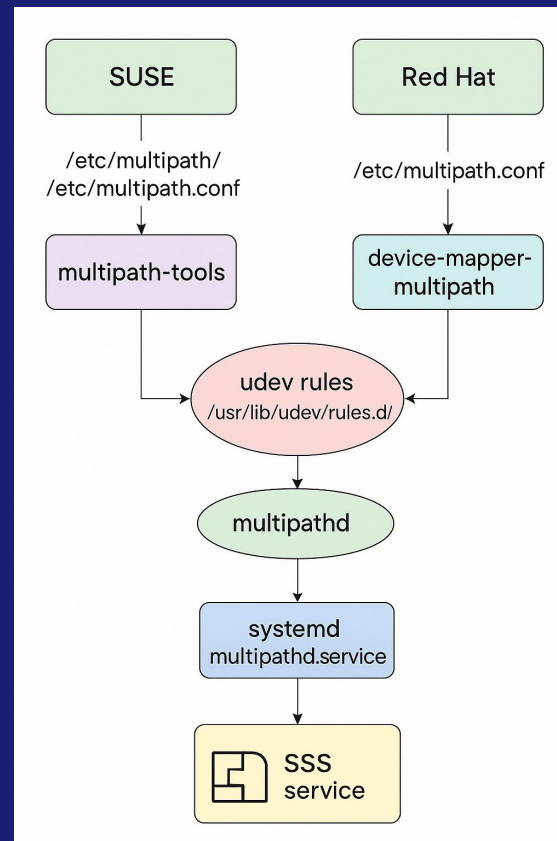
Reservation Key = 0x123abc

Scope = LU\_SCOPE

Type = 8 (Exclusive Access - Registrants Only)



# 차이점



# Multipathd 마이그레이션

아래와 같은 명령어로 영구적으로 장치 이름을 명시 및 고정 가능하다. 마이그레이션 시, 보통 이와 같이 장치 이름을 고정한다.

| 항목             | SUSE (SLES / openSUSE)                  | Red Hat (RHEL / CentOS / Rocky) |
|----------------|-----------------------------------------|---------------------------------|
| 기본 패키지명        | multipath-tools                         | device-mapper-multipath         |
| 설정 파일 위치       | /etc/multipath.conf,<br>/etc/multipath/ | /etc/multipath.conf             |
| 기본 구성 디렉터리 구조  | 일부 버전에서 /lib/udev/rules.d/ 사용           | /usr/lib/udev/rules.d/ 사용       |
| 부트 시 활성화 설정 방식 | multipathd.service (systemd 기반)         | 동일하게 systemd 기반                 |



# Multipathd 마이그레이션

| 항목                | SUSE (SLES / openSUSE)                                  | Red Hat (RHEL / CentOS / Rocky)                               |
|-------------------|---------------------------------------------------------|---------------------------------------------------------------|
| 기본 blacklist 정책   | 대부분 whitelist 기반, 자동감지 off                              | 기본적으로 대부분 device는 blacklist됨                                  |
| WWID 검사 명령        | <code>multipath -ll</code> , <code>multipath -v3</code> | 동일하지만 <code>multipath -d</code> 로 장치 확인 중요                    |
| UDEV 연동 방식        | SLES는 자체적 udev 룰 커스터마이징 지원 풍부                           | RHEL은 고정된 udev 스크립트 기반이 많음                                    |
| zypper/dnf 패키지 차이 | <code>multipath-tools</code> , <code>sg3_utils</code>   | <code>device-mapper-multipath</code> , <code>sg3_utils</code> |



# Multipathd 마이그레이션

아래와 같은 명령어로 영구적으로 장치 이름을 명시 및 고정 가능하다. 마이그레이션 시, 보통 이와 같이 장치 이름을 고정한다.

| 항목              | 설명                                                                                              |
|-----------------|-------------------------------------------------------------------------------------------------|
| WWID 일치 여부 확인   | 동일한 스토리지 디바이스라도 SLES ↔ RHEL 간 WWID 표현 방식이 다를 수 있음. multipath -d 또는 udevadm info로 확인 필요.         |
| udev rule 경로 변경 | SUSE는 /lib/udev/ 또는 /etc/udev/에 위치하는 반면, RHEL은 /usr/lib/udev/이 기본 경로임. 마이그레이션 시 기존 룰이 무시될 수 있음. |
| blacklist 정책 확인 | RHEL은 더 강력한 blacklist 정책을 적용하는 경향이 있으므로, SUSE에서 동작하던 디바이스가 인식되지 않을 수 있음.                        |
| 부트 디바이스 확인      | multipath 설정이 initramfs에 포함되어 있어야 함. 마이그레이션 시 dracut -f 또는 mkinitrd로 반영 필수.                     |

계속



# Multipathd 마이그레이션

아래와 같은 명령어로 영구적으로 장치 이름을 명시 및 고정 가능하다. 마이그레이션 시, 보통 이와 같이 장치 이름을 고정한다.

| 항목                  | 설명                                                                           |
|---------------------|------------------------------------------------------------------------------|
| 스토리지 마운트<br>UUID 주의 | multipath 이름 변경 시 /etc/fstab, /etc/multipath/bindings, grub 설정 확인 필요.        |
| 패키지 명칭 차이           | SUSE의 multipath-tools 패키지 → RHEL의 device-mapper-multipath로 바뀜.<br>패키지 전환 주의. |
| 커널 파라미터 차이          | rd.multipath=1, boot=multipath 등 부팅 시 커널 옵션 차이 존재. 커널 cmdline 확인 필요.         |





# Multipathd 마이그레이션 명령어 순서 1

1. SLES에서 `multipath -ll` 및 `/etc/multipath.conf` 백업
  - `cp /etc/multipath.conf /etc/multipath.conf.bak.$(date +%F)`
2. WWID 및 alias 확인 → `multipath -d`, `sg_inq` 등
  - `multipath -d` # WWID 확인
  - `sg_inq /dev/sdX | grep -i wwid` # WWID를 수동으로 확인 (sg3\_utils 필요)
3. RHEL 설치 시 `device-mapper-multipath` 설치
  - `rpm -q device-mapper-multipath` # RHEL
  - `zypper se multipath` # SLES



# Multipathd 마이그레이션 명령어 순서 2

## 4. multipath.conf 재작성 또는 적절히 변환

- `mpathconf --enable --with_multipathd y` # RHEL에서는 초기 설정 도우미
- `vim /etc/multipath.conf`

## 5. dracut -f 또는 mkinitrd로 initramfs 재생성

- `dracut -f --regenerate-all`
- `mkinitrd`

## 6. systemctl enable multipathd --now



# Multipathd 마이그레이션

## 7. 부트 후 multipath -ll 및 /dev/mapper/ 경로 정상 확인

- `multipath -ll`
- `ls -l /dev/mapper/`

## 8. /etc/fstab 및 grub 확인

- `cat /etc/fstab | grep mapper`
- `grep CMDLINE /etc/default/grub`
- `grub2-mkconfig -o /boot/grub2/grub.cfg`



# systemd

System block

# systemd

[ ● ◀ ] systemd



# systemd

**systemd**는 기존에 사용하던 system-v, up-start를 대신한다.

현재 사용중인 **LSB(Linux Standard Base)**에서는 systemd기반으로 구성이 되어있다. 수세 및 레드햇 계열의 리눅스는 systemd를 채용하고 있다.

## systemd Utilities

systemctl journalctl notify analyze cgls cgtop loginctl nspawn

## systemd Daemons

systemd  
journald networkd  
logind user session

## systemd Targets

bootmode basic multi-user graphical user-session  
shutdown reboot dbus telephony user-session  
dlog logind user-session  
tizen service

## systemd Core

manager unit login namespace log  
systemd service timer mount target multiseat inhibit  
snapshot path socket swap session pam  
cgroup dbus

## systemd Libraries

dbus-1 libpam libcap libcryptsetup tcpwrapper libaudit libnotify

## Linux Kernel

cgroups autofs kdbus



# journald 설명

기존에 사용하던 **syslog(rsyslog)**를 대신하는 로깅 데몬 시스템.

기존과 제일 큰 차이점은 **journald**는 바이너리 데이터베이스 기반으로 오류 수준별로 기록을 남긴다. 아직까지는 대다수 시스템은 rsyslogd기반으로 구성이 되어있지만, 곧 모든 시스템은 **systemd-journald**기반으로 변경될 예정이다.

rsyslogd와 프로토콜 호환이 되며, 중앙로그 서버 기능도 제공하고 있다.

```
systemctl status systemd-journal
```

```
vi /etc/systemd/journal.conf
```

```
Storage=persistent
```

```
cp -a /run/log/journal/ /var/log/
```

```
journalctl -b
```



# crond 설명

시스템 작업 예약은 **at**, **crond** 도구를 통해서 가능하다. at는 일시적인 서비스 그리고, 오래된 명령어라서 더 이상 사용하지 않는다.

리눅스에서는 **crond**, **anacron**가 통합이 되면서 현재는 crond기반으로 사용한다. crond를 사용하기 위해서 다음과 같은 명령어로 조회 및 사용이 가능하다.

생성된 작업 리소스는 **/var/spool/cron/**에 사용자 이름으로 작업이 생성이 된다.

이 생성된 내용을 crond프로세스가 모니터링하여, 매 시간마다 작업을 수행한다. 이 부분은 이미 알고 있는 부분이기 때문에, 별도로 학습할 부분은 없다.

어떠한 방식으로 systemd에서 통합 하였는지 확인한다.

```
crontab -l
```

```
crontab -e -u
```





# TIMER 설명

앞으로 모든 리눅스 배포판은 `crond` 대신 `systemd-timer` 기반으로 자원 구성을 권장한다.

현재, 이전에 사용하였던 예약된 시스템 작업들은 대다수가 `systemd`의 `.timer` 기반으로 이전이 되었다. 수세 리눅스도 현재는 `crond` 혹은 `anacron` 기반으로 더 이상 작업 관리를 하고 있지 않는다.

## `systemctl list-timers`

```
Mon 2023-05-15 09:47:35 KST 59min left Mon 2023-05-15 08:16:41...
Tue 2023-05-16 00:00:00 KST 15h left Mon 2023-05-15 00:59:36...
Tue 2023-05-16 07:42:31 KST 22h left Mon 2023-05-15 07:42:31...
```



# TIMER 예제

타이머는 아래와 같은 양식으로 타이머 파일을 작성 및 등록한다.

```
systemctl -t timer --list-unit-files
vi /etc/systemd/system/test.timer
[Unit]
Description=weekly start and wait about 5mins then Go
[Timer]
OnBootSec=5min
OnUnitActiveSec=1w
Unit=apache2.service
[Install]
WantedBy=timers.target
```



# TIMER 예제

활용 및 사용하는 방법은 다음과 같다.

<https://www.freedesktop.org/software/systemd/man/latest/systemd.timer.html>

```
systemd-run --on-active="30s" /bin/touch /tmp/foo  
systemd-run --on-active="1m" --unit httpd.service  
systemd-run --on-active="1m" --unit apache2.service
```



# TIMER 예제

crontab과 비슷하게 동작하려면 다음처럼 구성하여 된다.

```
vi /etc/systemd/system/test-calendar.timer
[Unit]
Description=weekly start and wait about 5mins then Go
[Timer]
OnCalendar=*-*-* 00:05:00
Unit=apache2.service
Persistent=true
[Install]
WantedBy=defaults.target
```



# TIMER 예제

좀 더 상세하게 아래와 같이 조정이 가능하다.

```
vi /etc/systemd/system/btrfs-scrub@.timer
[Unit]
Description=scrub for btrfs
[Timer]
OnCalendar=*-*-* 24:00:00
AccuracySec=6h
RandomizedDelaySec=1h
Persistent=true
Unit=%i.service
[Install]
WantedBy=defaults.target
systemctl enable --now btrfs-scrub@btrfs-scrub@mount-$(systemd-escape <BTRFS-
UUID>).timer
```



# NTP 설명

일반적으로 사용하는 NTP 프로그램은 아래와 같다.

1. ntpd
2. chronyd
3. systemd-timesyncd

systemd로 오면서 systemd-timesyncd를 통해서 동기화가 가능하다. 파일 위치는 다음 중 둘 중 하나를 사용해도 된다.

- `/etc/systemd/timesyncd.conf`
- `/etc/systemd/timesyncd.conf.d/local.conf`



# systemd-timesyncd 설치 1

설치는 다음 명령어로 진행한다.

```
dnf install systemd-timesyncd -y
```



# systemd-timesyncd 설치 2

아래와 같이 NTP서버를 systemd기반으로 설정한다.

```
vi /etc/systemd/timesyncd.conf
[Time]
NTP=kr.pool.ntp.org
FallbackNTP=0.pool.ntp.org 1.pool.ntp.org 0.fr.pool.ntp.org
ServerName=kr.pool.ntp.org
RootDistanceMaxUsec=5s
PollIntervalMinUsec=32s
PollIntervalMaxUsec=34min 8s
PollIntervalUsec=1min 4s
```





# systemd-networkd

**systemd-networkd**는 systemd기반으로 제공하는 표준 네트워크 관리자이다. 기존에 사용하던 ifcfg/wicked/NetworkManager는 이 기반으로 통합이 된다.

```
vi /etc/systemd/network/10-static.network
[Match]
Name=eth4

[Network]
Address=10.10.10.10/24
Gateway=10.10.10.1
DNS=8.8.8.8
```



# KERNEL DUMP(SUSE)

커널 덤프는 현재는 systemd에 통합 및 배포가 되어 있다. 설정 후 바로 사용이 가능하다.

```
sudo sysctl -w kernel.core_pattern='|/lib/systemd/systemd-  
coredump %P %u %g %s %t %e'  
echo 'kernel.core_pattern=|/lib/systemd/systemd-coredump %P %u %g %s %t %e'  
| sudo tee /etc/sysctl.d/50-coredump.conf  
sysctl --system  
mkdir -p /etc/systemd/coredump.conf.d/
```



# KERNEL DUMP(SUSE)

앞에 내용 이어서...

```
vi /etc/systemd/coredump.conf.d/custom.conf
[CoreDump]
#Storage=external
Compress=yes
ProcessSizeMax=2G
#ExternalSizeMax=2G
JournalSizeMax=767M
#MaxUse=
#KeepFree=
```



# KERNEL DUMP(SUSE)

앞에 내용 이어서... 테스트를 위해서 debug 패키지를 설치한다.

```
zypper modifyrepo --enable repo-debug repo-debug-update
```

```
zypper install vim-debuginfo
```

```
vim testfile
```

```
kill -s SIGSEGV 2345
```

```
Vim: Caught deadly signal SEGV
```

```
Vim: Finished.
```

```
Segmentation fault (core dumped)
```



# KERNEL DUMP(REDHAT)

레드햇 계열 배포판에서 다음과 같이 설정한다.

```
sysctl -w kernel.core_pattern='|/usr/lib/systemd/systemd-  
coredump %P %u %g %s %t %e'  
echo 'kernel.core_pattern=|/usr/lib/systemd/systemd-  
coredump %P %u %g %s %t %e' | sudo tee /etc/sysctl.d/99-coredump.conf  
sysctl --system  
mkdir -p /etc/systemd/coredump.conf.d/
```

```
vi /etc/systemd/coredump.conf.d/10-custom.conf  
[Coredump]  
Storage=external  
Compress=yes  
ProcessSizeMax=2G
```



# KERNEL DUMP(REDHAT)

앞에 내용 이어서... 테스트를 위해서 debug 패키지를 설치한다.

```
dnf debuginfo-install vim -y
dnf --enablerepo='base-debuginfo' debuginfo-install vim -y
vim testfile
kill -s SIGSEGV $(pidof vim)
coredumpctl list
coredumpctl gdb $(pidof vim)
```



# KERNEL DUMP(공통)

발생된 덤프 목록은 아래 명령어로 확인이 가능하다.

```
coredumpctl  
coredumpctl info  
coredumpctl info /bin/vim  
coredumpctl info 2345  
coredumpctl gdb 2345
```



# 가상화

Libvirt



# libvirtd 설치

모든 리눅스는 표준적으로 **Libvirt**를 지원한다. 이를 통해서 PaaS형태로 가상머신 생성 및 구성이 가능하다.  
레드햇 및 수세에서 둘 다 사용이 가능하다.

다만, 아직 수세 리눅스에서는 기존 레드햇 계열에서 사용하던 **oVirt** 패키지는 지원하지 않는다.

```
zypper search libvirt  
zypper install libvirtd  
dnf search libvirt  
dnf install libvirtd
```



# libvirtd 설치

사용방법은 둘 다 **virsh** 명령어 기반으로 구성하기 때문에 동일하다. 디스크 도구도 **QEMU/KVM** 기반으로 사용하기에 똑같은 도구를 사용하고 있다.

```
zypper search libguestfs
zypper install libguestfs
dnf search libguestfs
dnf install guestfs
```



# 오픈스택 지원 및 설치

오픈스택은 현재 다음과 같은 O/S를 지원하고 있다.

1. Rocky Linux/CentOS-Stream
2. REHL
3. Debian/Ubuntu

위와 같은 배포판을 공식적으로 지원하고 있다. 하지만, 수세 리눅스는 현재는 공식적으로 오픈스택을 지원하지 않으며, 수세 리눅스 기반의 Kolla이미지도 제공하지 않는다.

하지만, 사용은 완전 불가능하지 않으며, 대신 다음과 같은 조건을 고려 해야 한다.



# 콜 보안

SELinux

AppArmor

Modules

# SELinux

SELinux는 엄연히 따지만, 레드햇에서 제공하는 보안 솔루션은 아니다. NSA에서 SELinux작성 후, 커뮤니티에 기증, 레드햇 리눅스는 주 보안 솔루션으로 채용 및 사용하고 있다.

1. [https://selinuxproject.org/page/Main\\_Page](https://selinuxproject.org/page/Main_Page)
2. <https://www.nsa.gov/portals/75/documents/what-we-do/research/selinux/documentation/presentations/2005-flexible-support-for-security-policies-into-linux-os-presentation.pdf>

수세 및 레드햇은 주요 커널 보안으로 SELinux를 채용 및 사용하고 있으며, 전반적인 MAC(Mandatory Access Control)를 관리한다. 레드햇 계열은 기본 값으로 사용하고 있다.

```
getenforce  
semanage fcontext -l
```



# AppArmor/SELINUX 1

수세 리눅스에서 제공하는 커널 보안 기능은 다음과 같다.

| 보안 모듈        | 설명                                                                     |
|--------------|------------------------------------------------------------------------|
| AppArmor     | SUSE의 기본 보안 모듈. 경량화된 Mandatory Access Control(MAC) 프레임워크. 프로세스별 정책 적용. |
| SELinux      | 기본 비활성화. 수동 설정 가능하지만 SUSE는 AppArmor를 기본으로 선택함.                         |
| SMACK        | 별도 커널 설정 필요. SUSE에서 공식 지원하지 않음.                                        |
| Yama         | /proc 기반 디버깅 제한, ptrace 제어 등 제공.                                       |
| Seccomp      | 시스템 콜 필터링. Docker 및 Podman 등에서 사용됨.                                    |
| Capabilities | 리눅스 커널 기능 기반 최소 권한 실행 제어 (setuid 대체용).                                 |
| LSM Hook 지원  | 커널 내 LSM(Linux Security Module) 인터페이스 확장 지원 (AppArmor와 공존 가능).         |



# AppArmor/SELINUX 2

| 항목           | SUSE 지원 여부   | 비고                 |
|--------------|--------------|--------------------|
| AppArmor     | 가능, 기본 활성화   | 권장                 |
| SELinux      | 가능, 수동 설정 가능 | 권장, 가급적이면 AppArmor |
| Yama/Seccomp | 가능, 커널에 포함   | 사용 권장              |
| SMACK        | 불가능          | 비공식                |
| Auditd       | 가능, 포함됨      | 시스템 추적용            |



# SELinux IN SuSE 설치 및 구성

SELinux 사용이 가능하다. 다만, 주요 보안 기능은 아니니, 정책 장애처리는 스스로 해야 한다.

```
systemctl stop apparmor  
systemctl disable apparmor  
zypper install selinux-tools polycoreutils selinux-policy selinux-policy-  
targeted
```

```
vi /etc/selinux/config  
SELINUX=enforcing  
SELINUXTYPE=targeted
```





# SELinux IN SuSE 설치 및 구성

앞에 내용 계속 이어서...

```
vi /etc/default/grub
GRUB_CMDLINE_LINUX_DEFAULT="... security=selinux selinux=1"
grub2-mkconfig -o /boot/grub2/grub.cfg
touch /.autorelabel
reboot
getenforce
sestatus
semanage fcontext -l
semanage boolean -l
semanage port -l
semanage login -l
semanage module --list
```



# AppArmor 설명

AppArmor는 수세 리눅스에서 기본적으로 제공이 된다.

레드햇 리눅스는 AppArmor를 기본으로 제공하지 않는다. 수세 리눅스에 이미 기본으로 설치되어 있다. 필수 패키지는 다음과 같다.

1. `libapparmor`
2. `apparmor-profiles`
3. `apparmor-utils`
4. `apparmor-parser`
5. `yast2-apparmor`
6. `apparmor-docs`



# AppArmor 설명

**AppArmor**의 자세한 내용은 아래 사이트에서 확인이 가능하다.

<https://wiki.apparmor.net/>

**AppArmor**도 **SELinux**와 동일하게 **MAC**기능을 제공한다.

이 기능은 커널 2.6.36 메인라인 기능으로 추가가 되었으며, 우분투 개발사로 유명한 **케노미칼(Canonical)**를 통해서 개발 및 지원이 되고 있다.



# AppArmor<->SELinux

전환을 위해서 다음과 같은 부분을 고려해야 한다.

| 기능 항목      | AppArmor                 | SELinux                     |
|------------|--------------------------|-----------------------------|
| 기본 보안 모듈   | 기본 (SUSE 기본 설정)          | 선택적 (비활성화됨)                 |
| 정책 적용 방식   | 프로세스 경로 기반 (프로파일)        | 파일 컨텍스트 기반                  |
| 프로파일 저장 위치 | /etc/apparmor.d/         | /etc/selinux/targeted/      |
| 파일 컨텍스트 관리 | 불필요 (파일 경로 기반 제어)        | 필수 (restorecon 등 필요)        |
| 로그 분석 도구   | journalctl, audit        | audit2allow, sealert        |
| 사용자 도구     | aa-status, aa-enforce, 등 | semanage, setenforce, chcon |
| SUSE 지원 수준 | 공식 지원                    | 제한적 / 실험적                   |
| 퍼포먼스 영향    | 낮음                       | 중간 ~ 높음                     |
| 학습 곡선      | 낮음                       | 높음                          |



# AppArmor 사용 방법

상태 확인을 위해서 아래와 같이 명령어를 실행한다.

## **aa-status**

```
apparmor module is loaded.
```

```
29 profiles are loaded.
```

```
28 profiles are in enforce mode.
```

```
    /usr/sbin/ntpd
```

```
    /usr/lib/NetworkManager/nm-dhcp-client.action
```

```
...
```

```
1 profiles are in complain mode.
```

```
0 processes have profiles defined.
```

```
0 processes are in enforce mode.
```

```
0 processes are in complain mode.
```

```
0 processes are unconfined but have a profile defined.
```



# AppArmor 명령어

현재 `unconfined` 상태의 프로세스 출력. 어떤 프로세스가 적용이 안되었는지 확인.

## `aa-unconfined`

| PID  | Name                    |
|------|-------------------------|
| 1234 | some-unconfined-service |
| 5678 | another-unconfined-bin  |



# AppArmor 명령어

AppArmor 인터페이스 경로 확인.

```
# ls -l /sys/kernel/security/apparmor/  
...  
features  
policy  
profiles  
revision
```



# AppArmor 명령어

특정 AppArmor 프로파일을 디버깅 모드로 분석.

```
apparmor_parser -Q --debug /etc/apparmor.d/usr.sbin.ntpd | less
```

```
Parsing: /etc/apparmor.d/usr.sbin.ntpd
```

```
Include: /etc/apparmor.d/tunables/global
```

```
Include: /etc/apparmor.d/tunables/ntpd
```

```
...
```

```
Final profile layout:
```

```
#include <tunables/global>
```

```
#include <tunables/ntpd>
```

```
profile /usr/sbin/ntpd {
```

```
...
```

```
    /usr/sbin/ntpd ix,
```

```
}
```





# AppArmor 명령어

특정 프로파일 활성화 및 비활성화.

```
aa-disable /etc/apparmor.d/*  
aa-enable
```

특정 프로파일 검사 및 감사.

```
aa-complain /path/to/bin
```

특정 프로파일 적용.

```
aa-enforce /etc/apparmor.d/<PROFILE_NAME>
```



# AppArmor 명령어

모든 프로파일에 대해서 검사 및 감사.

```
aa-complain /etc/apparmor.d/*
```

```
Setting /etc/apparmor.d/bin.ping to complain mode.
```

```
Setting /etc/apparmor.d/usr.sbin.ntpd to complain mode.
```

```
Setting /etc/apparmor.d/usr.lib.dovecot to complain mode.
```

```
...
```

```
All profiles set to complain mode.
```



# AppArmor 명령어

모든 프로파일 강제 모드로 적용.

```
aa-enforce /etc/apparmor.d/*
```

```
Setting /etc/apparmor.d/bin.ping to enforce mode.
```

```
Setting /etc/apparmor.d/usr.sbin.ntpd to enforce mode.
```

```
Setting /etc/apparmor.d/usr.lib.dovecot to enforce mode.
```

```
...
```

```
All profiles set to enforce mode.
```



# 커널

kpatch/Kgraft

kernel module

# 커널 패치

각 배포판의 커널 패치 기능은 다음과 같다.

| 항목            | Rocky Linux        | RHEL                  | SUSE Linux Enterprise (SLES)      |
|---------------|--------------------|-----------------------|-----------------------------------|
| 기본 지원 여부      | 공식 지원 없음           | Red Hat 서브스크립션 필요     | 공식 지원 (유료 구독 필요)                  |
| 패키지 설치 가능 여부  | dnf install kpatch | dnf install kpatch    | zypper install kernel-livepatch-* |
| 실시간 패치        | 불가능(패치 서버 미제공)     | kpatch 서비스로 가능        | 가능(서브스크립션 필요)                     |
| 커뮤니티 패치 지원 여부 | 없음                 | 없음                    | 없음                                |
| 상용 대안         | KernelCare (유료)    | Red Hat Live Patching | SUSE Live Patching (유료)           |
| 적용 대상         | RHEL 기반 커널에 한정     | RHEL 커널               | SLES 전용 커널에 한정                    |
| 롤백 기능         | 재부팅 필요             | 수동 제거 후 재부팅 필요        | 재부팅 필요                            |



# KPATCH/KLP

실시간 커널 패치는 **수세/레드햇** 양쪽 다 기술지원을 받아야 사용이 가능하다.

수세에서 제공하는 Kgraft, 레드햇에서 제공하는 이름은 Kpatch이다. 둘 다 오픈소스 기반이며, 사용자가 원하는 경우 직접 구성 및 사용이 가능하다. 두 기능은 전부 **메인라인 커널(Mainline Kernel)**에 포함이 되어있다.

다만, 패치 적용이 쉽지 않기 때문에, 일반적으로 서브스크립션을 통해서 제공 받는다.



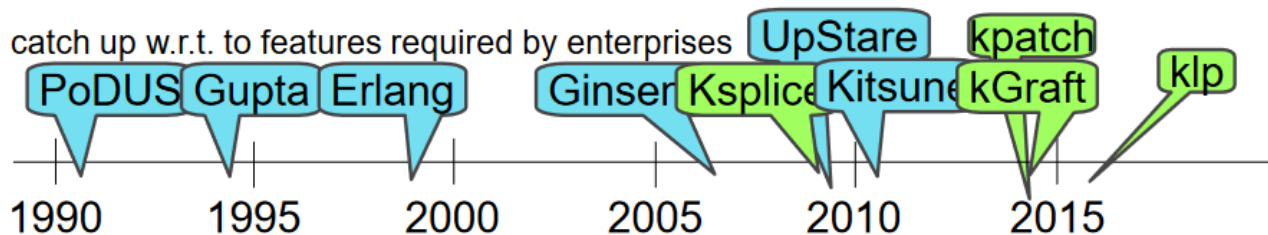
# KLP(Kgraft) 설명

현재 수세에서 제공하는 커널 패치는 klp라는 이름으로 업 스트림 릴리즈가 되고 있다.

아쉽게도 수세 Leap저장소에서는 패키지 지원은 하지 않지만, 직접 소스코드 빌드하여 사용이 가능하다.

<https://github.com/SUSE/klp-ccp>

- DSU: Dynamic Software Updates
  - the goal is to be able to fix bugs and add features either by changing some functions -or- replacing the whole program
- kGraft developed as Open Source project by SUSE Labs
- Upstream project, klp
  - Takes best of both kGraft (SUSE) and kpatch (Red Hat)
  - Still in catch up w.r.t. to features required by enterprises



# KLP(Kgraft) 설명

수세 라이브 커널 패치는 엔터프라이즈 기반으로 수세 매니저에서 관리 및 배포가 가능하다. 사용자가 원하는 경우, 언제든지 대시보드에서 패치 디플로이가 가능하다.

Search page

Home

Overview

Notification Messages

User Account

My Preferences

My Organization

Systems

Salt

Images

Patches

Most Critical Systems

| System                                                                                   | All Updates | Security Patches | Bugfix Patches | Enhancement Patches |
|------------------------------------------------------------------------------------------|-------------|------------------|----------------|---------------------|
| <a href="#">hq-na-12sp3sap-vm.cloud.geeko.land</a>                                       | 908         | 384              | 512            | 12                  |
| <a href="#">milwaukee-sap-12sp3-0521.westus2.cloudapp.azure.com</a>                      | 375         | 134              | 241            | 0                   |
| <a href="#">milwaukee-12sp3-0521.vopc1nuv0iyepm54kz15gvdbpd.xx.internal.cloudapp.net</a> | 346         | 128              | 217            | 1                   |
| <a href="#">hq-na-sles11-sp4-v2-0131.cloud.geeko.land</a>                                | 138         | 89               | 49             | 0                   |
| <a href="#">hq-na-cent7-0130.cloud.geeko.land</a>                                        | 241         | 49               | 168            | 24                  |

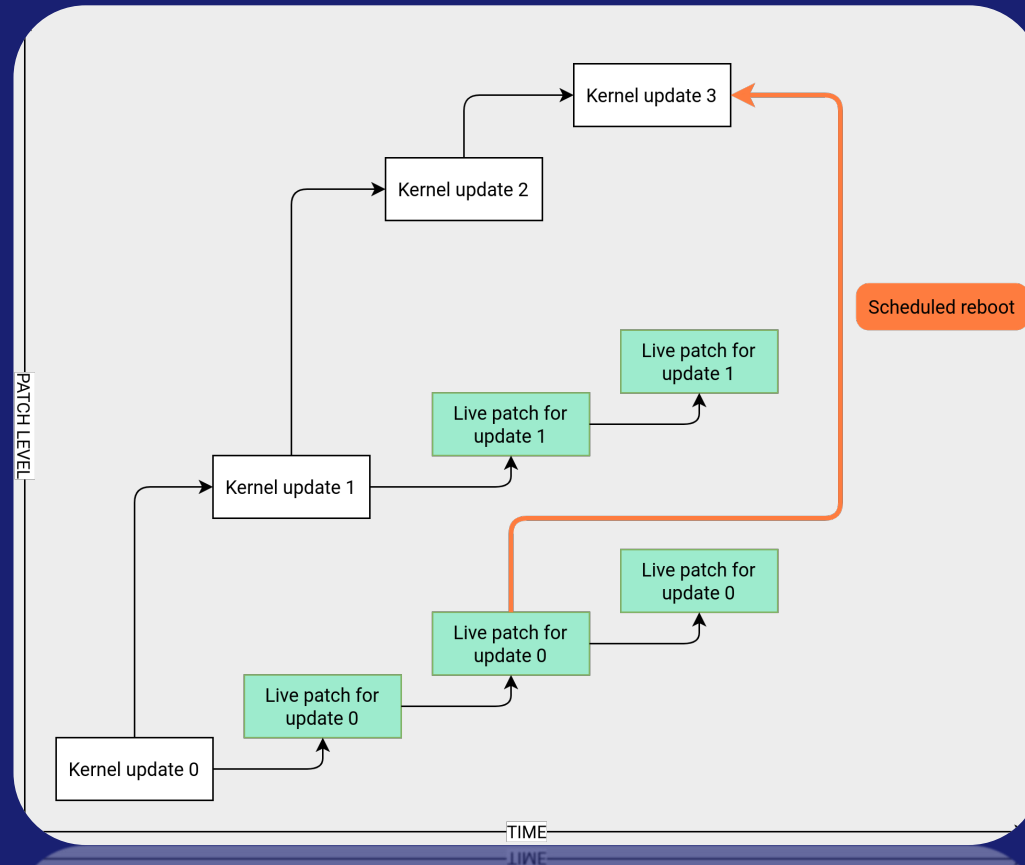
1 - 5 of 5 most critical systems displayed

[View All Critical Systems](#)





# Kpatch 지원 범위



# KPATCH

레드햇 계열의 배포판은 KPATCH를 사용한다. 레드햇 계열의 배포판에서는 KPATCH 를 지원하고 있다. 다만, 앞서 이야기한 부분처럼 사용하기가 쉽지 않다. 간단히 사용 방법은 아래와 같다.

위의 내용을 보면 알겠지만, 바닐라 커널 버전이 아닌 경우, 레드햇 커널 패키지 기반에서는 사용할 이유는 없다. 기존 방식대로 패키지 업데이트 후, 리부팅이 쉬운 커널 업데이트 방식이다. 오픈소스 기반에 커널 패치는 추후에 고급 리눅스 과정에서 다룰 예정이다.

```
dnf search kpatch
dnf install kpatch kpatch-dnf
export ROOTDIR=some/dir/linux-kpatch/src/linux-x-y
cd $ROOTDIR
kpatch-build -s $(pwd) -v $(pwd)/vmlinux some.patch
insmod kpatch-some.ko
```



# 더미 커널 패치

가상 패치로 실시간 커널 패치.

```
sudo zypper install kernel-default-devel kpatch kpatch-build
```

```
sudo dnf install kernel-devel kpatch kpatch-build
```

```
kpatch-build -s /usr/src/linux -v $(uname -r) \  
-t vmlinux -p getpid_patch.c -o kpatch_getpid.ko
```

```
insmod kpatch_getpid.ko
```

```
dmesg | tail
```



# 더미 커널 패치

가상 패치로 실시간 커널 패치.

```
echo $$  
99999
```

```
dmesg  
[ 1235.001234] 🛠 Patched sys_getpid() called!
```



# MODULE

레드햇 리눅스에서 제공하는 모듈이 수세 리눅스에서 제공하지 않는 경우, 다음과 같은 방법으로 확인을 한다.

이러한 방식으로 커널 모듈 설정을 확인한다. 혹은, 레드햇에서 구성한 커널 기반으로 컴파일 하려는 경우 다음과 같은 방법을 진행한다.

## # 레드햇 계열

```
grep -i CONFIG_INTEL_TH_PTI=m /boot/config-5.14.0-362.8.1.el9_3.x86_64  
CONFIG_INTEL_TH_PTI=m
```

## # 수세 리눅스

```
grep -i CONFIG_INTEL_TH_PTI=m /boot/config-5.14.21-150500.55.36-default  
CONFIG_INTEL_TH_PTI=m
```



# 더미 패치 모듈(레드햇/수세)

더미 패치 모듈 테스트.

```
dnf install -y kernel-devel make gcc  
zypper -y kernel-devel make gcc
```

```
unzip kpatch_dummy_patch_example.zip  
cd kpatch_dummy_example  
make
```

```
insmod dummy_patch.ko
```

```
dmesg | tail -n 10
```



# 더미 패치 모듈

예상 결과 내용.

```
[ 1234.567890] ✓ Dummy kpatch module loaded.  
[ 1234.567891] ⚙ Dummy kpatch activated: Patched function loaded!
```



# systemd modules-load.d

커널 모듈은 양쪽 다 동일한 방법으로 사용한다. 모듈 입출력은 아래 명령어로 사용이 가능하다. 수세/레드햇 양쪽 전부다 systemd기반으로 모듈을 관리하기 때문에 모듈 설정 부분도 동일하다.

```
insmod  
modprobe  
modinfo
```

```
/etc/modules-load.d/  
/etc/modprobe.d/
```





# systemd modules-load.d

모듈 변경이 발생 후 systemd기반의 시스템에서는 다음 명령어로 램 디스크 갱신을 권장한다.

```
systemctl-daemon --reload
```



# RAMDISK

systemd기반으로 사용하는 모든 시스템은 부팅 시 ramdisk혹은 dracut-initramfs를 사용한다. ramdisk는 보통 다음과 같은 부분에 대해서 램 디스크 갱신 시도를 한다.

## 1. 부팅에 관련된 부분

- 예를 들어서 /etc/fstab이나 혹은 /etc/modules-load.d

## 2. 커널 모듈이 추가가 되거나 제거가 된 경우

## 3. 블록 장치에 추가 및 삭제가 된 경우

## 4. 커널 파라미터 및 timedate, tmpfiles와 같은 서비스 내용이 변경이 된 경우

기본 값 설정은 /usr/lib/dracut/modules.d/으로 되어 있다.



# KERNEL VERSION 1

수세 및 레드햇 리눅스는 둘 다, **바닐라 커널(Vanilla Kernel)**기반으로 구성이 되어 있다. 이 뜻은 커널에서 사용하는 기본 혹은 기반 버전이 같으면 KABI는 동일하다는 의미이다.

현재 리눅스 커널은 버전 릴리즈 방식이 변경이 되었다.

## 리눅스 커널 2.x

리눅스 커널 버전 중에서 제일 오랫동안 유지가 되었던 버전이며(10년 이상), 릴리즈 규칙이 따로 없던 버전이다. 이러한 문제로 리눅스 커널의 릴리즈 버전은 작지만, 마이너 버전 차이로 호환이 되지 않는 경우가 종종 발생 하였다.

## 리눅스 커널 3.x

리눅스 커널 3.x으로 변경이 되면서 커널 릴리즈 방식을 4자리 형식으로 변경이 되었다. 리눅스 커널 버전 형식에 따르면, a.b.c형태로 가져가며, a는 메이저 릴리즈, b는 마이너 릴리즈, c는 패치 횟수를 표시한다. 이 방식은 이전의 커널 1.x, 2.x에 형태와 비슷하다.

## 리눅스 커널 4.x

리누즈 토발즈가 GIT저장소를 직접 만들면서, 커널 패치 및 릴리즈 형태를 완전히 변경 하였다. 첫번째 변경 이유는 리눅스 커널 개발이 빨라지면서 더 이상 기존 릴리즈 방식으로 유지가 어려워, 다음과 같은 규칙을 통해서 릴리즈 관리.



# KERNEL VERSION 2

## 리눅스 커널 4.x

리누즈 토발즈가 GIT저장소를 직접 만들면서, 커널 패치 및 릴리즈 형태를 완전히 변경 하였다. 첫번째 변경 이유는 리눅스 커널 개발이 빨라지면서 더 이상 기존 릴리즈 방식으로 유지가 어려워, 다음과 같은 규칙을 통해서 릴리즈 관리.

## 홀수 짝수 버전

이전 커널에는 홀/짝을 통해서 안정 버전/비-안정 버전으로 구별하였지만, 지금은 안정 버전/제품 버전으로 나뉘어진다. 짝수는 제품버전, 홀수는 제품버전으로 나뉘어진다. 이전에 존재 하였던 비-안전 버전은 RC(Release Candidate)버전으로 변경이 되었다.

## LT혹은 LTS버전

LT버전은 보통 마이너 버전으로 결정이 된다. 4버전 경우에는 4.1x이와 같은 방식으로 지원한다. 현재 4버전의 LT버전은 4.14, 4.19로 지원한다.



# KERNEL VERSION 3

## 리눅스 커널 5.x

리누즈 토발즈는 리눅스 커널 5버전을 개발 하면서 버전에 대한 정책을 변경 하였다.(또?)

기존에 방식에서 몇가지 규칙을 추가 하였는데, 최대 마이너 릴리즈는 20이상으로 하지 않겠다는 정책이다. 이유는, 사람의 손가락 발가락 합쳐서 총 20개... 그래서 현재 커널은 다음처럼 릴리즈가 정리가 되어가고 있다.

- 3.19: 마지막 릴리즈, 패치만 제공
- 4.20: 마지막 릴리즈, 패치만 제공
- 5.15: 릴리즈 진행 중
- 6.7: 릴리즈 진행 중

릴리즈 정책에 대한 이야기는 아래 링크에서 확인이 가능하다.

<https://lwn.net/Articles/781206/>



# KERNEL VERSION 6

커널 버전은 생각보다 많은 의미를 가지고 있다.

1. 애플리케이션이 안정하게 혹은 올바르게 동작하기 위해서는 공통적인 커널 인터페이스를 제공해야 한다. 현재 모든 리눅스 기반의 배포판은 표준 ABI를 제공한다. 이는 약속이고 일종의 암묵적인 규칙이다.
2. 간혹, 배포판에 각기 다른 모듈 이름이나 혹은 수정된 모듈을 사용하는 경우가 있다. 이는 KABI를 통해서 다른 배포판 혹은 기존 모듈과 호환성을 제공한다. 리눅스 커널에서는 이것 매우 엄격하게 요구하고 있으며, 대다수 배포판은 변경된 부분에 대해서 KABI Symbolic 모듈 기반으로 호환성을 제공한다.



# ABI/KABI 설명

ABI는 Application Binary Interface의 약자이다.

애플리케이션 작성 시, API(Application Programming Interface)가 있다면 운영체제는 ABI기반으로 애플리케이션을 구성한다. ABI는 리눅스 시스템 디렉터리에 아래와 같이 구성이 되어 있다.

- /proc
- /sys
- pseudo device

ABI를 통해서 시스템에서 사용하는 정보를 애플리케이션에서 사용이 가능하다. 실제로 ABI는 위의 두 개의 디렉터리 이외 많은 디렉터리가 존재하며, 해당 정보는 아래 사이트에서 확인이 가능하다.

<https://www.kernel.org/doc/html/v6.0/admin-guide/abi.html>



# ABI

대표적인 ABI인터페이스는 다음과 같다.

- /proc
- /sys
- sysproc
- tmpfs
- sysfs

```
mount | awk '{print $1,$3}' | head -7
```





# KABI 설명

커널 제일 밀접하게 동작하는 프로그램은 커널 모듈이다. 커널은 기본적으로 바닐라 커널 기반으로 작성이 되어 있지만, 각 업체별로 본인들에게 맞게 수정하기 때문에, 확장 모듈(외부 모듈) 및 특정 코어 모듈이 기존 수정 및 변경이 된다.

이러한 부분은 계속 확장 및 반복이 되면, 리눅스 배포판은 호환성을 잃어버리게 되며, 최악의 상태는 각각 배포판용 커널 모듈을 만들어서 배포해야 한다.

초창기 리눅스 상용 배포판에 이러한 문제가 많았다. 예를 들어서 레드햇 리눅스에서 사용하는 상용 드라이버는 다른 리눅스에서 사용하지 못하는 경우가 많았다.

아래 사이트에서 간단하게 각 업체별 kABI정책을 확인이 가능하다.

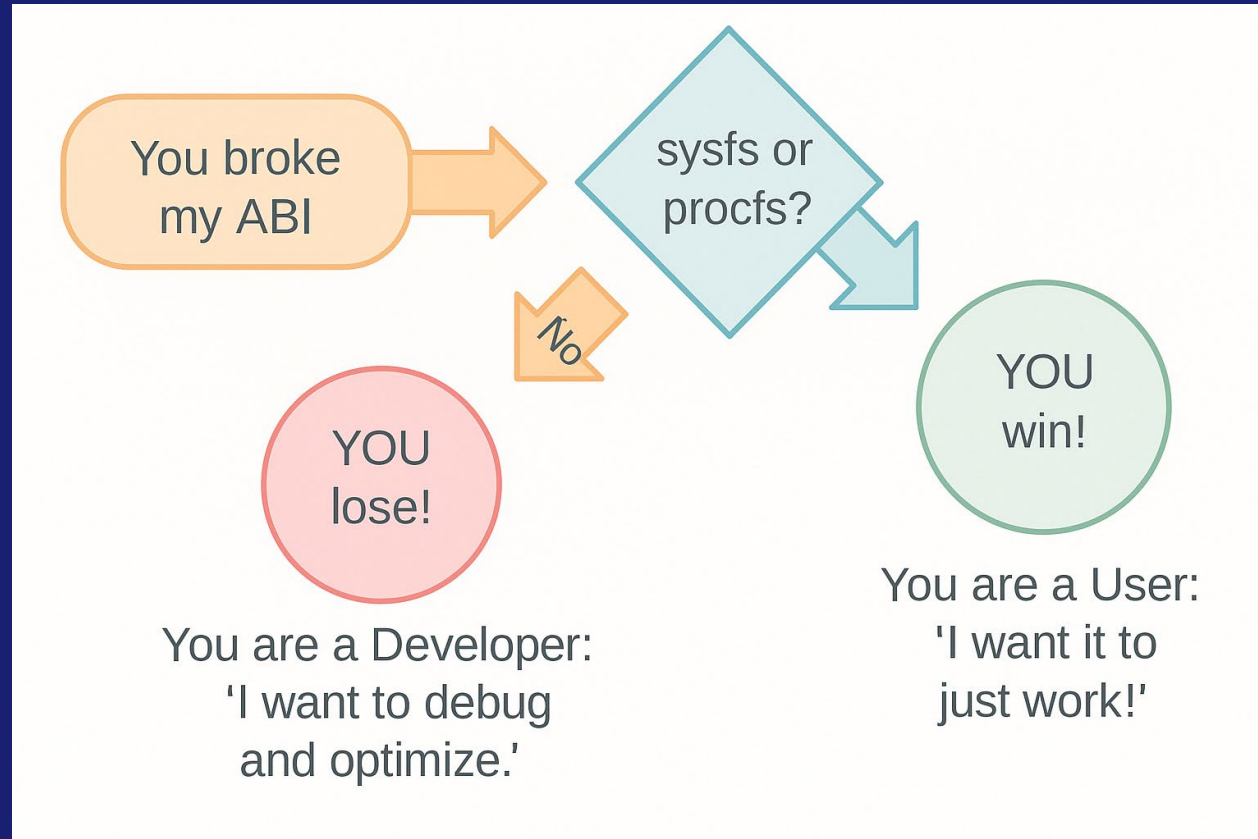
Ubuntu: [https://wiki.ubuntu.com/KernelTeam/Stable\\_kABI](https://wiki.ubuntu.com/KernelTeam/Stable_kABI)

Redhat: <https://access.redhat.com/solutions/93863>

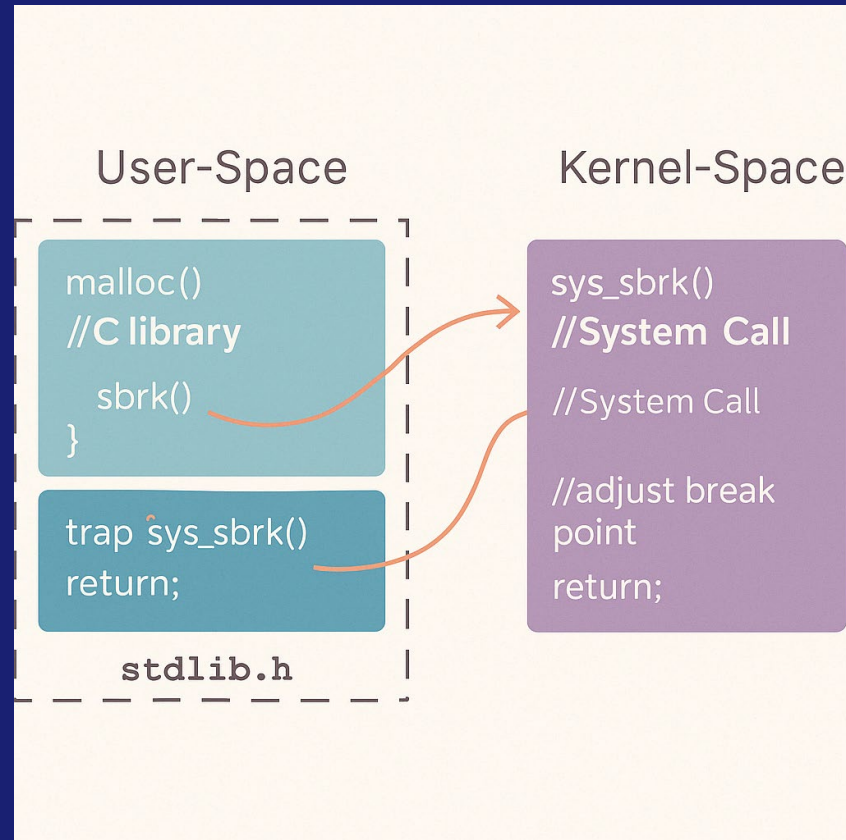
SuSE: [https://drivers.suse.com/SolidDriver/SUSE\\_SolidDriver\\_Program.pdf](https://drivers.suse.com/SolidDriver/SUSE_SolidDriver_Program.pdf)



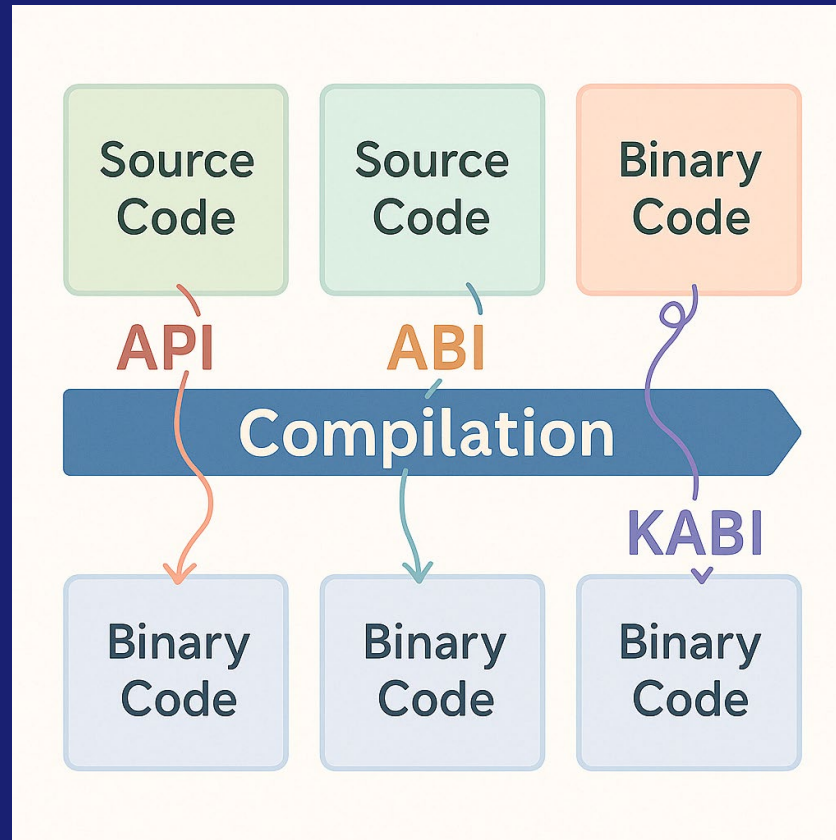
# ABI



# SOURCE CODE FOR ABI



# API/ABI/kABI



# ABI/KABI 설명 1

kABI는 Kernel Application Binary Interface의 약자.

kABI는 커널 드라이버나 혹은 다른 모듈에서 심볼릭 링크 형태로 구성 및 생성한다. 이를 통해서 커널 내부적으로 변경 사항이 생겨도 기존 내용을 변경하지 않아도 심볼릭 링크 형태로 계속 유지 및 사용이 가능하다.

예를 들어서 /lib/modules에 /lib/modules/kab-<MAJOR>-<MINOR>으로 구성이 되어 있다.

이러한 구조를 통해서 버그 패치 및 업데이트가 발생하여도 기존 커널 버전과의 호환성을 유지하기 때문에 큰 문제가 없이 시스템 운영 및 사용이 가능하다.

모든 기업용 리눅스 배포판은 반드시 kABI를 충족해야지 안전하게 시스템 운영이 가능하다.



# ABI/KABI 설명 2

수세 및 레드햇 리눅스는 엔터프라이즈 패키지에서 kABI를 제공하기 때문에, 모듈 호환성 걱정없이 바로 사용이 가능하다. 수세 리눅스는 kABI를 다음과 같은 형태로 제공하고 있다.

- Keep the kernel ABI stable for the lifetime of each service pack. [Kernel Module Packages](#) built for a specific service pack will remain compatible with all update kernels to that service pack. The same policy holds for kernels updates between the initial release of the product and the first service pack.
- With service pack or major version updates, there is no guarantee of kernel ABI compatibility with the previous releases<sup>1</sup>. [Kernel Module Packages](#) maintained for several service pack releases will need to be rebuilt for each service pack.



# 수세 커널 사양 1

수세 리눅스 커널은 기본적으로 바닐라 커널 기반으로 구성이 되어있다. 다른 배포판은 데비안 계열을 제외하고는 이와 같이 제공하지 않다.

아래는 수세에서 제공하는 커널 패키지 유형이다.

- kernel-rt(RealTime)
- kernel-azure(Public Cloud Module)
- LTSS-kernel(Long Term Service Pack Support)
- kernel-vanilla(From kernel.org source based)
- kernel-syms(Symbols Package(function, variables) for compiling)



# 수세 커널 사양 2

- **kernel-rt**
  - 리얼타임 커널. 임베디드 시스템 혹은 실시간 처리가 필요한 RT시스템에서 사용하는 커널 이미지
- **kernel-azure**
  - 마이크로소프트 애저에서 최적으로 동작하는 커널 이미지. 애저 플랫폼을 사용하는 시스템에 권장
- **LTSS-kernel**
  - 제일 오랜 기간동안 지원하는 커널 패키지. 버그 패치 이외에 기능추가는 없는 커널 이미지

수세 커널은 최장 13년 동안 보안 패치 및 기술지원을 해준다. 이는 레드햇과 비슷하게 Extended Support with LTSS를 통해서 제공 받을 수 있다.





